Sohag University

Faculty of Engineering

Electronics and Communication Engineering Department

AI-Based Intrusion Prevention System

Submitted in Partial Fulfillment for the Degree of B.Sc. in Electronics and Communication Engineering

Prepared By:

Ahmed Mohamed Ismail

AL Moataz Bellah Mahmoud Mohamed

Hatem Harby Mohamed

Hossam Fathy El-Hawy

Kerolos Nader Fekry

Mohamed Hamdy Mahmoud

Mohamed Ibrahim Ragab

Mohamed Khaled Mohamed

Saleh Ali Ahmed

Aya Ali Ahmed

Israa Khalaf Mohamedin

Roaa Ahmed Mahmoud

Sara Atef Ahmed

Supervised By

Assoc. Prof. Safwat Mohamed Ramzy

Eng. Mohamed Elsagheer

Academic Year

2025 – 2026

ACKNOWLEDGEMENT

To everyone who has contributed to our graduation project, we would like to extend our sincere gratitude.

We would like to express our appreciation to **Dr. Safwat Mohamed**, our supervisor. His support, guidance, and knowledge have been crucial to the growth of our project.

Additionally, we would like to thank **Eng. Mohamed Elsagheer** for his patience, dedication, and hands-on assistance during the project's development. Your collective support has made this experience truly meaningful.

Thank you.

Abstract

This project presents the design and implementation of an AI-Based Intrusion Prevention System (IPS) that combines real-time network traffic analysis, machine learning, and automated prevention mechanisms to enhance cybersecurity defenses. Traditional Intrusion Prevention Systems primarily rely on predefined signatures and rules, making them less effective against evolving and previously unseen attack techniques. To address these limitations, the proposed system adopts a behavioral detection approach based on artificial intelligence.

The system captures network traffic using NFStream and extracts flow-based statistical features for analysis. A multimodal dual-head deep learning architecture was developed using TensorFlow and Keras to perform both multi-class attack classification and binary anomaly detection simultaneously. The model integrates network-flow statistics with application-layer payload information, enabling the detection of both network-based and web-based attacks. The training process utilized a combination of the CICIDS2017 benchmark dataset, custom-generated traffic collected in a laboratory environment, and synthetically augmented data generated through a cluster-based bootstrapping methodology with statistical perturbation.

The overall solution consists of a traffic capture module, feature extraction and preprocessing pipeline, AI detection engine, event aggregation component, prevention engine, and web-based management dashboard. The prevention subsystem automatically blocks malicious traffic using WinDivert-based packet filtering after validating attack behavior through batch-level correlation. The management platform was implemented using ASP.NET Core MVC, SQL Server, SignalR, and FastAPI, while deployment and monitoring were supported through Docker, AWS EC2, Prometheus, and Grafana.

Extensive testing was conducted using multiple attack scenarios, including DDoS, DoS-HULK, DoS-GoldenEye, Slowloris, FTP Brute Force, SSH Brute Force, Port Scanning, SQL Injection, Cross-Site Scripting (XSS), and Heartbleed attacks. Experimental results demonstrated successful detection, classification, alert generation, and automated prevention capabilities while maintaining low inference latency and real-time responsiveness. The proposed AI-Based IPS provides improved adaptability, behavioral analysis, and threat response compared to traditional signature-based intrusion prevention systems.

Table of Contents

List of Figures

List of Tables

Abbreviations

Abbreviation	Description
ACL	Access Control List
AD	Active Directory
AI	Artificial Intelligence
API	Application Programming Interface
AWS	Amazon Web Services
CICIDS2017	Canadian Institute for Cybersecurity Intrusion Detection System Dataset 2017
CI/CD	Continuous Integration / Continuous Deployment
CNN	Convolutional Neural Network
CVE	Common Vulnerabilities and Exposures
DNS	Domain Name System
DDoS	Distributed Denial of Service
DMZ	Demilitarized Zone
DPI	Deep Packet Inspection
FTP	File Transfer Protocol
GAN	Generative Adversarial Network
GMM	Gaussian Mixture Model
GNS3	Graphical Network Simulator-3
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IPS	Intrusion Prevention System
IP	Internet Protocol
JSON	JavaScript Object Notation
LAN	Local Area Network
LOIC	Low Orbit Ion Cannon

LSTM	Long Short-Term Memory
MAC	Media Access Control
MLP	Multilayer Perceptron
MVC	Model View Controller
NAS	Network Attached Storage
OSPF	Open Shortest Path First
PCAP	Packet Capture
PDF	Portable Document Format
PSH	Push Flag
RST	Reset Flag
RBAC	Role-Based Access Control
REST	Representational State Transfer
SIEM	Security Information and Event Management
SPLT	Sequence of Packet Length and Time
SQL	Structured Query Language
SQLi	SQL Injection
SSH	Secure Shell
TCP	Transmission Control Protocol
TLS	Transport Layer Security
UDP	User Datagram Protocol
URI	Uniform Resource Identifier
VLAN	Virtual Local Area Network
VPC	Virtual Private Cloud
WAN	Wide Area Network
XGBoost	Extreme Gradient Boosting
XSS	Cross-Site Scripting

Chapter 1 Introduction

1.1 Background

The rapid advancement of information technology and the widespread adoption of interconnected systems have significantly transformed the way organizations operate. Modern enterprises rely heavily on computer networks, cloud services, web applications, and digital communication channels to conduct daily business operations. While these technological developments have improved efficiency and accessibility, they have also introduced numerous cybersecurity challenges. As networks become larger and more complex, they become attractive targets for cybercriminals seeking unauthorized access, financial gain, data theft, or disruption of services.

Cyberattacks have evolved considerably over the last decade. Traditional attacks that relied on known vulnerabilities and static signatures have been supplemented by sophisticated threats such as Advanced Persistent Threats (APTs), zero-day exploits, ransomware campaigns, insider threats, and distributed denial-of-service (DDoS) attacks. These attacks are often designed to evade conventional security mechanisms and adapt to changing environments. Consequently, organizations require intelligent and adaptive security solutions capable of identifying both known and unknown threats in real time.

Intrusion Detection Systems (IDS) and Intrusion Prevention Systems (IPS) play a critical role in modern cybersecurity infrastructures. An IDS monitors network traffic and system activities to identify suspicious behavior and generate alerts when potential threats are detected. An IPS extends these capabilities by actively preventing malicious activities through automated response mechanisms such as blocking traffic, terminating connections, or updating security policies.

Traditional IPS solutions primarily rely on signature-based detection techniques. While effective against known threats, these systems struggle to identify novel attacks and often require continuous updates to maintain effectiveness. Furthermore, signature-based approaches may generate high false-positive rates, leading to alert fatigue and reduced operational efficiency for security teams.

Artificial Intelligence (AI) and Machine Learning (ML) have emerged as promising technologies for enhancing cybersecurity systems. AI-based security solutions can analyze large volumes of network traffic, identify hidden patterns, learn from historical data, and detect anomalies that may indicate malicious activity. By leveraging machine learning algorithms, IPS solutions can improve detection accuracy, reduce false positives, and adapt to emerging threats without relying solely on predefined signatures.

In recent years, the integration of Artificial Intelligence (AI), cloud computing, and DevOps practices has enabled the development of intelligent cybersecurity platforms capable of providing real-time threat detection, automated response, centralized monitoring, and scalable

deployment. These technologies offer organizations the ability to improve their security posture while reducing manual intervention and operational overhead. Therefore, combining AI-driven threat analysis with modern software and cloud technologies represents a promising approach for building next-generation intrusion prevention systems.

This project focuses on the design and implementation of an AI-Based Intrusion Prevention System (IPS) capable of monitoring network traffic, extracting relevant features, classifying traffic using machine learning models, and automatically responding to detected threats. The proposed system aims to provide real-time protection while maintaining scalability, accuracy, and adaptability in dynamic network environments.

1.2 Problem Statement

The increasing complexity of cyber threats presents significant challenges for traditional network security solutions. Existing Intrusion Prevention Systems often depend heavily on manually created signatures and predefined rules. While these approaches remain effective against previously identified attacks, they are insufficient for detecting new and evolving threats.

Several limitations characterize traditional IPS solutions:

1. Inability to detect zero-day attacks and previously unseen threats.
2. High dependence on signature databases that require frequent updates.
3. Increased false-positive rates resulting in unnecessary alerts.
4. Limited scalability in high-speed network environments.
5. Delayed response to emerging attack techniques.
6. Difficulty adapting to changing network behaviors.

Organizations face significant financial and operational losses due to cyberattacks that bypass conventional security controls. Attackers continuously develop new methods to evade detection, making static defense mechanisms increasingly ineffective.

Moreover, the exponential growth of network traffic generated by cloud computing, IoT devices, and remote work environments has increased the volume of data that security systems must process. Traditional IPS solutions often struggle to analyze such large-scale traffic efficiently while maintaining acceptable performance levels.

Therefore, there is a need for an intelligent intrusion prevention system capable of:

- Detecting known and unknown attacks.
- Learning from network behavior patterns.
- Reducing false positives.

- Providing real-time automated responses.
- Scaling to accommodate modern network environments.

In addition to detection challenges, many organizations face difficulties in managing security events across multiple systems. Traditional solutions often provide limited visibility, fragmented monitoring capabilities, and insufficient integration between detection and response mechanisms. As a result, security teams may experience delayed incident response and reduced situational awareness during cyberattacks.

This project addresses these challenges by integrating artificial intelligence techniques into intrusion prevention operations, enabling proactive and adaptive threat detection and mitigation.

1.3 Project Motivation

Cybersecurity incidents continue to increase in both frequency and sophistication. Organizations across all sectors, including healthcare, finance, education, government, and industry, face growing risks from cyberattacks. The financial impact of successful attacks can be substantial, including data breaches, service disruptions, regulatory penalties, and reputational damage.

The motivation behind this project stems from several factors.

First, traditional IPS technologies are becoming less effective against modern attack techniques. Attackers frequently modify malware signatures, employ encryption, and utilize stealth techniques to evade conventional detection methods.

Second, advances in machine learning and artificial intelligence provide new opportunities to improve network security. AI models can analyze large datasets and discover complex relationships that are difficult to identify using manual rule creation.

Third, automated threat detection and response can significantly reduce the time required to contain attacks. Human operators may require several minutes or hours to analyze security alerts, whereas AI systems can respond within seconds.

Fourth, the increasing availability of cybersecurity datasets and open-source security tools enables researchers and developers to build advanced security solutions at relatively low cost.

Fifth, organizations increasingly require centralized platforms that combine threat detection, prevention, monitoring, alert management, and reporting capabilities within a single environment. Integrating these functionalities into one platform can improve operational efficiency and simplify cybersecurity management.

Finally, the project provides valuable practical experience in cybersecurity, networking, machine learning, software engineering, and system integration. It demonstrates how interdisciplinary technologies can be combined to solve real-world security challenges.

The development of an AI-driven IPS represents an opportunity to create a more adaptive and intelligent defense mechanism capable of protecting modern network infrastructures against rapidly evolving cyber threats.

1.4 Project Objectives

The primary objective of this project is to design and implement an AI-driven Intrusion Prevention System capable of detecting and preventing malicious network activities in real time.

The specific objectives include:

General Objective

To develop an intelligent intrusion prevention system that utilizes machine learning techniques to improve threat detection and automated response capabilities.

Specific Objectives

1. Capture and monitor network traffic in real time.
2. Extract network flow features using NFStream.
3. Train and evaluate AI models using cybersecurity datasets.
4. Detect multiple categories of cyberattacks and anomalous activities.
5. Implement an automated prevention mechanism capable of blocking malicious traffic.
6. Generate real-time alerts and security notifications.
7. Store events, alerts, and system information within a centralized database.
8. Develop a web-based management dashboard for monitoring and administration.
9. Provide reporting and visualization capabilities for security events.
10. Support containerized deployment using Docker and Kubernetes technologies.
11. Integrate cloud infrastructure and monitoring services.
12. Evaluate system performance using standard machine learning and cybersecurity metrics.

1.5 Scope of the Project

This project focuses on the development of a prototype AI-based Intrusion Prevention System for real-time network security monitoring and automated attack mitigation.

Included Scope

- Network traffic monitoring.
- Flow-based traffic analysis.
- Feature extraction using NFStream.
- AI-based attack classification.
- Real-time threat detection.
- Automated prevention and blocking mechanisms.
- Event logging and alert management.

- Web-based administration dashboard.
- Database management and event storage.
- Containerized deployment using Docker.
- Cloud deployment and infrastructure integration.
- Monitoring and visualization services.
- Performance evaluation and testing.

The project is intended as a proof-of-concept demonstrating the feasibility of integrating artificial intelligence techniques into intrusion prevention systems.

Excluded Scope

- Commercial-scale deployment.
- Full Security Information and Event Management (SIEM) functionality.
- Advanced threat intelligence feeds.
- Endpoint Detection and Response (EDR) capabilities.
- Hardware-based acceleration.
- Decryption and inspection of all encrypted traffic.
- Large-scale enterprise deployment.

1.6 Project Methodology

The project follows a structured development methodology consisting of several phases.

Phase 1: Research and Analysis

A comprehensive study of cybersecurity concepts, intrusion detection systems, intrusion prevention systems, machine learning techniques, and existing security solutions was conducted.

Phase 2: Requirements Gathering

Functional and non-functional requirements were identified based on the desired capabilities of the proposed IPS.

Phase 3: Dataset Collection and Preparation

Public cybersecurity datasets were analyzed and prepared for model training. Data preprocessing activities included cleaning, normalization, feature selection, and label encoding.

Phase 4: Machine Learning Model Development

Multiple machine learning algorithms were evaluated and trained using the prepared datasets. Performance metrics were used to identify the most suitable model.

Phase 5: System Design

The architecture of the AI-driven IPS was designed, including:

- Traffic collection module
- Feature extraction module
- AI detection engine
- Prevention engine
- Logging system
- Dashboard interface

Phase 6: System Development and Integration

The system was developed through multiple integrated modules, including:

- AI Detection Module
- Cybersecurity Testing Module
- Web Application Module
- Database Module
- Dashboard and Reporting Module
- DevOps and Cloud Deployment Module

These modules were integrated into a unified intrusion prevention platform capable of real-time threat detection, prevention, monitoring, and management.

Phase 7: Testing and Evaluation

The system was tested against various attack scenarios to assess detection accuracy, response speed, and overall effectiveness.

Phase 8: Documentation

All design decisions, implementation details, testing procedures, and results were documented in this report.

1.7 Report Organization

This report is organized into seven chapters.

Chapter 1: Introduction

Provides the background, problem statement, motivation, objectives, scope, methodology, and overall organization of the project.

Chapter 2: Literature Review

Reviews previous research, existing intrusion detection and prevention systems, machine learning techniques, and related cybersecurity solutions.

Chapter 3: System Analysis and Design

Presents the system requirements, overall architecture, cybersecurity module design, AI detection architecture, web application design, database design, cloud deployment architecture, and system workflows.

Chapter 4: Implementation

Describes the technologies, tools, software components, and implementation procedures used to develop the proposed system.

Chapter 5: Testing and Evaluation

Explains the testing environment, evaluation methodology, performance metrics, and experimental procedures.

Chapter 6: Results and Discussion

Presents the obtained results, analyzes system performance, and compares the proposed solution with traditional approaches.

Chapter 7: Conclusion and Future Work

Summarizes the achievements of the project, discusses limitations, and proposes future enhancements and research directions.

Chapter 2 Literature Review

2.1 Introduction

Cybersecurity has become one of the most critical concerns in modern information systems due to the increasing reliance on digital technologies, cloud computing, web applications, and interconnected networks. Organizations across various sectors depend on information systems to manage sensitive data, support business operations, and provide essential services. Consequently, cyberattacks have become more frequent, sophisticated, and damaging, creating significant risks to organizational assets, financial resources, and operational continuity. Effective cybersecurity mechanisms are therefore essential for protecting information systems against unauthorized access, data breaches, service disruptions, and other malicious activities (Stallings & Brown, 2018).

Traditional security mechanisms such as firewalls, antivirus software, and signature-based detection systems have provided foundational protection for many years. However, the rapidly evolving threat landscape has exposed several limitations of these approaches. Modern attackers continuously develop new techniques to bypass conventional defenses, making it increasingly difficult for static security controls to detect and mitigate emerging threats effectively. As a result, cybersecurity research has shifted toward more intelligent and adaptive solutions capable of identifying both known and unknown attack patterns (Scarfone & Mell, 2007).

Intrusion Detection Systems (IDSs) and Intrusion Prevention Systems (IPSs) have emerged as essential components of modern cybersecurity architectures. These systems continuously monitor network traffic and system activities to identify suspicious behavior, detect malicious actions, and respond to security incidents. Recent advances in Artificial Intelligence (AI), Machine Learning (ML), and Deep Learning (DL) have significantly enhanced the capabilities of IDS and IPS technologies by enabling automated learning, anomaly detection, and improved classification performance (Sowmya & Anita, 2023).

The growing volume of network traffic generated by cloud services, Internet of Things (IoT) devices, remote work environments, and web applications has created new challenges for security monitoring systems. Traditional approaches often struggle to process large amounts of traffic efficiently while maintaining high detection accuracy. Consequently, researchers have increasingly explored AI-based cybersecurity solutions capable of analyzing complex traffic patterns and supporting real-time threat detection and prevention (Ogonowski et al., 2024).

This chapter presents the theoretical foundations necessary for understanding the proposed AI-Based Intrusion Prevention System. It begins by discussing fundamental cybersecurity concepts, information security principles, cyber threats, and cybersecurity frameworks. The chapter then examines intrusion detection and prevention technologies, traditional security approaches,

artificial intelligence techniques in cybersecurity, and recent research contributions relevant to intelligent intrusion prevention systems.

2.2 Cybersecurity Fundamentals

Cybersecurity is a multidisciplinary field that focuses on protecting computer systems, networks, applications, and data from unauthorized access, misuse, disruption, or destruction. As modern organizations increasingly rely on interconnected digital infrastructures, the importance of establishing strong security mechanisms has become essential to ensure operational continuity, data protection, and system reliability. Cybersecurity not only involves technical solutions but also includes policies, procedures, and organizational practices designed to manage and reduce security risks in complex digital environments (Stallings & Brown, 2018).

2.2.1 Information Security Concepts

Cybersecurity refers to the collection of technologies, policies, processes, and practices designed to protect computer systems, networks, applications, and digital information from unauthorized access, misuse, modification, disclosure, disruption, or destruction. The primary objective of cybersecurity is to ensure the protection of organizational assets while maintaining the reliability, trustworthiness, and resilience of information systems. As organizations continue to expand their digital infrastructure, cybersecurity has evolved into a strategic requirement that affects operational continuity, regulatory compliance, and organizational reputation (Whitman & Mattord, 2022).

Information security forms the foundation of cybersecurity and focuses specifically on protecting information assets regardless of their form, whether digital, physical, or verbal. The goal of information security is to preserve the confidentiality, integrity, and availability of information throughout its lifecycle. Effective information security requires a combination of administrative, technical, and physical controls designed to minimize risks and protect valuable organizational resources (Whitman & Mattord, 2022).

Modern organizations face increasingly complex security environments characterized by interconnected systems, cloud computing platforms, mobile devices, and Internet-enabled applications. These environments introduce numerous attack surfaces that adversaries may exploit to gain unauthorized access or disrupt operations. Consequently, cybersecurity strategies must adopt a defense-in-depth approach that incorporates multiple layers of protection rather than relying on a single security mechanism (Stallings & Brown, 2018).

The effectiveness of any cybersecurity program depends on its ability to identify vulnerabilities, assess risks, implement appropriate controls, and continuously monitor security events. Security controls are generally categorized as preventive, detective, and corrective controls. Preventive controls attempt to stop attacks before they occur, detective controls identify suspicious activities, and corrective controls help restore systems following security incidents. Together,

these controls form a comprehensive security posture capable of addressing modern cyber threats (NIST, 2024).

2.2.2 The CIA Triad

One of the most widely accepted frameworks in information security is the Confidentiality, Integrity, and Availability (CIA) Triad. The CIA Triad serves as a foundational model for designing, implementing, and evaluating security policies and technologies. It provides a structured approach for understanding the core objectives of information security and remains a fundamental concept in both academic research and industrial practice (Whitman & Mattord, 2022).

Confidentiality refers to protecting information from unauthorized access and disclosure. Only authorized users, processes, or systems should be permitted to access sensitive information. Organizations typically implement confidentiality through mechanisms such as authentication, authorization, access control lists, encryption, and multi-factor authentication. Breaches of confidentiality may result in financial losses, identity theft, regulatory violations, and reputational damage (Whitman & Mattord, 2022).

Integrity ensures that information remains accurate, complete, consistent, and trustworthy throughout its lifecycle. Unauthorized modification, deletion, or corruption of data can compromise decision-making processes and business operations. Mechanisms such as cryptographic hashing, checksums, digital signatures, and audit trails are commonly used to maintain data integrity and detect unauthorized changes (Stallings & Brown, 2018).

Availability ensures that information systems, services, and data remain accessible to authorized users whenever needed. Maintaining availability requires organizations to protect systems against hardware failures, software errors, natural disasters, and cyberattacks such as Denial-of-Service (DoS) and Distributed Denial-of-Service (DDoS) attacks. Techniques such as redundancy, fault tolerance, load balancing, backups, and disaster recovery planning are commonly employed to support high availability (Whitman & Mattord, 2022).

The three components of the CIA Triad are interconnected and must be balanced carefully. Excessive emphasis on one component may negatively affect another. For example, strict confidentiality controls may reduce system accessibility, while maximizing availability without adequate access restrictions may increase security risks. Therefore, effective cybersecurity strategies seek to achieve an appropriate balance among confidentiality, integrity, and availability based on organizational requirements and risk tolerance (Whitman & Mattord, 2022).

For intrusion detection and prevention systems, the CIA Triad serves as a guiding principle for evaluating security events. Cyberattacks often target one or more elements of the triad. For instance, data breaches threaten confidentiality, unauthorized data modifications compromise integrity, and DDoS attacks impact availability. Consequently, IDS and IPS technologies play a

critical role in preserving the security objectives represented by the CIA Triad (Scarfone & Mell, 2007).

2.2.3 Cyber Threats and Attack Types

A cyber threat is any circumstance, event, or actor capable of exploiting vulnerabilities within information systems to compromise security objectives. Cyber threats may originate from external attackers, malicious insiders, organized cybercrime groups, hacktivists, or nation-state actors. The increasing sophistication of these threats has significantly expanded the challenges faced by modern organizations (Stallings & Brown, 2018).

Malware remains one of the most common categories of cyber threats. Malware includes viruses, worms, Trojans, ransomware, spyware, rootkits, and botnets. These malicious programs are designed to steal information, disrupt operations, gain unauthorized access, or provide attackers with persistent control over compromised systems. Modern malware frequently employs obfuscation and evasion techniques to avoid detection by traditional security tools (Whitman & Mattord, 2022).

Network-based attacks represent another major threat category. Examples include port scanning, reconnaissance activities, brute-force attacks, Man-in-the-Middle (MitM) attacks, Denial-of-Service (DoS) attacks, and Distributed Denial-of-Service (DDoS) attacks. These attacks target network resources, communication channels, and system availability, often causing service disruptions or enabling further exploitation (Scarfone & Mell, 2007).

Web application attacks have become increasingly prevalent due to the widespread adoption of online services and cloud-based applications. Common web attacks include SQL Injection (SQLi), Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), and authentication bypass attacks. These attacks exploit weaknesses in application logic and input validation mechanisms to compromise sensitive information or gain unauthorized access. Such attacks are particularly relevant to the proposed project because they are among the attack categories evaluated during system testing (OWASP Foundation, 2021).

2.2.4 Cybersecurity Frameworks and Defense Strategies

To manage cybersecurity risks effectively, organizations frequently adopt structured cybersecurity frameworks that provide guidance for implementing security controls and managing incidents. Among the most widely recognized frameworks is the NIST Cybersecurity Framework (CSF), which provides a risk-based approach for improving organizational cybersecurity capabilities (NIST, 2024).

The NIST Cybersecurity Framework organizes cybersecurity activities into five core functions: Identify, Protect, Detect, Respond, and Recover. These functions collectively support the entire cybersecurity lifecycle and provide a systematic approach to managing security risks.

Organizations can use the framework to assess their current security posture, prioritize improvements, and establish continuous security monitoring processes (NIST, 2024).

The Detect and Respond functions are particularly relevant to intrusion detection and prevention systems. The Detect function focuses on identifying anomalous activities and potential security incidents through continuous monitoring and analysis. The Respond function addresses incident handling, containment, mitigation, and recovery activities. IDS and IPS technologies directly contribute to these functions by monitoring network traffic, identifying malicious activities, generating alerts, and initiating automated defensive actions (Scarfone & Mell, 2007).

Modern cybersecurity strategies increasingly rely on layered defense mechanisms that combine preventive, detective, and corrective controls. This defense-in-depth approach recognizes that no single security technology can completely eliminate cyber risks. Instead, organizations deploy multiple security layers, including firewalls, intrusion detection systems, intrusion prevention systems, endpoint protection platforms, security information and event management systems, and AI-based analytics. Such integrated approaches provide greater resilience against sophisticated cyber threats and form the conceptual foundation for the AI-Based Intrusion Prevention System proposed in this project (Stallings & Brown, 2018).

2.3 Intrusion Detection Systems (IDS)

Intrusion Detection Systems (IDS) are one of the core components of modern cybersecurity defense mechanisms, designed to continuously monitor network traffic or host activities to identify suspicious behavior, policy violations, or potential security breaches. The primary objective of IDS is not to prevent attacks directly but to provide visibility into malicious activities so that security teams can respond appropriately. IDS solutions are widely deployed in enterprise environments as part of layered security architectures, often complementing firewalls and intrusion prevention systems to strengthen overall defense posture. These systems are essential in detecting a wide range of threats such as malware communication, unauthorized access attempts, privilege escalation, and denial-of-service attacks (Scarfone & Mell, 2007).

2.3.1 Overview and Concept of IDS

An Intrusion Detection System functions as a monitoring and analysis tool that inspects system or network activity for signs of malicious or abnormal behavior. IDS operates by collecting data from network packets, system logs, or application activities and comparing them against predefined rules or behavioral baselines. When suspicious behavior is detected, the system generates alerts and notifies administrators for further investigation. Unlike active security mechanisms, IDS does not intervene in traffic flow, which makes it a passive security control focused on detection and reporting (Bace & Mell, 2001).

IDS can be deployed at strategic points within a network such as perimeter gateways, internal segments, or on individual hosts. The placement of IDS sensors is critical to ensuring visibility

across different attack surfaces. In modern architectures, IDS is also integrated with Security Information and Event Management (SIEM) systems to centralize alert correlation and improve incident response efficiency (Sommer & Paxson, 2010).

2.3.2 Types of Intrusion Detection Systems

IDS can be categorized based on their deployment location and monitoring scope. The two primary categories are Network-based IDS (NIDS) and Host-based IDS (HIDS).

Network-based IDS monitors traffic flowing across the entire network by analyzing packets in real time. It is typically deployed at network entry and exit points, where it can observe all incoming and outgoing traffic. NIDS is effective in identifying network-level attacks such as port scanning, brute force attempts, and denial-of-service attacks. However, it may struggle with encrypted traffic unless additional decryption mechanisms are applied (Scarfone & Mell, 2007).

Host-based IDS operates at the individual system level and monitors internal system activities such as file integrity, system calls, application logs, and user behavior. HIDS provides deeper visibility into host-level attacks such as unauthorized file modifications or privilege escalation attempts. While HIDS is highly detailed, it requires installation and maintenance on each monitored host, making it more resource-intensive in large environments (Northcutt & Novak, 2002).

Hybrid IDS solutions combine both network-based and host-based approaches to provide comprehensive coverage. This combination enhances detection accuracy by correlating network-level anomalies with host-level activities.

2.3.3 Detection Techniques in IDS

IDS relies on multiple detection methodologies to identify malicious behavior, each with its own strengths and limitations.

Signature-based detection is the most traditional approach, where IDS compares observed traffic against a database of known attack signatures. If a match is found, an alert is generated. This method is highly effective for detecting known threats but fails to identify zero-day attacks that do not have predefined signatures (Splunk, 2023).

Anomaly-based detection establishes a baseline of normal system or network behavior and identifies deviations from this baseline as potential threats. This approach is capable of detecting unknown attacks but often suffers from high false-positive rates due to variations in legitimate behavior (Sommer & Paxson, 2010).

Stateful protocol analysis examines the state and sequence of protocol interactions to ensure compliance with expected behavior. It tracks session states and identifies abnormal sequences of commands or unexpected protocol usage. This method is particularly useful in detecting protocol

exploitation attacks but requires significant computational resources and accurate protocol modeling (Calyptix, 2025).

Modern IDS implementations increasingly adopt hybrid detection models that combine all three approaches, often enhanced by machine learning algorithms to improve accuracy and adaptability in dynamic environments.

2.3.4 IDS Architecture and Components

A typical IDS architecture consists of several key components including sensors, analyzers, a management console, and a centralized database. Sensors are responsible for collecting raw data from network traffic or host activities. The analyzer processes this data and applies detection algorithms to identify potential threats. The management console provides administrators with a user interface to view alerts, configure rules, and manage system behavior.

The IDS database stores historical logs, signatures, and behavioral profiles used for detection. Integration with external systems such as SIEM platforms enhances correlation capabilities by combining IDS alerts with other security events across the organization (Bace & Mell, 2001).

2.3.5 Limitations and Challenges of IDS

Despite its importance, IDS has several limitations that affect its effectiveness. One major challenge is the high rate of false positives, particularly in anomaly-based detection systems, where normal variations in behavior may be incorrectly flagged as malicious. This can lead to alert fatigue among security analysts.

Another limitation is the inability of IDS to actively prevent attacks. Since IDS is passive, it relies on human intervention or external systems to respond to threats, which may delay mitigation efforts. Additionally, encrypted traffic poses a significant challenge, as IDS may be unable to inspect payload content without decryption capabilities (Sommer & Paxson, 2010).

Scalability is also a concern in high-speed networks, where IDS must process large volumes of data in real time without introducing latency or packet loss.

2.4 Intrusion Prevention Systems (IPS)

Intrusion Prevention Systems (IPS) represent an evolution of IDS technology, designed not only to detect malicious activity but also to actively block or mitigate threats in real time. IPS is deployed inline within the network traffic flow, allowing it to inspect packets and immediately take corrective actions such as dropping malicious packets, resetting connections, or blocking IP addresses. This proactive capability makes IPS a critical component in modern cybersecurity infrastructures where real-time protection is essential (IBM, 2023).

2.4.1 Overview and Concept of IPS

IPS operates as a real-time security enforcement mechanism that analyzes network traffic continuously and applies security policies to prevent attacks before they reach their targets. Unlike IDS, which is passive, IPS sits directly in the communication path, meaning all traffic must pass through it. This positioning allows IPS to take immediate action against threats without requiring human intervention (DMJ Security, 2025).

IPS is commonly integrated into next-generation firewalls and unified threat management systems, where it works alongside other security controls to provide layered protection. It is particularly effective in preventing known attack patterns, exploitation attempts, and certain classes of unknown threats when combined with anomaly detection techniques.

2.4.2 Types of IPS

IPS can be categorized into several types based on deployment:

Network-based IPS (NIPS) monitors traffic across entire network segments and is typically deployed at network boundaries. It provides centralized protection against external threats.

Host-based IPS (HIPS) is installed on individual systems and protects against local attacks, malicious processes, and unauthorized system modifications.

Wireless IPS (WIPS) focuses on detecting and preventing attacks within wireless networks, including rogue access points and unauthorized devices.

2.4.3 Detection and Prevention Mechanisms in IPS

IPS uses similar detection methods as IDS, including signature-based, anomaly-based, and stateful protocol analysis. However, the key difference is that IPS takes immediate action upon detection.

In signature-based prevention, IPS blocks traffic that matches known attack patterns. In anomaly-based prevention, IPS identifies deviations from normal behavior and applies predefined mitigation actions. Stateful protocol analysis ensures that traffic follows proper protocol sequences and blocks any deviation that indicates exploitation attempts (IBM, 2023).

The ability to enforce prevention policies in real time makes IPS highly effective against fast-moving threats such as worms, botnets, and automated exploitation tools.

2.4.4 Advantages of IPS

One of the main advantages of IPS is its ability to provide immediate response to threats, reducing the window of exposure significantly. This real-time protection helps prevent damage before it occurs, unlike IDS which only alerts after detection.

IPS also reduces the workload on security teams by automating response actions. It enhances overall network security by integrating detection and prevention in a single system. Additionally, IPS can be fine-tuned to reduce exposure to known vulnerabilities and enforce strict security policies across the network (Scarfone & Mell, 2007).

2.4.5 Limitations and Challenges of IPS

Despite its advantages, IPS has several limitations. One major issue is the risk of false positives, where legitimate traffic may be incorrectly blocked, potentially disrupting business operations. This makes proper configuration and tuning critical.

Another challenge is performance overhead, as IPS must inspect all traffic in real time, which can introduce latency in high-speed networks. IPS systems also require frequent updates to signature databases to remain effective against new threats (DMJ Security, 2025).

Additionally, encrypted traffic remains a challenge for IPS, as deep inspection requires decryption capabilities that may not always be available or practical.

2.4.6 IDS vs IPS (Conceptual Distinction)

The fundamental difference between IDS and IPS lies in their operational behavior. IDS is passive and focuses on detection and alerting, while IPS is active and focuses on prevention and blocking. IDS provides visibility and forensic data, whereas IPS enforces security policies in real time.

In practice, organizations often deploy both systems together to achieve layered security. IDS provides monitoring and analysis, while IPS provides immediate threat mitigation, resulting in a more robust defense architecture.

2.5 Traditional Security Solutions

Traditional security solutions form the foundation of cybersecurity defense strategies and have been widely used long before the emergence of modern artificial intelligence and advanced behavioral analytics. These solutions primarily rely on predefined rules, static configurations, and perimeter-based defense mechanisms to protect systems and networks from unauthorized access and malicious activities. Although they are still widely deployed today, traditional approaches are increasingly considered insufficient on their own due to the rapid evolution of cyber threats and the growing complexity of modern IT environments. Their main strength lies in simplicity and predictability, but their major weakness is the inability to adapt dynamically to new and unknown attacks (Stallings, 2017).

2.5.1 Firewalls

Firewalls are one of the earliest and most fundamental security mechanisms used to control incoming and outgoing network traffic based on predetermined security rules. They act as a

barrier between trusted internal networks and untrusted external networks, such as the internet. Firewalls inspect packets and determine whether to allow or block them based on IP addresses, ports, protocols, and rule sets defined by administrators. Despite their importance, traditional firewalls are limited in their ability to inspect complex application-layer attacks and encrypted traffic without additional enhancements (Cheswick, Bellovin & Rubin, 2003).

Stateful firewalls improved upon earlier packet-filtering systems by tracking active connections and making decisions based on the state of network sessions. However, even stateful inspection remains insufficient against sophisticated attacks that mimic legitimate traffic behavior or exploit application-layer vulnerabilities.

2.5.2 Antivirus and Anti-malware Systems

Antivirus software represents another key traditional security solution, designed to detect, quarantine, and remove malicious software such as viruses, worms, trojans, and spyware. These systems primarily rely on signature-based detection, where known malware patterns are stored in a database and used for comparison against scanned files and processes. While effective against known threats, this approach struggles with zero-day attacks and polymorphic malware that can change its structure to evade detection (Symantec, 2022).

Modern antivirus solutions have evolved into endpoint protection platforms (EPP), incorporating heuristic and behavioral analysis to improve detection capabilities. However, they still depend heavily on updated signature databases and may generate false negatives when encountering previously unseen malware variants.

2.5.3 Access Control Mechanisms

Access control is a fundamental component of traditional security systems that regulates who can access specific resources within a system. It is typically implemented through authentication, authorization, and accounting (AAA) frameworks. Authentication verifies user identity, authorization determines access rights, and accounting tracks user activities for auditing purposes.

Common access control models include discretionary access control (DAC), mandatory access control (MAC), and role-based access control (RBAC). RBAC is widely used in enterprise environments due to its scalability and ease of management. However, traditional access control systems are often static and may not adapt well to dynamic environments such as cloud computing or distributed systems (Sandhu et al., 1996).

2.5.4 Limitations of Traditional Security Solutions

Despite their widespread use, traditional security solutions suffer from several critical limitations. First, they rely heavily on predefined rules and signatures, making them ineffective against unknown or evolving threats. Second, they are typically perimeter-focused, assuming that

internal networks are safe once the perimeter is secured, which is no longer valid in modern architectures due to insider threats and lateral movement attacks.

Additionally, traditional systems often generate high false negative rates when dealing with sophisticated attacks that mimic normal behavior. They also require continuous manual updates and configuration, which increases operational overhead and reduces scalability in large or cloud-based environments (Stallings, 2017).

2.5.5 Evolution Toward Modern Security Approaches

The limitations of traditional security mechanisms have driven the evolution toward more adaptive and intelligent security solutions. Organizations have gradually shifted from static rule-based systems to dynamic, behavior-based, and intelligence-driven models. This transition has led to the integration of machine learning, artificial intelligence, and big data analytics into cybersecurity frameworks.

This evolution is particularly evident in modern Security Information and Event Management (SIEM) systems, next-generation firewalls, and AI-powered intrusion detection and prevention systems, which aim to overcome the limitations of traditional approaches by enabling real-time threat detection and adaptive response mechanisms (Sommer & Paxson, 2010).

2.6 Artificial Intelligence in Cybersecurity

Artificial Intelligence (AI) has become a transformative force in cybersecurity, enabling systems to detect, analyze, and respond to threats with a level of speed and accuracy that surpasses traditional rule-based approaches. AI-driven cybersecurity systems leverage machine learning algorithms, deep learning models, and behavioral analytics to identify complex attack patterns, detect anomalies, and predict potential threats before they occur. This shift toward intelligent security systems is driven by the increasing volume, velocity, and sophistication of cyberattacks targeting modern digital infrastructures (Goodfellow, Bengio & Courville, 2016).

2.6.1 Overview of AI in Cybersecurity

AI in cybersecurity refers to the application of machine learning and intelligent algorithms to automate threat detection, response, and prevention. Unlike traditional systems that rely on predefined rules, AI-based systems learn from historical data and continuously improve their detection capabilities over time. This adaptive learning process enables AI systems to identify both known and unknown threats with higher accuracy and lower false-positive rates.

AI is widely used in areas such as intrusion detection, malware classification, phishing detection, fraud prevention, and behavioral analytics. Its ability to process large-scale data in real time makes it particularly suitable for modern high-speed network environments (Buczak & Guven, 2016).

2.6.2 Machine Learning in Threat Detection

Machine learning (ML) is a core component of AI-based cybersecurity systems. It enables models to learn patterns from labeled and unlabeled data and make predictions about future events. In intrusion detection systems, supervised learning algorithms such as decision trees, support vector machines, and random forests are commonly used to classify network traffic as benign or malicious.

Unsupervised learning techniques, such as clustering and anomaly detection, are used to identify previously unseen attacks by detecting deviations from normal behavior. These approaches are particularly useful in zero-day attack detection scenarios where no prior knowledge of the attack exists (Sommer & Paxson, 2010).

2.6.3 Deep Learning Applications

Deep learning, a subset of machine learning, has significantly improved cybersecurity capabilities by enabling the analysis of complex and high-dimensional data. Neural networks such as convolutional neural networks (CNNs) and recurrent neural networks (RNNs) are used to detect patterns in network traffic, system logs, and malware behavior.

Deep learning models are particularly effective in identifying advanced persistent threats (APTs) and sophisticated malware variants that evade traditional detection systems. However, these models require large datasets and significant computational resources, which can be a limitation in real-world deployments (Goodfellow et al., 2016).

2.6.4 AI in Intrusion Detection and Prevention

AI enhances both IDS and IPS systems by improving detection accuracy and enabling predictive capabilities. AI-based IDS can analyze network traffic in real time and identify subtle anomalies that traditional systems may overlook. Similarly, AI-powered IPS systems can automatically respond to detected threats by blocking malicious traffic or adjusting security policies dynamically.

The integration of AI into IDS/IPS systems has led to the development of intelligent security frameworks capable of self-learning and continuous adaptation. These systems reduce reliance on manual configuration and improve response times significantly (Buczak & Guven, 2016).

2.6.5 Advantages of AI-Based Cybersecurity

AI-based cybersecurity systems offer several advantages over traditional approaches. They provide higher detection accuracy, faster response times, and improved scalability. AI systems can process massive volumes of data in real time, making them suitable for cloud computing environments and large enterprise networks.

Additionally, AI systems are capable of detecting previously unknown threats through behavioral analysis and anomaly detection. This proactive capability significantly enhances overall security posture and reduces the risk of successful cyberattacks (IBM Security, 2023).

2.6.6 Challenges and Limitations of AI in Cybersecurity

Despite its advantages, AI in cybersecurity also faces several challenges. One major issue is the dependency on high-quality training data, as biased or incomplete datasets can lead to inaccurate predictions. Another challenge is adversarial attacks, where attackers manipulate input data to deceive AI models.

Additionally, AI systems require significant computational resources and expertise to develop and maintain. The lack of interpretability in some deep learning models also makes it difficult for security analysts to understand how decisions are made, which can reduce trust in automated systems (Goodfellow et al., 2016).

2.6.7 Future Trends in AI-Driven Security

The future of cybersecurity is expected to be heavily influenced by AI and automation. Emerging trends include the integration of AI with zero trust architectures, autonomous threat hunting systems, and AI-driven Security Operations Centers (SOCs). These advancements aim to create fully adaptive security ecosystems capable of responding to threats without human intervention.

As cyber threats continue to evolve, AI will play a central role in enabling predictive security, where systems not only detect attacks but also anticipate and prevent them before they occur (Buczak & Guven, 2016).

2.7 Machine Learning Techniques for Intrusion Detection

Machine Learning (ML) techniques have become a fundamental component in modern Intrusion Detection Systems (IDS) due to their ability to automatically learn patterns from data and detect both known and unknown cyber threats. Unlike traditional rule-based approaches, machine learning models analyze large volumes of network traffic and system behavior to identify anomalies, classify attacks, and improve detection accuracy over time. The adoption of ML in cybersecurity has significantly enhanced the capability of IDS to operate in dynamic and complex environments where attack patterns continuously evolve (Buczak & Guven, 2016).

2.7.1 Overview of Machine Learning in IDS

Machine learning in intrusion detection involves training algorithms on historical network data to distinguish between normal and malicious behavior. These models learn statistical patterns and correlations within the dataset, enabling them to generalize and detect future attacks. ML-based IDS systems are particularly useful in environments where manual rule creation is not scalable due to the high volume and diversity of network traffic.

ML techniques are generally categorized into supervised learning, unsupervised learning, and semi-supervised learning approaches, each offering different advantages depending on the availability of labeled data (Sommer & Paxson, 2010).

2.7.2 Supervised Learning Techniques

Supervised learning is one of the most widely used approaches in intrusion detection systems. It relies on labeled datasets where each data instance is classified as either normal or malicious. The algorithm learns the relationship between input features and output labels during the training phase and applies this knowledge to classify new unseen data.

Common supervised learning algorithms include Decision Trees, Random Forests, Support Vector Machines (SVM), K-Nearest Neighbors (KNN), and Logistic Regression. Decision Trees are widely used due to their simplicity and interpretability, while Random Forests improve accuracy by combining multiple decision trees to reduce overfitting. SVM is effective in high-dimensional spaces and is often used for binary classification problems in network intrusion detection (Patcha & Park, 2007).

However, supervised learning requires large labeled datasets, which can be difficult and expensive to obtain in real-world cybersecurity environments.

2.7.3 Unsupervised Learning Techniques

Unsupervised learning does not rely on labeled data and instead focuses on identifying hidden patterns or structures within the dataset. In intrusion detection, unsupervised learning is primarily used for anomaly detection, where the system learns what constitutes normal behavior and flags deviations as potential threats.

Clustering algorithms such as K-Means, DBSCAN, and hierarchical clustering are commonly used in IDS applications. These methods group similar data points together and identify outliers that may represent malicious activity. Unsupervised learning is particularly useful for detecting zero-day attacks and previously unseen threats, but it often suffers from higher false-positive rates compared to supervised methods (Chandola, Banerjee & Kumar, 2009).

2.7.4 Semi-Supervised Learning

Semi-supervised learning combines both labeled and unlabeled data to improve detection accuracy. This approach is particularly useful in cybersecurity, where labeled attack data is limited but large amounts of unlabeled network traffic are available.

Semi-supervised models use a small amount of labeled data to guide the learning process while leveraging the structure of unlabeled data to improve generalization. This technique helps reduce the dependency on expensive labeling processes while maintaining relatively high detection performance.

2.7.5 Challenges of Machine Learning in IDS

Despite its effectiveness, machine learning in intrusion detection faces several challenges. One major issue is data imbalance, where normal traffic significantly outweighs malicious traffic, leading to biased models. Another challenge is concept drift, where attack patterns evolve over time, making previously trained models less effective.

Additionally, adversarial attacks pose a serious threat to ML-based IDS systems, where attackers intentionally manipulate input data to evade detection. These challenges highlight the need for continuous model retraining and adaptive learning techniques in cybersecurity systems (Sommer & Paxson, 2010).

2.8 Deep Learning Techniques

Deep Learning (DL) is an advanced subset of machine learning that utilizes multi-layered neural networks to automatically extract complex features from raw data. In cybersecurity, deep learning has significantly improved intrusion detection capabilities by enabling systems to detect highly sophisticated and previously unknown attack patterns. DL models are particularly effective in analyzing large-scale, high-dimensional datasets such as network traffic flows, system logs, and behavioral data (Goodfellow, Bengio & Courville, 2016).

2.8.1 Overview of Deep Learning in Cybersecurity

Deep learning models differ from traditional machine learning techniques in their ability to automatically learn feature representations without manual feature engineering. This makes them highly suitable for intrusion detection tasks, where feature extraction from raw network traffic can be complex and time-consuming.

DL-based IDS systems are widely used in modern Security Operations Centers (SOCs) to enhance real-time threat detection and reduce reliance on manual rule configuration.

2.8.2 Convolutional Neural Networks (CNNs)

Convolutional Neural Networks (CNNs) are primarily used for spatial data analysis but have been adapted for intrusion detection by treating network traffic as structured input. CNNs are capable of automatically extracting hierarchical features from raw data, making them highly effective in identifying patterns in packet-level data.

In IDS applications, CNNs can detect malicious traffic patterns by analyzing flow characteristics and identifying spatial correlations between features. Their ability to reduce feature engineering requirements makes them a powerful tool in cybersecurity analytics (Kim et al., 2018).

2.8.3 Recurrent Neural Networks (RNNs) and LSTM

Recurrent Neural Networks (RNNs) are designed to handle sequential data, making them highly suitable for analyzing time-dependent network traffic. Long Short-Term Memory (LSTM) networks, a variant of RNNs, are particularly effective in capturing long-term dependencies in sequential data.

In intrusion detection, LSTM models are used to analyze network flow sequences and detect abnormal behavioral patterns over time. This makes them highly effective in identifying advanced persistent threats (APTs) and multi-stage attacks that unfold over long periods (Kim et al., 2016).

2.8.4 Autoencoders for Anomaly Detection

Autoencoders are unsupervised deep learning models used for anomaly detection in IDS. They work by compressing input data into a lower-dimensional representation and then reconstructing it. If the reconstruction error is high, the input is considered anomalous.

In cybersecurity, autoencoders are trained on normal network traffic, allowing them to detect deviations that may indicate malicious activity. This approach is particularly useful for zero-day attack detection and environments with limited labeled data (Chalapathy & Chawla, 2019).

2.8.5 Advantages of Deep Learning in IDS

Deep learning provides several advantages over traditional machine learning techniques. It eliminates the need for manual feature engineering, improves detection accuracy, and can handle large-scale and high-dimensional data efficiently. DL models are also highly effective in detecting complex attack patterns that are difficult for traditional systems to identify.

Additionally, deep learning systems can continuously improve performance as more data becomes available, making them suitable for adaptive cybersecurity environments (Goodfellow et al., 2016).

2.8.6 Limitations of Deep Learning in IDS

Despite its strengths, deep learning also has limitations. One major challenge is the requirement for large datasets and high computational power, which can make deployment expensive. Another issue is the lack of interpretability, as deep learning models often function as “black boxes,” making it difficult for security analysts to understand decision-making processes.

Furthermore, deep learning models are vulnerable to adversarial attacks, where small perturbations in input data can lead to incorrect classifications. These limitations highlight the need for hybrid approaches combining deep learning with traditional security mechanisms (Chalapathy & Chawla, 2019).

2.8.7 Future Directions of Deep Learning in IDS

The future of deep learning in intrusion detection is expected to focus on explainable AI, real-time processing, and hybrid architectures combining multiple learning techniques. Research is also moving toward lightweight deep learning models that can operate efficiently in resource-constrained environments such as IoT and edge devices.

As cyber threats continue to evolve, deep learning will play a critical role in enabling intelligent, adaptive, and autonomous intrusion detection systems capable of defending against sophisticated attacks in real time (Buczak & Guven, 2016).

2.9 Review of Existing Solutions

Modern cybersecurity systems for intrusion detection and prevention have evolved significantly over the past decades, leading to a wide range of commercial, open-source, and research-based solutions. These systems vary in architecture, detection techniques, scalability, and intelligence level. Reviewing existing solutions is essential to understand the strengths and weaknesses of current technologies and to identify gaps that can be addressed through advanced approaches such as machine learning and deep learning-based IDS. Most existing systems still rely on a combination of signature-based detection and rule-based policies, while only a subset incorporates adaptive or AI-driven capabilities (Scarfone & Mell, 2007).

2.9.1 Signature-Based Intrusion Detection Systems

Signature-based IDS solutions are among the most widely deployed security tools in enterprise environments. These systems rely on a predefined database of known attack patterns (signatures) to detect malicious activities. When network traffic matches a stored signature, the system triggers an alert or blocks the traffic depending on configuration. Solutions such as Snort and Suricata are prominent examples of signature-based IDS widely used in both academic and industrial environments.

Snort, in particular, is an open-source IDS/IPS system that provides real-time traffic analysis and packet logging capabilities. It is highly customizable and supports rule-based detection, making it suitable for network administrators and security researchers. However, its dependency on known signatures limits its ability to detect zero-day attacks or previously unseen threats (Roesch, 1999).

Suricata enhances Snort’s capabilities by supporting multi-threaded processing and improved performance in high-speed networks. It also integrates better protocol analysis and file extraction features, making it more suitable for modern enterprise networks (Open Information Security Foundation, 2023).

2.9.2 Anomaly-Based Intrusion Detection Systems

Anomaly-based IDS solutions attempt to identify deviations from normal system behavior rather than relying on predefined signatures. These systems establish a baseline of normal network or host activity and flag any significant deviation as suspicious. This approach is particularly effective in detecting unknown or zero-day attacks.

However, anomaly-based systems suffer from a major limitation: high false-positive rates. Because normal behavior in real-world networks is highly dynamic, distinguishing between legitimate and malicious anomalies can be challenging. Despite this limitation, anomaly-based detection remains a critical area of research and is widely used in academic prototypes and AI-driven security systems (Chandola, Banerjee & Kumar, 2009).

2.9.3 Hybrid IDS/IPS Solutions

Hybrid solutions combine both signature-based and anomaly-based detection techniques to improve overall accuracy and detection coverage. These systems aim to balance the precision of signature-based methods with the adaptability of anomaly detection.

Modern hybrid systems are often integrated into Next-Generation Firewalls (NGFWs), which combine traditional firewall capabilities with intrusion prevention, deep packet inspection, and application-level filtering. Vendors such as Cisco, Palo Alto Networks, and Fortinet provide commercial solutions that incorporate hybrid IDS/IPS functionality.

While hybrid systems improve detection performance, they still require significant manual tuning and continuous updates to maintain effectiveness against emerging threats (Stallings, 2017).

2.9.4 Machine Learning-Based Security Solutions

Recent years have seen the emergence of machine learning-based IDS solutions designed to overcome the limitations of traditional approaches. These systems analyze network traffic patterns and automatically learn to classify behavior as normal or malicious. Unlike signature-based systems, ML-based solutions can detect unknown attacks by identifying statistical anomalies in data.

Research-based solutions often use datasets such as KDD99, NSL-KDD, and UNSW-NB15 to train classification models. Algorithms such as Random Forest, Support Vector Machines, and Gradient Boosting are commonly applied in these systems.

Despite their advantages, many ML-based IDS solutions face challenges related to dataset imbalance, overfitting, and poor generalization in real-world environments. This gap between research performance and real-world deployment remains a key limitation (Sommer & Paxson, 2010).

2.9.5 Deep Learning-Based Security Solutions

Deep learning-based IDS solutions represent a more advanced class of cybersecurity systems that leverage neural networks to automatically extract features and detect complex attack patterns. These systems are capable of analyzing raw network flows and system logs without requiring manual feature engineering.

Common implementations include CNN-based IDS for spatial feature extraction and LSTM-based models for sequential traffic analysis. Autoencoder-based anomaly detection systems are also widely used for identifying unknown attacks.

Although deep learning solutions provide high accuracy in controlled environments, they require large datasets, high computational resources, and are often difficult to interpret, which limits their adoption in operational security environments (Goodfellow, Bengio & Courville, 2016).

2.9.6 Commercial Security Solutions

Commercial cybersecurity platforms provide integrated solutions that combine firewall protection, intrusion detection, intrusion prevention, endpoint security, and cloud security capabilities. Examples include Cisco Firepower, Palo Alto Networks NGFW, and Fortinet FortiGate.

These systems are designed for enterprise scalability and provide centralized management dashboards, automated threat intelligence updates, and real-time monitoring capabilities. However, they are often expensive and may not provide full transparency into their detection algorithms, limiting customization for research or specialized use cases.

2.9.7 Open-Source Security Solutions

Open-source security tools play a significant role in both research and real-world deployments. Tools such as Snort, Suricata, Zeek (formerly Bro), and OSSEC provide flexible and customizable security monitoring capabilities.

Zeek, for example, focuses on network traffic analysis and provides detailed logging and behavioral insights rather than direct prevention. OSSEC is a host-based intrusion detection system that monitors file integrity, log files, and system activity.

While open-source solutions offer flexibility and cost advantages, they often require significant configuration expertise and lack the advanced automation features found in commercial platforms (Zeek Project, 2023).

2.9.8 Research Gaps in Existing Solutions

Despite advancements in cybersecurity technologies, several research gaps still exist in current IDS/IPS solutions. One major gap is the limited ability of traditional systems to detect zero-day

and advanced persistent threats in real time. Another limitation is the lack of adaptability in signature-based systems, which require continuous manual updates.

Machine learning and deep learning solutions, while promising, still face challenges in deployment due to data quality issues, computational complexity, and lack of interpretability. Additionally, many existing systems are not optimized for high-speed modern networks or encrypted traffic analysis.

These limitations highlight the need for intelligent, adaptive, and scalable intrusion detection frameworks that combine the strengths of machine learning, deep learning, and traditional security mechanisms into a unified system (Buczak & Guven, 2016).

2.10 Related Work Comparison

Research in the field of intrusion detection and cybersecurity has evolved significantly over the past two decades, with numerous studies proposing different approaches ranging from traditional signature-based systems to advanced machine learning and deep learning models. A comparison of related work reveals clear differences in detection accuracy, scalability, adaptability, and computational complexity. Early studies primarily focused on rule-based and statistical approaches, while recent research emphasizes intelligent, data-driven systems capable of detecting unknown and evolving threats (Buczak & Guven, 2016).

2.10.1 Traditional vs Intelligent Approaches

Traditional intrusion detection systems rely heavily on predefined signatures and manually crafted rules. These systems are effective in detecting known attacks but fail to identify zero-day or previously unseen threats. In contrast, intelligent systems based on machine learning and deep learning can learn patterns directly from data, allowing them to generalize and detect anomalies without explicit programming.

While traditional systems offer simplicity and low computational cost, they lack adaptability. Intelligent systems, however, provide higher detection accuracy but require large datasets and computational resources. This trade-off remains a central theme in cybersecurity research (Sommer & Paxson, 2010).

2.10.2 Machine Learning vs Deep Learning Approaches

Machine learning-based IDS solutions, such as Random Forests, Support Vector Machines, and K-Nearest Neighbors, have demonstrated strong performance in classification tasks using structured network traffic data. However, these models depend heavily on feature engineering, which limits their scalability in complex environments.

Deep learning approaches, including Convolutional Neural Networks (CNNs) and Long Short-Term Memory (LSTM) networks, eliminate the need for manual feature extraction by automatically learning hierarchical representations from raw data. This allows deep learning

models to capture more complex patterns in network traffic, improving detection of advanced persistent threats and multi-stage attacks (Goodfellow, Bengio & Courville, 2016).

Despite their advantages, deep learning models require significant computational resources and large labeled datasets, making them more suitable for high-performance environments such as cloud-based security systems and Security Operation Centers (SOCs).

2.10.3 Comparative Summary of Existing Research

Existing research can be broadly categorized into three main groups:

- Signature-based systems (e.g., Snort, Suricata)
- Machine learning-based IDS (e.g., Random Forest, SVM-based models)
- Deep learning-based IDS (e.g., CNN, LSTM, Autoencoders)

Signature-based systems offer high precision for known attacks but fail against unknown threats. Machine learning models provide better generalization but depend on feature quality and labeled data. Deep learning models achieve the highest accuracy in complex scenarios but suffer from high computational cost and lack of interpretability.

This comparison highlights that no single approach is sufficient to handle all cybersecurity challenges, motivating hybrid and AI-driven solutions (Chandola, Banerjee & Kumar, 2009).

2.11 Research Gap

The analysis of existing intrusion detection and prevention systems, along with comparative studies presented in the previous sections, reveals several persistent limitations in both traditional and modern cybersecurity approaches. Although significant progress has been made through signature-based, machine learning, and deep learning-based solutions, there remains a clear gap between research prototypes and real-world deployable systems. These gaps are derived from the limitations observed across existing studies, commercial tools, and academic models (Sommer & Paxson, 2010; Buczak & Guven, 2016).

2.11.1 Limited Real-Time Deployment in Existing Studies

Most reviewed research focuses on offline datasets and controlled environments, where models are trained and evaluated using historical data. However, real-world network environments require continuous real-time processing of high-speed traffic. Existing solutions often fail to demonstrate full real-time deployment capability, particularly when integrated with streaming network data sources such as live flow-based monitoring systems.

2.11.2 Lack of Unified Flow-Based AI Systems

Although many studies propose machine learning or deep learning models for intrusion detection, they are often developed independently from network flow extraction tools. There is a noticeable gap in integrating flow-based feature extraction systems (such as NFStream or similar frameworks) directly with AI pipelines for end-to-end detection. This disconnect reduces the practicality and scalability of research solutions in real operational environments.

2.11.3 Dependency on Static Datasets

A major limitation in related work is the heavy reliance on benchmark datasets such as KDD99, NSL-KDD, and UNSW-NB15. While these datasets are useful for evaluation, they do not fully represent modern network traffic patterns, encrypted communication, or evolving attack techniques. This leads to models that perform well in research but fail to generalize effectively in live environments.

2.11.4 Insufficient Handling of Encrypted and Modern Traffic

Most traditional IDS/IPS and even several machine learning-based models focus on packet-level inspection. However, modern networks increasingly rely on encrypted protocols such as TLS and HTTPS, which limit payload visibility. Existing research does not sufficiently address encrypted traffic analysis using behavioral or flow-based features.

2.11.5 Scalability and Performance Constraints

Deep learning-based intrusion detection systems often achieve high accuracy but suffer from computational overhead and latency issues. Many existing studies do not evaluate scalability under high-throughput network conditions, making them unsuitable for enterprise-scale or ISP-level deployment scenarios.

2.11.6 Lack of Adaptive and Continuous Learning Mechanisms

Another key gap is the absence of continuous learning mechanisms in most proposed systems. Cyber threats evolve rapidly, yet many models are trained once and deployed statically without adaptation. This results in performance degradation over time, especially under concept drift conditions where attack patterns change.

2.11.7 Summary of Research Gap

In summary, the literature reveals a clear gap in developing a unified, real-time, scalable, and adaptive intrusion detection framework that integrates flow-based network analysis with machine learning or deep learning techniques. Addressing these limitations forms the foundation for the proposed system in this research, which aims to bridge the gap between theoretical models and practical cybersecurity deployment environments.

Chapter 3 System Analysis and Design

3.1 System Requirements Analysis

The requirements analysis phase forms the foundational basis of the AI-Based Intrusion Prevention System (IPS). Through a thorough evaluation of existing intrusion detection limitations and the operational demands of modern enterprise networks, a comprehensive set of system requirements was derived. This analysis considered the functional scope of each sub-module — network infrastructure, AI detection engine, web application, and DevOps pipeline — to ensure a coherent and complete system design.

The analysis identified that traditional IDS/IPS solutions relying on static signature databases could not adequately address zero-day attacks, evolving malware variants, and distributed attack campaigns. Consequently, the system requirements were formulated to demand AI-driven detection, real-time inference capabilities, multi-class attack categorization, and automated prevention mechanisms without human intervention.

The following sections detail the functional and non-functional requirements that govern the design and implementation of the proposed system.

3.2 Functional Requirements

The functional requirements specify the behavioral capabilities that the system must provide. They are organized by subsystem to reflect the modular architecture of the proposed IPS.

Traffic Capture and Flow Processing

- The system shall capture live network traffic from one or more monitored interfaces using NFStream.
- The system shall extract flow-level features including volume, statistical, temporal, directional, and TCP behavioral attributes.
- The system shall group analyzed traffic events into batches of 20 for aggregated analysis and reduced alert noise.

AI Detection and Classification

- The system shall classify each network flow as either benign or malicious using a trained binary anomaly detection model.
- The system shall identify the specific attack category (DDoS, DoS, Port Scan, Brute Force, SQL Injection, XSS, etc.) using a multi-class classification model.
- The system shall analyze HTTP payloads from web traffic to detect application-layer attacks.
- The system shall produce a confidence score for each prediction.

- The system shall process and classify traffic in near real-time, sustaining high-throughput environments.

Prevention and Response

- The system shall block traffic classified as malicious and allow benign.
- The system shall enforce blocking policies through integration with firewall rules.
- The system shall generate automated security alerts for all detected malicious traffic.

Monitoring and Reporting

- The system shall provide a real-time monitoring dashboard accessible via a web browser.
- The system shall display recent detection events, threat statistics, and attack distribution charts.
- The system shall push real-time updates to the dashboard without requiring manual page refresh.
- The system shall generate downloadable PDF security reports containing event summaries, threat logs, and system status.

User and Access Management

- The system shall support multi-user access with role-based authorization (Administrator / Viewer).
- The system shall enforce email verification prior to account activation.
- The system shall allow administrators to approve or reject requests for elevated role assignments.

3.3 Non-Functional Requirements

Non-functional requirements define the quality attributes and operational constraints that govern the overall system performance, reliability, and security.

Attribute	Requirement
Performance	The AI inference engine shall produce classification results within 200 ms per batch under normal load conditions, enabling near real-time intrusion prevention.
Scalability	The system shall be deployable on Kubernetes (k3s), allowing horizontal scaling of the web application and AI engine containers to handle increased network traffic volumes.
Availability	The system shall maintain a target uptime of 99.5% or higher. Kubernetes automatic pod restarts and AWS RDS managed database service shall ensure service continuity.
Security	All inter-component communication shall be authenticated. The web application shall enforce HTTPS, role-based access control, and secure password hashing via ASP.NET Identity.
Usability	The dashboard interface shall be intuitive, requiring no specialized training for security analysts. All critical metrics shall be visible on the primary dashboard without navigation.
Maintainability	The system shall be containerized using Docker, enabling isolated updates to individual components without disrupting the overall system.
Portability	All components shall be packaged as Docker containers and deployable on any Linux-based cloud or on-premises server without dependency conflicts.
Observability	System health metrics including CPU usage, memory, and request throughput shall be collected by Prometheus and visualized through Grafana dashboards in real time.

Table 1: Non-Functional System Requirements

3.4 Proposed System Overview

The proposed system is an AI-Based Intrusion Prevention System designed to monitor, detect, and block malicious network activity in enterprise environments. Unlike conventional rule-based IPS solutions, the proposed system employs a deep learning model trained on both network flow statistics and web application payloads, enabling it to identify a broad spectrum of attack categories including network-layer floods, reconnaissance scans, brute-force credential attacks, and application-layer injection attacks.

The system is composed of four tightly integrated sub-systems:

- Network Infrastructure Layer — A multi-tier enterprise network built in GNS3 and VMware, incorporating FortiGate firewalls, routers and switches, VLANs, OSPF routing, a DMZ, and a Data Center with Active Directory, DHCP, DNS, and web services.
- AI Detection Engine — A multimodal dual-head deep neural network deployed as a containerized REST API. It processes network flow features and HTTP payloads concurrently, producing binary anomaly predictions and multi-class attack category labels.
- Web Application Platform — An ASP.NET Core MVC web application providing the management and monitoring interface. It receives AI classification results, stores them in SQL Server, and presents them on a real-time SignalR-powered dashboard.
- DevOps and Infrastructure Pipeline — A CI/CD pipeline using GitHub Actions automates building, testing, security scanning, and deployment of Docker containers onto an AWS EC2 instance managed by Kubernetes. Prometheus and Grafana provide operational monitoring.

Traffic entering the monitored network is intercepted, processed into flow records, submitted to the AI engine for classification, and — if deemed malicious — blocked by the firewall. Security events and alerts are simultaneously logged to the database and broadcast to the live dashboard, giving administrators immediate situational awareness.

Data Sources and Traffic Processing

The proposed AI-Based Intrusion Prevention System utilizes multiple sources of network traffic data to improve detection performance and generalization capabilities. The primary benchmark dataset used during development was the CICIDS2017 dataset. However, because benchmark datasets alone may not accurately represent real-world network environments, additional custom and synthetic traffic datasets were incorporated into the training and evaluation process.

Network traffic is processed using NFStream, which converts raw packet captures and live network traffic into flow-based records. NFStream was selected after evaluating alternative flow generation tools due to its superior flow aggregation capabilities, lower feature noise, improved session consistency, and support for real-time traffic analysis. The extracted flow records serve as the primary input to the AI detection engine.

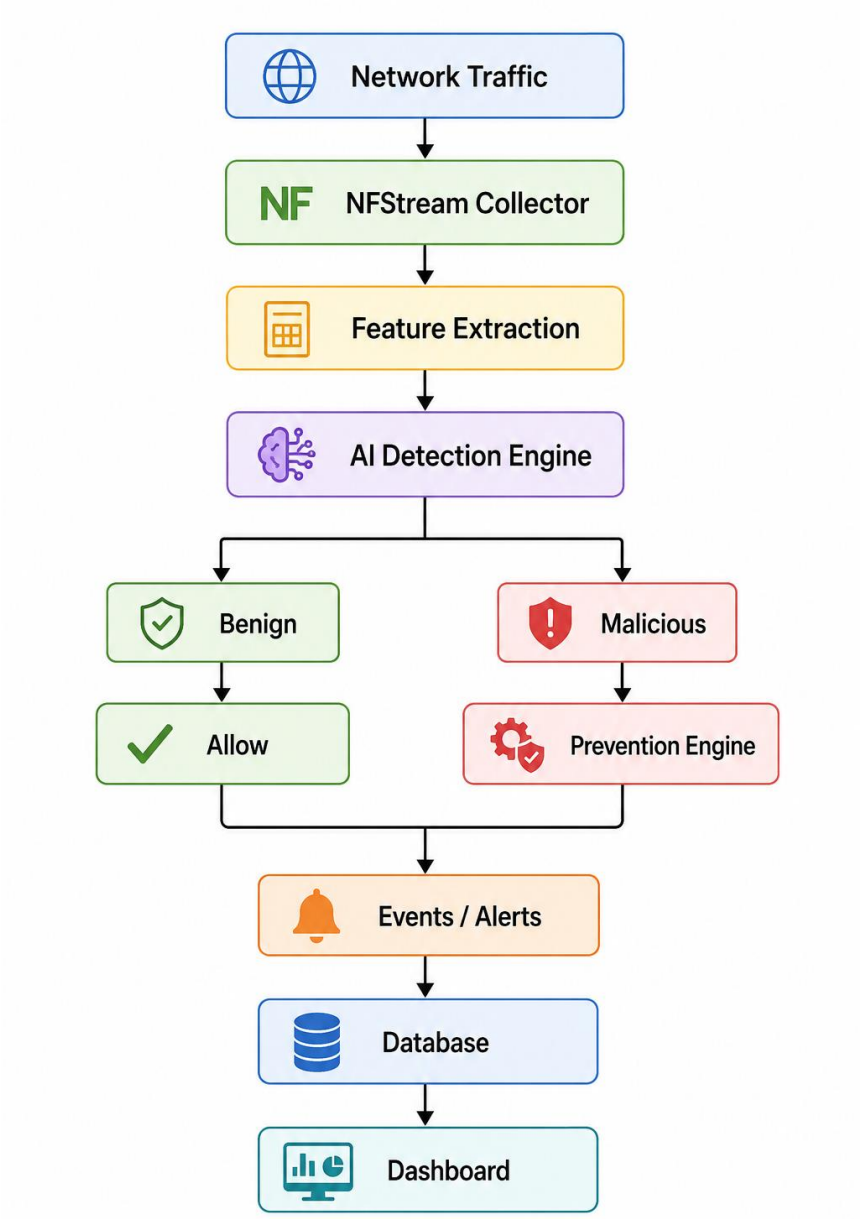

Figure 1: High-Level Overview of the Proposed AI-Based Intrusion Prevention System

3.5 System Architecture

The overall system architecture follows a pipeline-oriented design in which network traffic traverses sequential processing stages from raw packet capture to dashboard visualization. Figure below illustrates the high-level architecture of the proposed system.

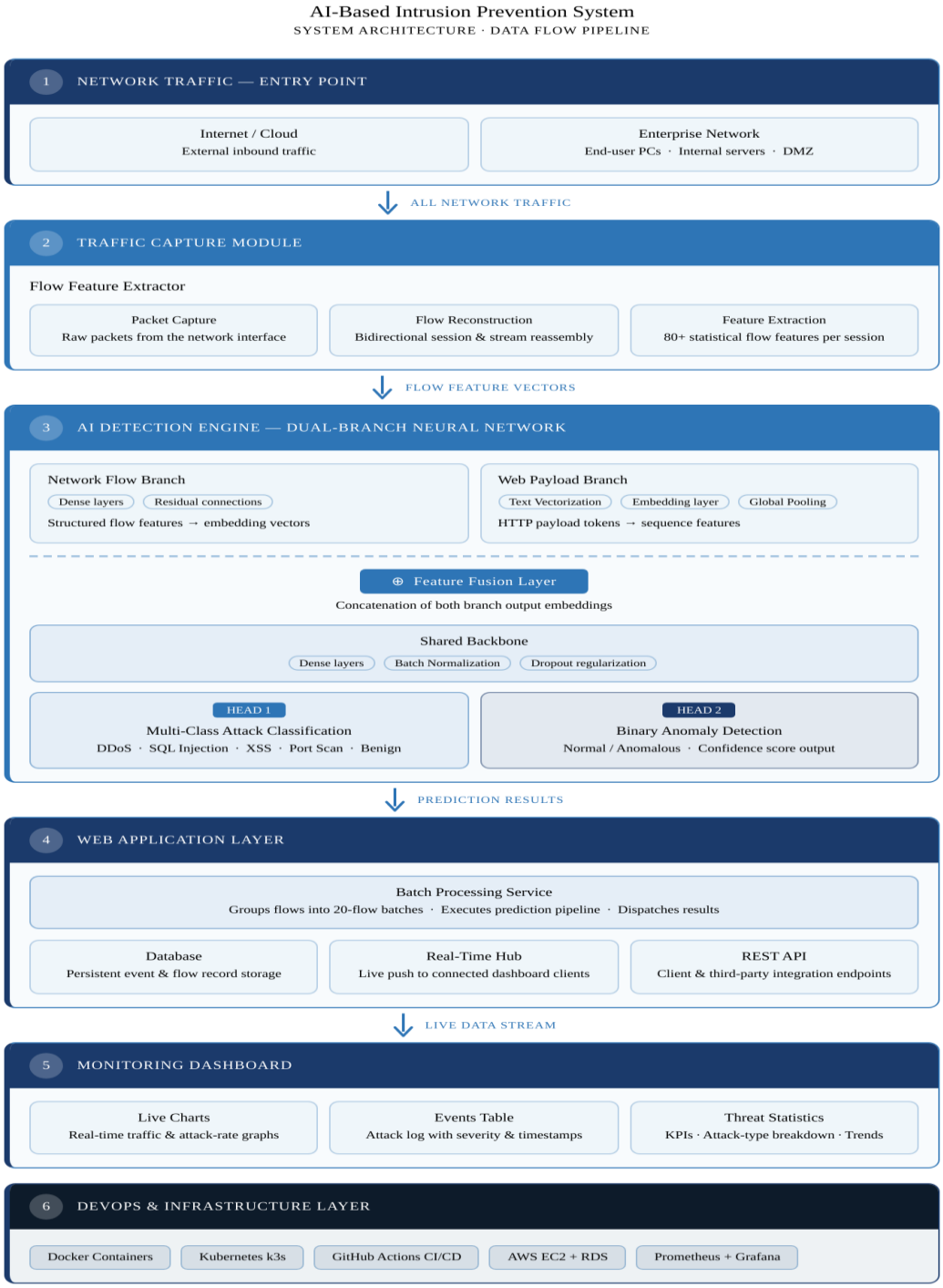

Figure 2: System Architecture

The architecture enforces a strict separation of concerns: the AI engine operates independently of the web application, communicating only via HTTP API calls. This separation enables independent scaling, testing, and redeployment of each component without disrupting the overall system.

3.6 Network Architecture

The proposed AI-Based Intrusion Prevention System was deployed within a simulated enterprise network environment designed according to the Hierarchical Network Design Model. This architecture separates network functions into multiple layers to improve scalability, security, fault isolation, and operational efficiency. The environment was implemented using GNS3 and VMware Workstation and emulates a realistic enterprise infrastructure containing internal users, servers, security devices, and attacker machines.

The network architecture consists of three primary layers:

Access Layer

The Access Layer provides connectivity for end-user devices and workstation systems. It represents the entry point through which users access network resources and services.

Devices connected at this layer include:

- Windows 10 client machines
- Administrative workstations
- User endpoints
- Virtual machines used for testing

The Access Layer is responsible for:

- Device connectivity
- User access control
- VLAN implementation
- Basic network security
- Traffic segmentation

VLANs are used to separate users and services into different logical networks, reducing broadcast traffic and improving security.

Distribution Layer

The Distribution Layer serves as the intermediary between the Access Layer and the Core Layer. It aggregates traffic from multiple access switches and applies security and routing policies.

Its responsibilities include:

- Inter-VLAN routing
- Traffic aggregation
- Access Control List (ACL) enforcement
- Security policy implementation
- Traffic filtering

By implementing ACLs at this layer, unauthorized communication between network segments can be restricted before traffic reaches critical infrastructure components.

Core Layer

The Core Layer acts as the backbone of the enterprise network. Its primary objective is to provide fast, reliable, and highly available communication between all network segments.

Core Layer functions include:

- High-speed packet forwarding
- Redundant communication paths
- Dynamic route propagation
- High availability
- Network stability

The network uses the Open Shortest Path First (OSPF) routing protocol to provide automatic route discovery and efficient path selection between network segments.

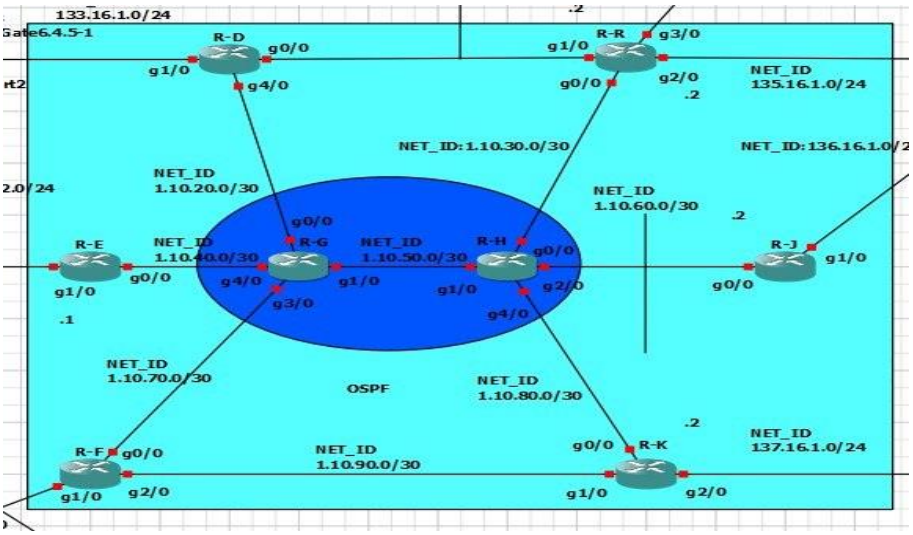

Figure 3: Core Layer Connectivity and OSPF Routing

Data Center Infrastructure

The Data Center hosts the critical enterprise services required for network operations and user management.

The deployed services include:

- Active Directory Domain Controller
- DNS Server
- DHCP Server
- File Sharing Server
- Group Policy Services

These services provide centralized authentication, name resolution, address assignment, and resource management across the enterprise network.

The Data Center represents the most protected zone within the network and is accessible only through controlled security policies enforced by the firewall infrastructure.

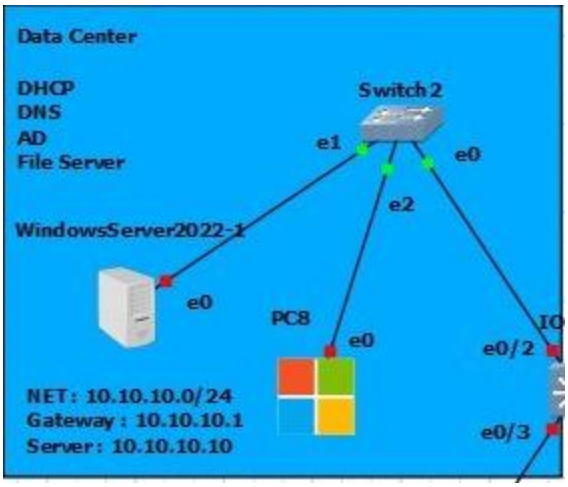

Figure 4: Data Center Architecture

Demilitarized Zone (DMZ)

A Demilitarized Zone (DMZ) was implemented to host public-facing services while maintaining isolation from the internal corporate network.

The DMZ contains:

- Web Servers
- FTP Servers
- Remote Desktop Services

The DMZ is positioned between the Internet and the internal network and is protected by dedicated firewall policies.

This architecture reduces the risk of internal compromise if a publicly accessible service becomes vulnerable or is successfully attacked.

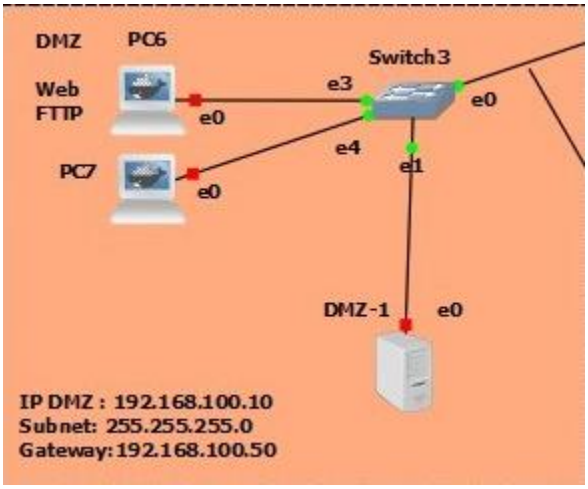

Figure 5: DMZ Network Segment

Security Architecture

The network employs a defense-in-depth strategy through the use of FortiGate firewall technology.

The firewall performs:

- Stateful packet inspection
- Access control enforcement
- Network Address Translation (NAT)
- Traffic logging
- Intrusion Prevention
- Security monitoring

Separate firewall policies govern traffic between:

- Internet and DMZ
- DMZ and Internal Network
- Internal Network and Data Center

The firewall also serves as the primary enforcement point for automated blocking actions generated by the AI-Based IPS.

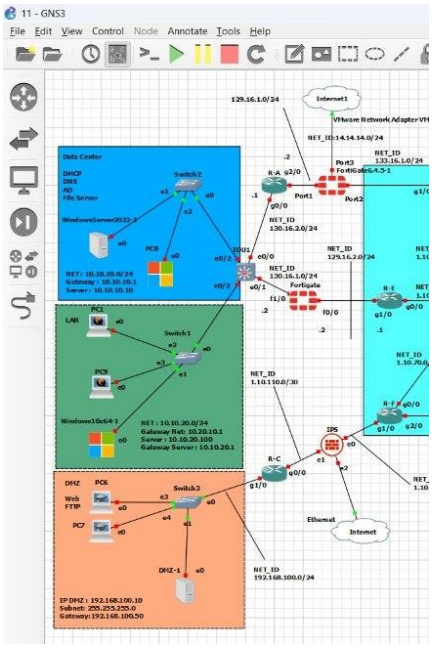

Figure 6: Firewall Placement and Security Zones

Attack Simulation Environment

To evaluate the effectiveness of the AI-Based IPS, a dedicated attack environment was deployed.

The attack infrastructure includes:

- Kali Linux
- Metasploit Framework
- Custom attack generation tools

These systems simulate:

- DDoS attacks
- DoS attacks
- Port scanning
- Brute-force attacks
- Web application attacks
- Lateral movement scenarios

The attack environment allows realistic testing of detection and prevention capabilities without affecting production systems.

3.7 AI Detection Architecture

The AI Detection Engine represents the intelligence layer of the proposed Intrusion Prevention System. Its primary objective is to identify malicious network activities in real time while simultaneously classifying attack categories. The architecture was designed to support both binary anomaly detection and multiclass attack classification through a unified deep learning framework.

The AI subsystem processes network traffic flows generated by NFStream and applies a sequence of preprocessing, feature engineering, and prediction stages before producing security decisions.

3.7.1 Flow-Based Feature Extraction

Incoming network traffic is transformed into bidirectional flow records using NFStream. The extracted features describe various characteristics of network communication, including:

- Traffic volume statistics
- Flow duration
- Packet size distributions
- Packet timing behavior
- Directional communication patterns
- TCP behavioral indicators

These features provide a numerical representation of network activity suitable for machine learning processing.

3.7.2 Dataset Preparation

Training data was collected from multiple sources to improve robustness and model generalization.

The primary dataset was CICIDS2017, which contains realistic benign and malicious network traffic scenarios. To overcome limitations associated with benchmark-only training, additional traffic samples were generated through custom traffic collection and synthetic augmentation techniques.

The final training dataset therefore consisted of:

- CICIDS2017 traffic
- Custom captured traffic
- Synthetic traffic samples

This combination reduces dataset bias and improves the model's ability to detect attacks in previously unseen environments.

3.7.3 Synthetic Dataset Generation

Several synthetic data generation techniques were evaluated during system development.

Initial experiments included:

- Conditional Tabular Generative Adversarial Networks (CTGAN)
- Gaussian Mixture Models (GMM)

Although both approaches generated synthetic samples, validation revealed that they frequently violated structural relationships within network flow records, resulting in unrealistic traffic representations.

To address these limitations, a cluster-based bootstrapping methodology with statistical perturbation was developed. This approach preserves network traffic integrity while introducing controlled behavioral variations that improve model generalization.

The resulting synthetic dataset was incorporated into the training pipeline to increase diversity and reduce model overfitting.

3.7.4 Evaluated Machine Learning Models

Multiple machine learning and deep learning architectures were evaluated before selecting the final deployment model.

The evaluated models included:

- Random Forest
- CNN
- CNN-BiLSTM
- CNN-GRU
- CNN-BiGRU
- Dual-Head Deep Neural Network

Each model was trained and evaluated using identical preprocessing pipelines and performance metrics. Comparative analysis demonstrated that deep learning architectures consistently outperformed traditional machine learning approaches when detecting complex attack behaviors.

3.7.5 Final Dual-Head Detection Model

Based on experimental evaluation, a custom Dual-Head Deep Neural Network was selected as the final detection architecture.

The model performs two prediction tasks simultaneously:

1. Binary anomaly detection
2. Multi-class attack classification

A shared feature extraction backbone learns generalized network behavior patterns, while two independent output layers specialize in their respective prediction tasks.

This architecture improves the ability to detect previously unseen attacks while maintaining accurate attack categorization.

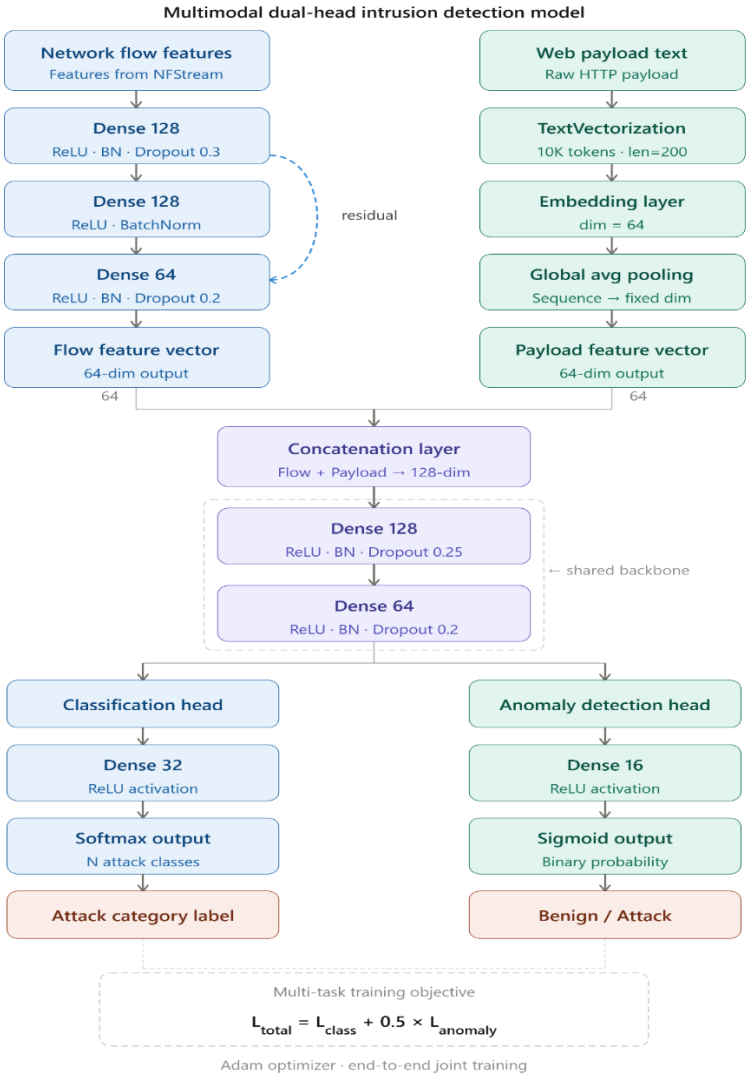

Figure 7: AI Detection Architecture

3.8 Database Design

The system's persistent data layer is implemented in Microsoft SQL Server, managed via Entity Framework Core. The schema is structured around eight primary entities that collectively support user management, traffic event logging, threat cataloging, alert generation, and system status tracking.

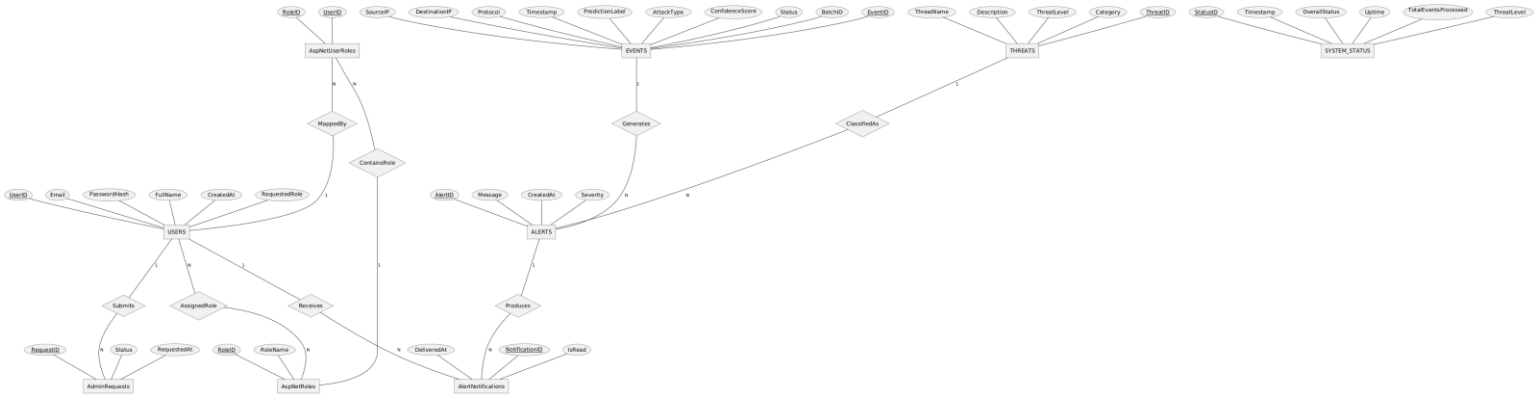

Figure 8: Database Entity Relationship Diagram (ERD)

The EVENTS table serves as the primary operational log, recording every analyzed traffic batch with its source and destination IP addresses, prediction outcome, attack category, model confidence score, and enforcement action (Allowed or Blocked). The THREATS table maintains a catalog of known attack types with descriptions and severity levels, enabling threat context enrichment. The ALERTS and AlertNotifications tables drive the real-time notification system, ensuring administrators are informed of active threats immediately upon detection.

AI Prediction Storage

The database stores both binary anomaly detection results and multiclass attack classifications generated by the AI engine. Each event record includes prediction confidence scores, attack category labels, timestamps, source and destination network identifiers, and prevention decisions. These records support dashboard visualization, reporting, forensic investigations, and future model evaluation activities.

3.9 Use Case Diagram

The use case diagram defines the interactions between the system actors and the functional capabilities of the IPS. Two primary actors are identified: the Security Administrator, who interacts with the web application to monitor the network and manage the system, and the AI Engine, which operates autonomously to process traffic and generate classification results.

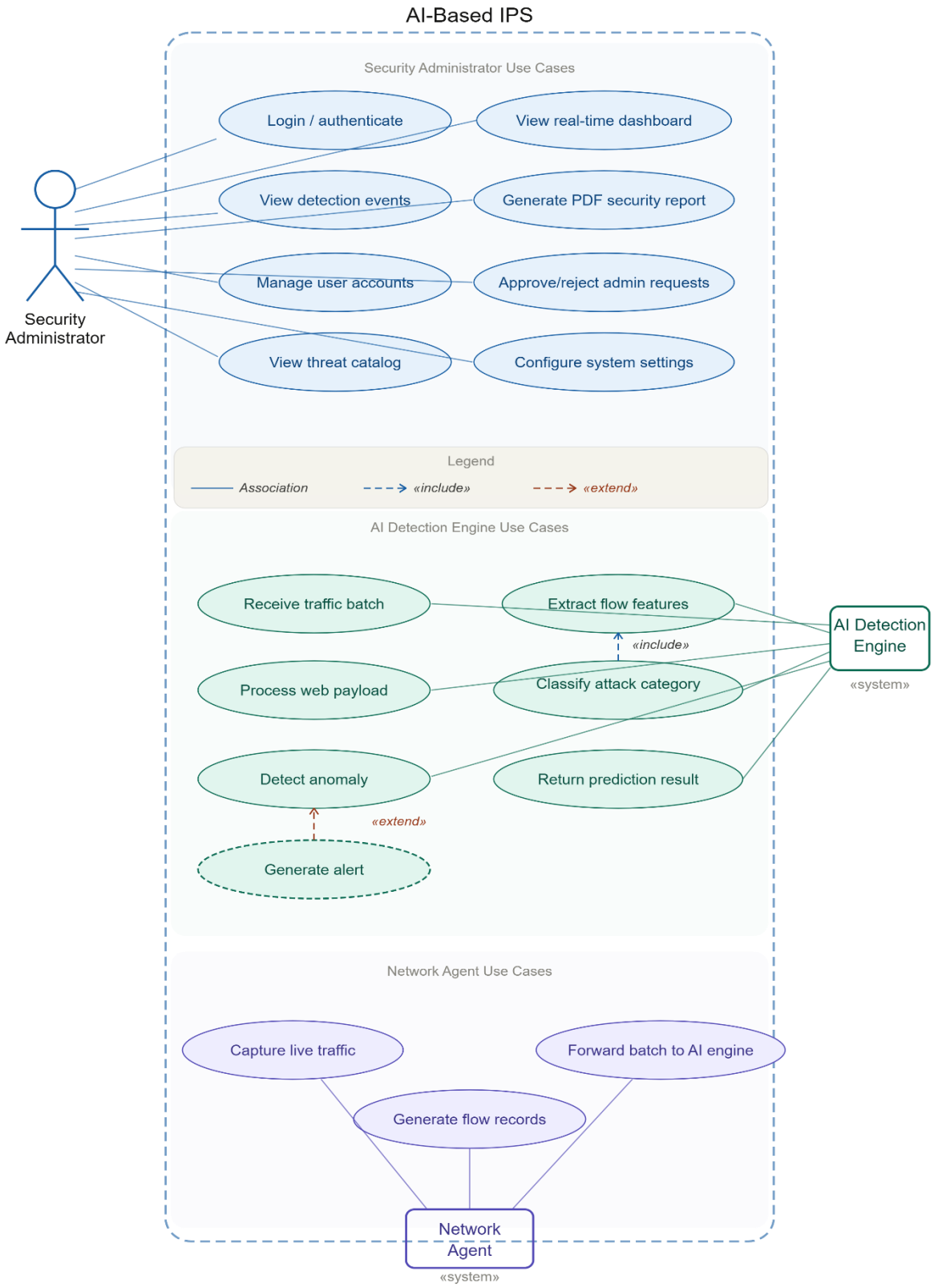

Figure 9: Use Case Diagram

3.10 Activity Diagrams

Activity diagrams describe the dynamic behavioral workflows of the system's key processes. Two primary workflows are modeled: the end-to-end traffic analysis and batch processing workflow, and the alert generation and notification workflow.

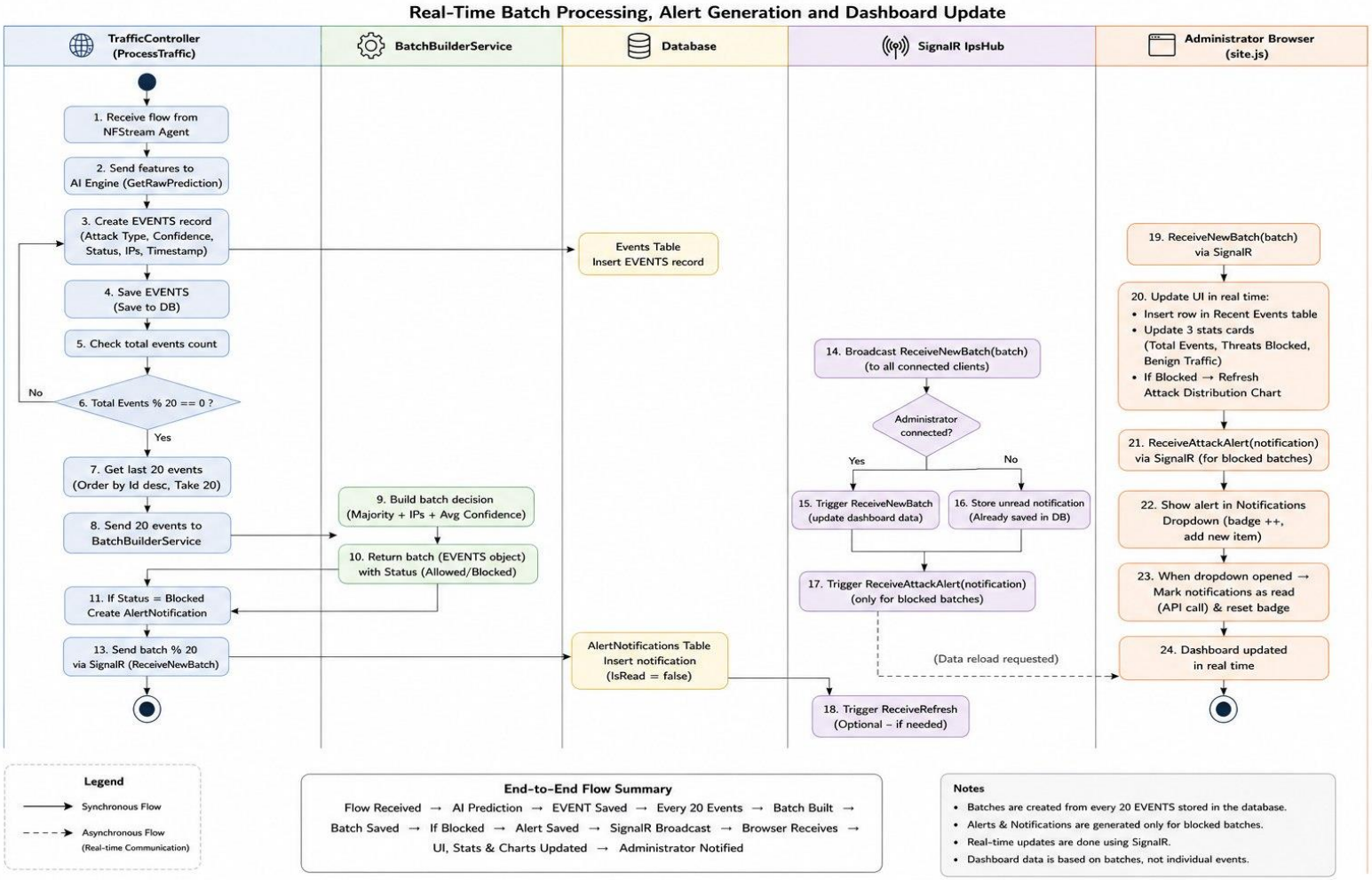

Figure 10: Activity Diagram

3.11 Sequence Diagrams

Sequence diagrams illustrate the time-ordered interactions between system components for key operational scenarios. The primary sequence modeled is the end-to-end traffic processing and dashboard update cycle.

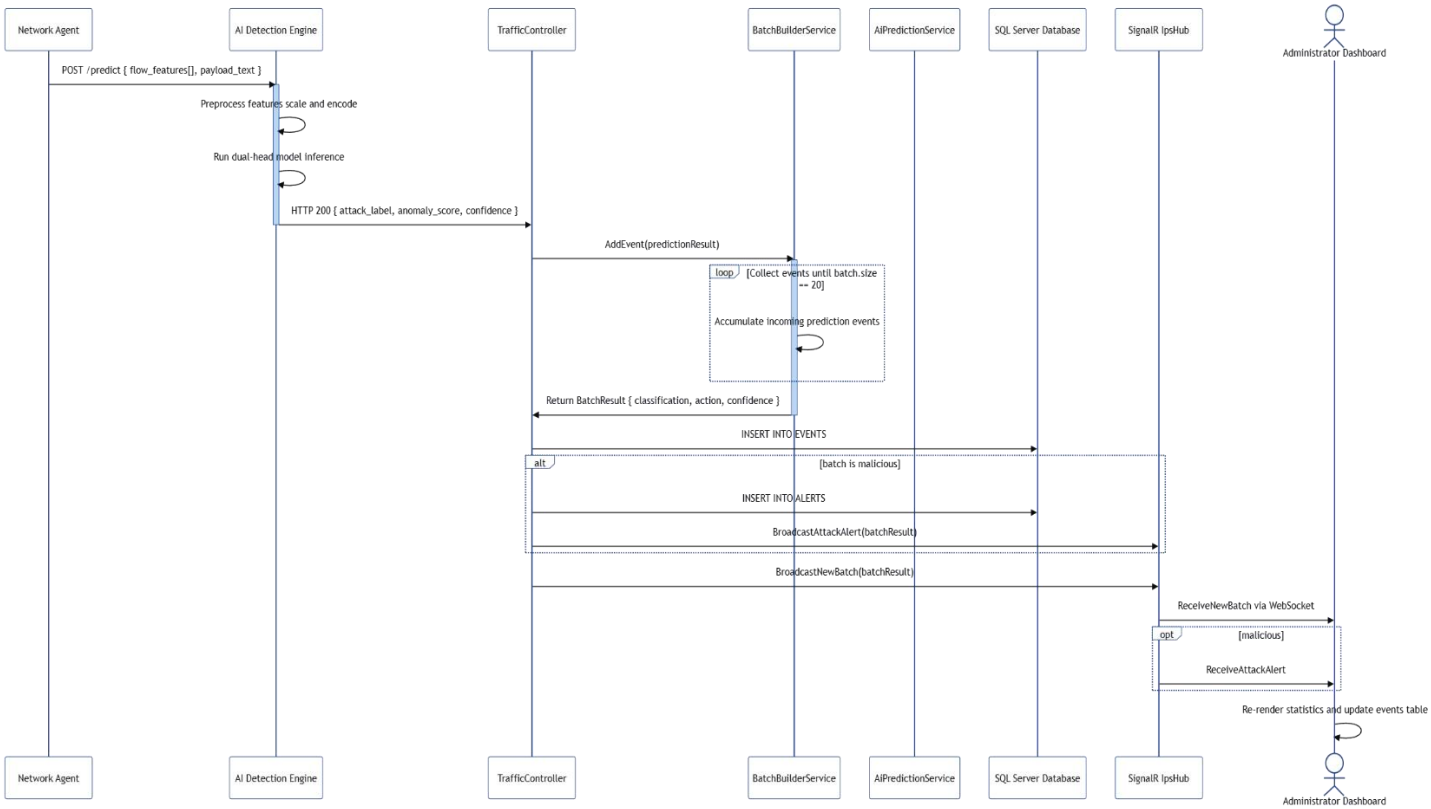

Figure 11: Sequence diagram

3.12 Dashboard Design

The dashboard serves as the primary operational interface for security administrators, providing centralized real-time visibility into the network's security posture. The design prioritizes clarity, information density, and responsiveness to ensure that critical security information is immediately accessible without navigating multiple screens.

The dashboard layout is organized into the following UI components:

- Summary Statistics Cards (top row): Three prominent cards display the total number of analyzed batches (Total Events), the number of malicious batches blocked (Threats Blocked), and the number of benign batches permitted (Benign Traffic). Each card uses color coding: blue for total events, red for threats, green for benign traffic.
- Recent Detection Events Table: A paginated table beneath the summary cards displays the most recent batch analysis results. Each row presents the timestamp, source IP address,

destination IP address, prediction label (attack type or Benign), confidence score, and enforcement action (Allowed/Blocked) with status badge color coding.

- Attack Distribution Chart: An interactive chart visualizes the distribution of detected attack categories across all malicious batches. This enables administrators to identify prevalent attack vectors and correlate detection patterns over time.
- Real-Time Update Mechanism: SignalR WebSocket connections ensure that all dashboard components refresh automatically when new batches are processed, eliminating the need for manual page reloads and providing near-zero-latency situational awareness.
- Navigation Bar: Provides access to the Alerts module, Reporting module, Threats catalog, User Management panel, and System Settings.

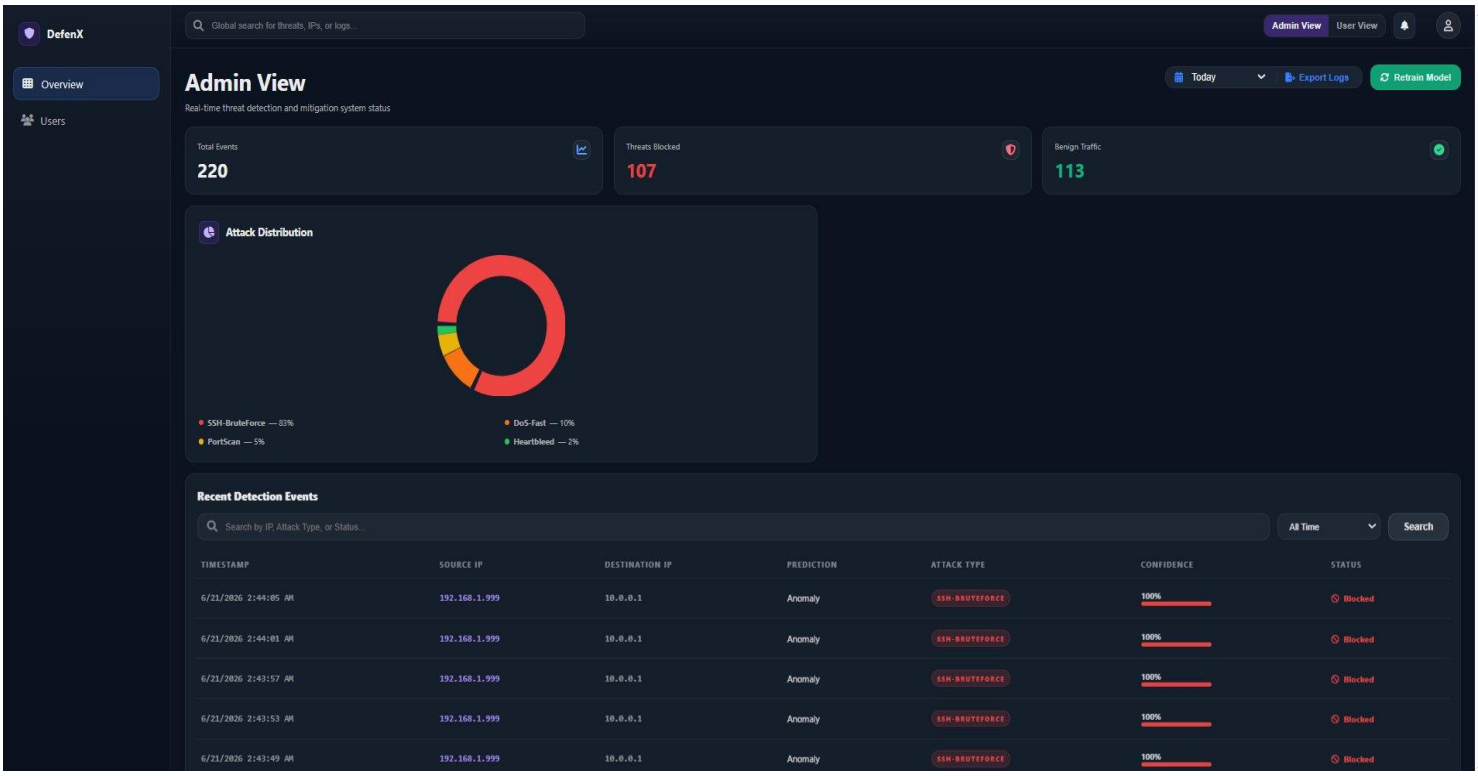

Figure 12: Dashboard UI

3.13 Security Design

Security is a fundamental design principle throughout the proposed AI-Based Intrusion Prevention System. The security architecture follows a defense-in-depth strategy in which multiple security layers work together to protect the network, application, infrastructure, and data. Each layer provides independent protection mechanisms to reduce the impact of potential security breaches and ensure continuous system operation.

Network Security

The enterprise laboratory environment incorporates multiple network security controls, including network segmentation, VLAN isolation, Access Control Lists (ACLs), FortiGate firewall policies, and a Demilitarized Zone (DMZ). The FortiGate firewall enforces zone-based access policies between the Internet, DMZ, internal LAN, and data center segments. Stateful packet inspection, NAT, traffic filtering, and route control mechanisms help prevent unauthorized access and limit lateral movement within the network.

The network architecture follows a layered design in which public-facing services are isolated within the DMZ, while critical infrastructure components such as Active Directory, DNS, DHCP, and file servers remain protected inside the internal network. This separation reduces the attack surface and limits the impact of potential compromises.

AI Data Integrity and Model Security

To maintain the reliability of detection decisions, all AI predictions are generated using a centrally managed deployment model. Prediction results are recorded with confidence scores and timestamps to support traceability and forensic analysis. Training datasets undergo preprocessing and validation procedures before model training to reduce labeling errors and improve prediction reliability. Additionally, the separation of the AI engine from the web application allows independent model updates without affecting operational system availability.

Intrusion Prevention and Blocking Security

The primary prevention mechanism of the proposed system is implemented through a dedicated desktop-based IPS service integrated with the AI detection engine and web management platform. When malicious traffic is identified, the source IP address is added to a centralized blacklist managed through the ASP.NET Core dashboard.

The web application synchronizes blocking and unblocking actions with the desktop IPS service using authenticated HTTP requests. The desktop service receives these commands through a local HTTP listener and applies network-level enforcement using the WinDivert packet filtering driver.

When a malicious IP address is blocked, WinDivert dynamically creates filtering rules that prevent packets originating from the identified source from reaching the host system. When an

administrator removes an address from the blacklist, the corresponding filtering rules are automatically removed, restoring normal network communication. This approach enables immediate and automated response to detected threats without requiring manual intervention.

Application Authentication and Authorization

The web application implements secure user authentication and authorization using ASP.NET Identity. User passwords are securely hashed and stored using industry-standard cryptographic techniques. Email verification is required before account activation to prevent unauthorized account creation.

Role-Based Access Control (RBAC) is used to restrict access to sensitive system functions. Different privilege levels are assigned to users based on their responsibilities, ensuring that administrative operations such as blacklist management, user management, and report generation can only be performed by authorized personnel. Administrative privilege requests are processed through a dedicated approval workflow to maintain accountability and control.

Inter-Component Communication Security

Communication between system components is secured through controlled API interactions and network isolation mechanisms. The AI Detection Engine, Web Application, Database Server, and Desktop IPS Service communicate using authenticated service interfaces and structured JSON messages.

Within the deployment environment, communication between containers occurs through private Kubernetes networking. External access to the web application is provided through HTTPS, ensuring encrypted communication between clients and the server. Database services are not exposed directly to external networks and can only be accessed by authorized application components through controlled security group configurations.

Data Security and Database Protection

The system stores operational information, user accounts, security events, alerts, threat records, and blacklist entries within Microsoft SQL Server. Access to database resources is restricted through application-level authorization and infrastructure-level security controls.

Sensitive information is protected through controlled access policies, while audit records and event histories are preserved to support incident investigation and forensic analysis. Database backup and recovery mechanisms ensure data availability and business continuity in the event of system failures.

Container and Infrastructure Security

The deployment environment utilizes Docker containers and Kubernetes orchestration to isolate application components and reduce the impact of potential compromises. Each major subsystem operates independently, limiting direct access between services and improving overall resilience.

Before deployment, container images are automatically scanned using Trivy to identify known Common Vulnerabilities and Exposures (CVEs). Security scanning is integrated into the GitHub Actions CI/CD pipeline, ensuring that only verified images are deployed into the production environment.

The cloud infrastructure is hosted on Amazon Web Services (AWS), where application services run on AWS EC2 instances while the database operates independently on AWS RDS. Network-level protections, security groups, and access restrictions further strengthen the security posture of the deployed environment.

Monitoring, Logging, and Auditability

Continuous monitoring is provided through Prometheus and Grafana, which collect and visualize infrastructure and application performance metrics. These tools enable administrators to identify abnormal behavior, resource bottlenecks, and operational issues in real time.

At the application level, the IPS maintains detailed logs of security events, attack detections, blacklist operations, administrative actions, and system activities. SignalR-based notifications provide immediate visibility into newly detected threats and blocking actions through the monitoring dashboard.

The combination of infrastructure monitoring, security event logging, audit trails, and real-time alerting supports incident response, forensic investigation, and long-term security analysis while maintaining accountability across all system operations.

CHAPTER 4: IMPLEMENTATION

4.1 Development Environment

The development and testing of the AI-Based Intrusion Prevention System was conducted across multiple machines and virtual environments. The following describes the primary development environment configuration.

Component	Specification / Tool
Operating System	Windows 10 Pro (host) with Ubuntu 22.04 LTS (WSL2 / VM) for AI and DevOps workloads
Virtualization Platform	VMware Workstation 17 Pro — used to run Windows Server 2022, Windows 10 clients, Kali Linux, and Metasploit VMs for network simulation and attack generation
Network Simulator	GNS3 (Graphical Network Simulator-3) v2.2 — used to emulate Cisco 7200 routers, Cisco switches, and multi-tier network topology with OSPF, VLANs, and FortiGate firewall integration
IDE — Web App	Microsoft Visual Studio 2022 with ASP.NET Core workload, Entity Framework Core tools, and C# 14
IDE — AI Engine	Visual Studio Code with Python 3.10, Jupyter Notebook extension, TensorFlow/Keras plugin
Source Control	Git with GitHub repository; branches per feature with pull-request-based merges into main
Package Management	NuGet (C#/.NET), pip (Python), npm (JavaScript)
Database Tool	SQL Server Management Studio (SSMS) 19 for schema design, query testing, and migration verification
Container Runtime	Docker Desktop 4.x — for building and testing containerized images locally before cloud deployment
Cloud Environment	Amazon Web Services — EC2 (t3.medium) for application and AI containers, RDS (SQL Server Express) for managed database hosting

Table 2: Development environment configuration

4.2 Technologies and Tools

The system was implemented using a carefully selected stack of technologies chosen for their maturity, community support, and suitability for enterprise security applications. Each technology selection is justified below.

Backend and Web Application

- ASP.NET Core MVC 10.0 — Chosen for its high performance on Linux and Windows, mature ecosystem, and native integration with Entity Framework Core and SignalR. Its Model-View-Controller architecture enforces clean separation of concerns across the application.
- Entity Framework Core 10 — Used as the ORM layer to manage database migrations, schema versioning, and type-safe LINQ-based queries. Code-first migrations enabled iterative database design alongside application development.
- ASP.NET Core Identity — Provides a production-grade authentication and authorization framework with built-in support for password hashing, token generation, email verification, and role management.
- SignalR — Microsoft's real-time communication library enabling full-duplex WebSocket connections between the server and connected dashboard clients. Used to implement IpsHub (traffic notifications) and SecurityHub (system status updates).
- QuestPDF — A fluent C# API for generating structured PDF reports. Selected for its code-first approach, eliminating the need for external report design tools while producing professional-grade output.

AI and Machine Learning

- Python 3.10 — Primary language for AI development, data preprocessing, feature engineering, and model training.
- TensorFlow 2.x / Keras — Deep learning framework used to define, train, and serialize the multimodal dual-head neural network. The Functional API was used to construct the branching multi-input, multi-output architecture.
- NFStream 6.x — Network flow extraction library used for both offline PCAP processing and live traffic monitoring. Provides 80+ flow-level features per bidirectional flow.
- scikit-learn — Used for data preprocessing operations including StandardScaler normalization, label encoding, train-test splitting, and evaluation metric computation.
- Pandas / NumPy — Core data manipulation libraries used throughout the preprocessing and feature engineering pipeline.

DevOps and Infrastructure

- Docker — All deployable components (web application and AI engine) are packaged as Docker images using multi-stage build Dockerfiles to minimize final image size and attack surface.
- Kubernetes (k3s) — Lightweight Kubernetes distribution deployed on the AWS EC2 instance. Manages pod lifecycle, health checks via liveness and readiness probes, and automatic pod restarts on failure.
- GitHub Actions — CI/CD automation platform. Pipelines trigger on push to the main branch, executing build, test, Trivy security scan, Docker image publish, and Kubernetes rolling deployment steps.
- Amazon Web Services (AWS) — EC2 (t3.medium) hosts the Kubernetes cluster; RDS SQL Server hosts the production database. Security Groups restrict database access to the EC2 instance's private IP only.
- Prometheus & Grafana — Prometheus scrapes metrics from application pods; Grafana renders them as operational dashboards. Alerts are configured for abnormal CPU spikes and pod crash-restart events.
- Trivy — Open-source container image vulnerability scanner integrated into the GitHub Actions pipeline. Images with HIGH or CRITICAL CVEs are blocked from deployment.

Network and Security Tools

- FortiGate Firewall — Enterprise-grade Next-Generation Firewall (NGFW) configured with zone-based policies (LAN, WAN, DMZ), NAT, OSPF integration, and IPS signature modules.
- GNS3 with Cisco 7200 IOS — Network emulation platform hosting the multi-tier enterprise topology with realistic routing protocol behavior.
- Kali Linux / Metasploit Framework — Used during the traffic generation phase to execute controlled attack scenarios (DDoS, DoS, Brute Force, Port Scan, SQLi, XSS) for training data acquisition and system validation.
- Burp Suite, SQLMap, Hydra, LOIC, Wireshark - Attack-specific tools used to generate diverse, realistic attack traffic samples captured as PCAP files for AI model training.

4.3 Network Implementation

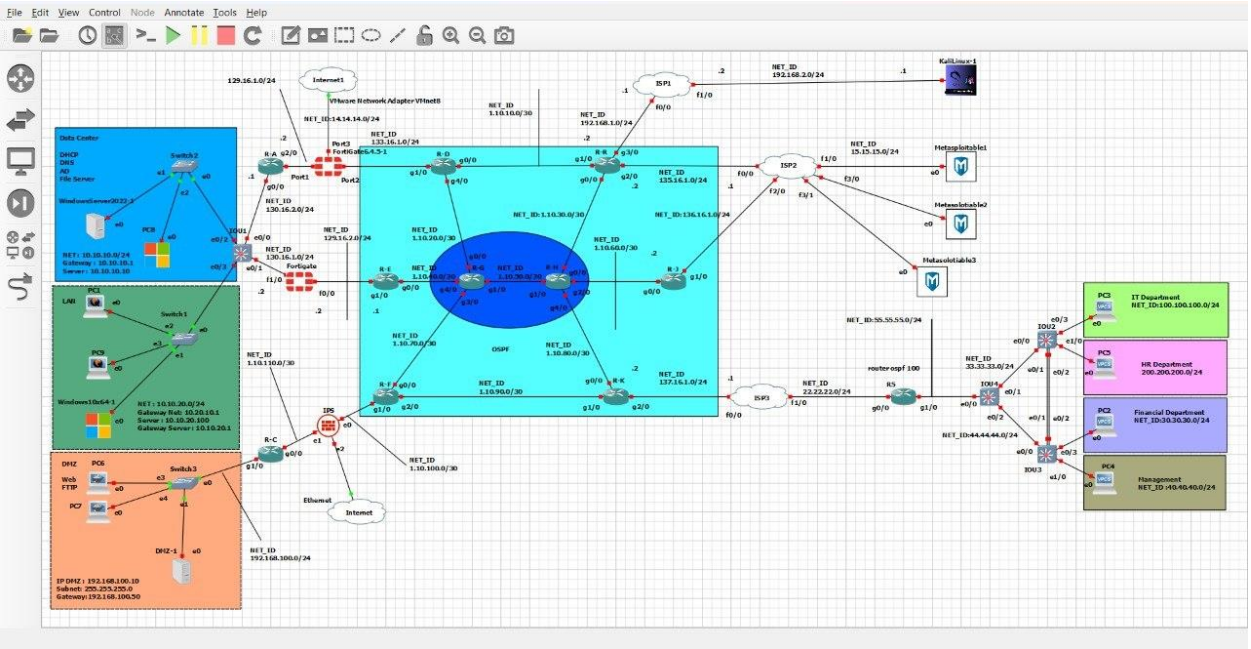

Figure 13: Network Topology

The network infrastructure was implemented using a combination of GNS3 and VMware Workstation to create a realistic enterprise penetration testing environment. The implementation follows the hierarchical network architecture presented in Chapter 3 and integrates enterprise services, security controls, and attack simulation platforms.

The implementation consists of multiple interconnected zones:

Internal Corporate Network

The internal network contains user workstations and administrative systems connected through virtual switches and routers. Windows 10 virtual machines were deployed to simulate employee endpoints and generate legitimate network traffic.

VLAN segmentation was implemented to separate departments and reduce unnecessary communication between network segments.

Data Center Deployment

The Data Center was implemented using Windows Server 2022.

The following enterprise services were configured:

- Active Directory Domain Services (AD DS)
- Domain Name System (DNS)

- Dynamic Host Configuration Protocol (DHCP)
- File Sharing Services
- Group Policy Management

The Domain Controller provides centralized authentication and policy enforcement across the network.

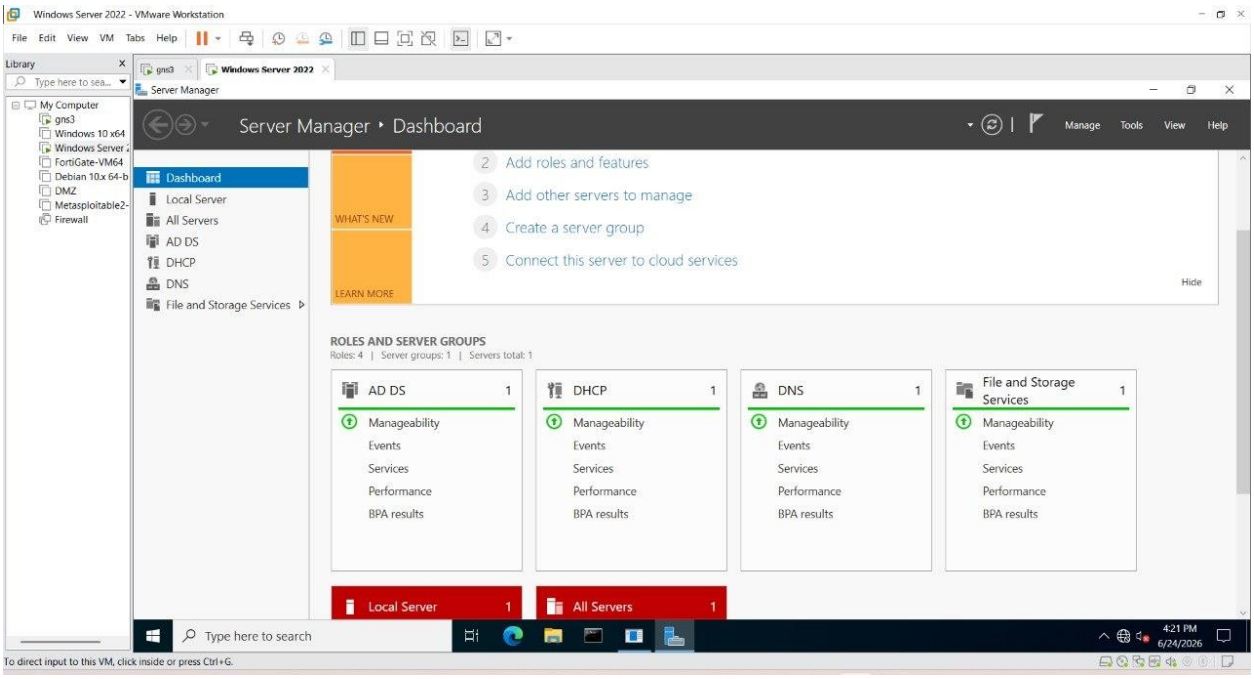

Figure 14: Windows Server Configuration

DMZ Deployment

The DMZ was deployed as an isolated network segment separated by firewall policies.

Services deployed in the DMZ include:

- IIS Web Server
- FTP Server
- Remote Desktop Services

The DMZ allows controlled external access while preventing direct communication with internal corporate assets.

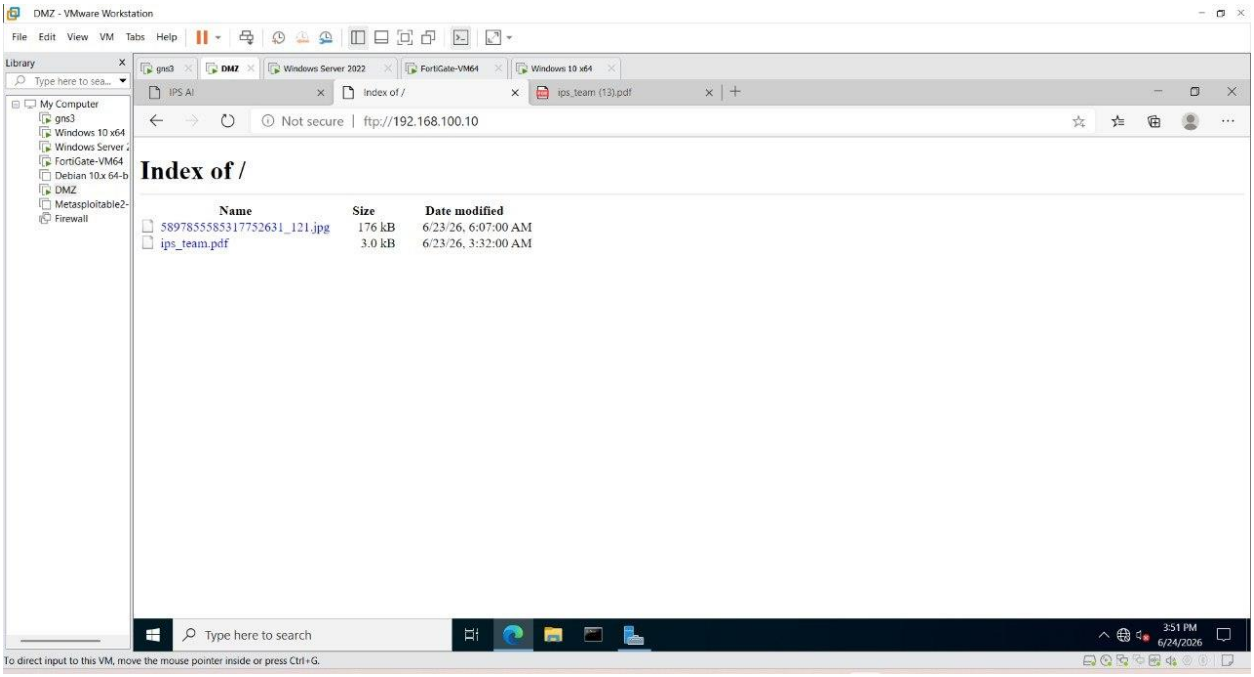

Figure 15: DMZ Implementation and Public Services (FTP)

FortiGate Firewall Configuration

A FortiGate firewall was configured as a perimeter security device.

The firewall performs:

- Traffic filtering
- NAT
- Logging
- Security monitoring
- Intrusion Prevention

Multiple firewall policies were created to regulate traffic between:

- WAN and DMZ
- WAN and Internal Network
- DMZ and Internal Network
- Internal Network and Data Center

OSPF routing was configured to support dynamic route exchange across network segments.

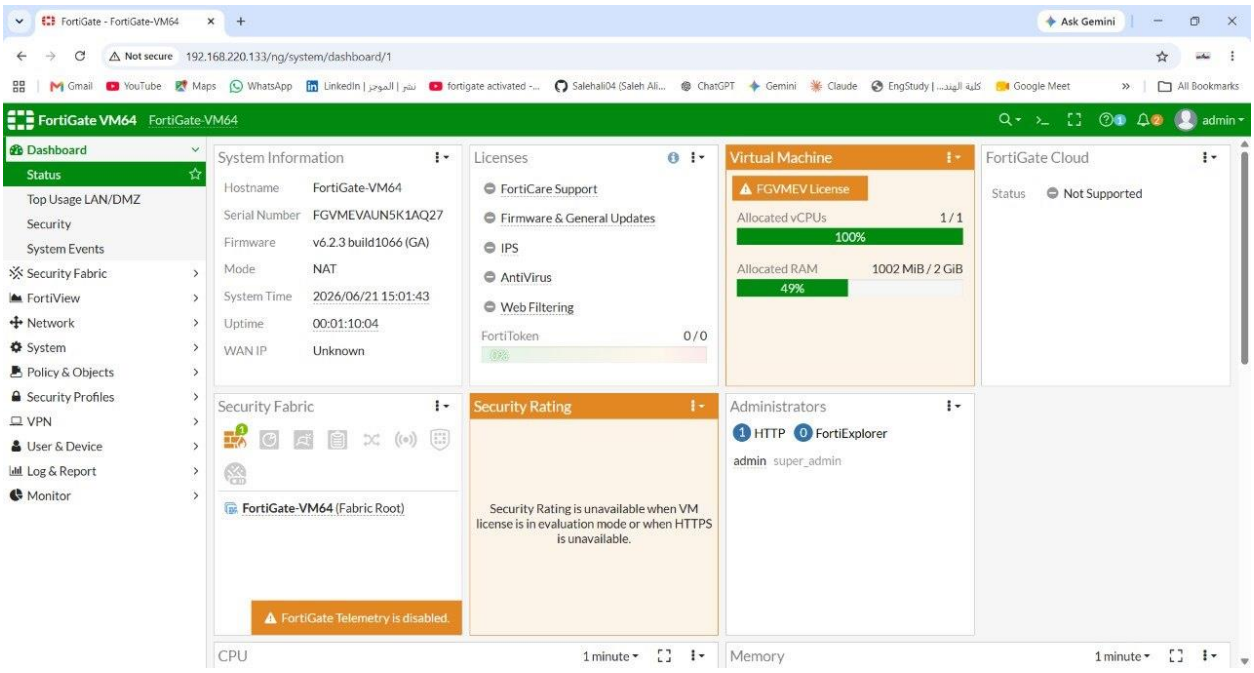

Figure 16: FortiGate Configuration

Attack Environment Deployment

A dedicated attacker network was created using Kali Linux and Metasploit Framework virtual machines.

These machines were used to generate:

- Port scanning traffic
- Brute-force attacks
- SQL Injection attempts
- XSS attacks
- DoS attacks
- DDoS attacks

The generated traffic was captured and analyzed by the AI-Based IPS for testing and validation purposes.

IPS Integration

The AI-Based IPS was integrated directly into the network environment.

Traffic generated across the network is:

1. Captured using NFStream.

2. Converted into network flow records.
3. Processed by the AI Detection Engine.
4. Classified as benign or malicious.
5. Forwarded to the prevention engine.
6. Displayed on the monitoring dashboard.
7. Automatically blocked when malicious behavior is confirmed.

This integration allows real-time monitoring and automated response against cyber threats within the simulated enterprise environment.

4.4 Traffic Capture Module

The Traffic Capture Module serves as the entry point of the proposed AI-Based Intrusion Prevention System. Its primary responsibility is to acquire raw network traffic and convert it into structured flow records suitable for feature extraction, machine learning analysis, and real-time intrusion detection.

To improve model robustness and reduce dependency on a single traffic source, traffic was collected from two complementary datasets:

1. The publicly available CICIDS2017 benchmark dataset.
2. A custom enterprise traffic dataset generated within the penetration testing laboratory developed for this project.

Both datasets were processed using a unified NFStream-based feature extraction pipeline to ensure feature consistency throughout the system.

4.4.1 CICIDS2017 Benchmark Dataset

The Canadian Institute for Cybersecurity Intrusion Detection System 2017 (CICIDS2017) dataset was selected as the primary benchmark dataset for initial model development and evaluation. CICIDS2017 is one of the most widely used intrusion detection datasets in cybersecurity research and contains both benign and malicious traffic captured in a controlled network environment.

The dataset includes multiple attack categories such as:

- DDoS Attacks
- DoS Hulk
- DoS GoldenEye

- DoS Slowloris
- FTP Brute Force
- SSH Brute Force
- Port Scanning
- Web Attacks

The dataset provides a valuable benchmark for comparing results against existing intrusion detection research. However, several limitations were identified:

- The dataset was generated in 2017 and may not fully represent modern attack behaviors.
- Traffic was collected in a controlled laboratory environment.
- Certain application-layer attack scenarios are underrepresented.
- Models trained exclusively on CICIDS2017 may learn dataset-specific characteristics rather than generalized attack behavior.

To address these limitations, an additional custom traffic collection phase was conducted.

4.4.2 Custom Enterprise Traffic Collection

A custom enterprise penetration testing laboratory was developed to generate realistic attack traffic under controlled conditions. The laboratory simulated a complete enterprise network consisting of:

- Active Directory Infrastructure
- DNS Services
- DHCP Services
- File Sharing Services
- Web Servers
- FTP Servers
- VLAN Segmentation
- DMZ Infrastructure
- OSPF-Based Routing

Traffic captures were collected using Wireshark from multiple attack execution sessions.

More than fifty PCAP files were generated and organized according to attack category and execution timestamp.

The attack scenarios included:

Category	Attack Type
DDoS	LOIC
DoS	Hulk
DoS	GoldenEye
DoS	Slowloris
Brute Force	FTP Brute Force
Brute Force	SSH Brute Force
Reconnaissance	Port Scan
Web Attacks	SQL Injection
Web Attacks	Cross-Site Scripting (XSS)

Table 3: Attack Senarios

The use of a realistic enterprise environment provided traffic patterns that differed significantly from benchmark datasets and enabled evaluation of the system under conditions closer to real-world deployment scenarios.

4.4.3 Unified NFStream Processing Pipeline

Although CICIDS2017 originally provides features extracted using CICFlowMeter, all benchmark traffic and custom-generated PCAP files were reprocessed using NFStream.

This decision was made to eliminate feature inconsistencies between datasets and ensure that all training and testing samples were generated using the same extraction methodology.

NFStream was selected after comparative evaluation against CICFlowMeter for several reasons:

- More accurate bidirectional flow reconstruction.
- Better handling of long-lived connections.
- Reduced flow fragmentation.
- Native Python integration.
- Efficient real-time deployment capabilities.
- Built-in nDPI protocol identification.

Using a unified extraction pipeline ensures that the machine learning model learns behavioral characteristics of attacks rather than artifacts introduced by different feature extraction tools.

4.4.4 NFStream-Based Live Traffic Collection

During deployment, NFStream continuously monitors the selected network interface and reconstructs bidirectional flows from captured packets in real time.

Each flow remains active until either:

- A TCP FIN/RST packet terminates the session, or
- The configured timeout threshold is reached.

After flow completion, NFStream generates a structured flow record containing statistical, temporal, directional, and protocol-related features.

The following implementation illustrates the live traffic collection agent responsible for capturing flows and forwarding them to the AI detection engine.

```
# NFStream-Based Traffic Collection Agent
from nfstream import NFStreamer
import requests
import time

API_URL = "http://localhost:5111/api/Traffic/ProcessTraffic"

INTERFACE = r"\Device\NPF_{1E55FFAD-B898-41FB-99DD-FA798249767F}"

def main():

    streamer = NFStreamer(
        source=INTERFACE,
        statistical_analysis=True,
        active_timeout=60,
        idle_timeout=10
    )

    for flow in streamer:

        flow_data = {
            # Extracted NFStream features
        }

        payload = {
            "source_ip": flow.src_ip,
            "destination_ip": flow.dst_ip,
            "protocol": int(flow.protocol),
            "data": flow_data
        }
```

```
        requests.post(API_URL, json=payload)

if __name__ == "__main__":
    main()
```

4.4.5 Traffic Forwarding and Event Generation

After flow extraction, the traffic collection agent forwards each completed flow to the AI Detection Engine through a REST API request.

The ASP.NET Core backend receives the flow, validates the incoming data, applies preprocessing, and forwards the extracted features to the deployed AI model for inference.

The AI engine returns:

- Predicted Attack Class
- Classification Confidence
- Anomaly Probability
- Binary Anomaly Status

The backend then creates a security event record and stores the result in the database for visualization and further analysis.

The following controller implementation illustrates the traffic processing workflow.

```
[HttpPost("ProcessTraffic")]
public async Task<IActionResult> ProcessTraffic([FromBody] TrafficIncomingRequest
incoming)
{
    try
    {
        // ========================================
        // Step 1: Perform DDoS Protection Check
        // ========================================
        var ddosCheck = await
_ddosProtection.CheckAndProtect(incoming.source_ip);

        // If the source IP is blocked due to DDoS activity,
        // return the appropriate response immediately.
        if (ddosCheck.IsBlocked)
        {
            return StatusCode(
                ddosCheck.StatusCode,
                new
```

```
            {
                error = ddosCheck.Reason,
                ip = incoming.source_ip
            }
        );
    }

    // ========================================
    // Step 2: Validate Incoming Request
    // ========================================
    if (incoming == null || incoming.data == null)
    {
        return BadRequest(new { error = "Missing traffic data" });
    }

    // ========================================
    // Step 3: Convert JSON Features to Numeric Values
    // ========================================
    var cleanedData = new Dictionary<string, double>();

    foreach (var item in incoming.data)
    {
        if (item.Value is JsonElement element)
        {
            if (element.ValueKind == JsonValueKind.True)
                cleanedData[item.Key] = 1.0;

            else if (element.ValueKind == JsonValueKind.False)
                cleanedData[item.Key] = 0.0;

            else if (element.ValueKind == JsonValueKind.Number)
                cleanedData[item.Key] = element.GetDouble();
        }
    }

    // Ensure protocol information exists
    if (!cleanedData.ContainsKey("protocol"))
    {
        cleanedData["protocol"] = (double)incoming.protocol;
    }

    // ========================================
    // Step 4: Send Features to AI Model
    // ========================================
    var rawResult = await _aiService.GetRawPredictionAsync(cleanedData);
```

```
        // Parse the JSON response returned by the AI model
        using var doc = JsonDocument.Parse(rawResult);
        var root = doc.RootElement;

        // Handle single-object or array-based responses
        var result = root.ValueKind == JsonValueKind.Array
            ? root[0]
            : root;

        // Check for model-side errors
        if (result.TryGetProperty("error", out var errorProp))
        {
            return BadRequest(new
            {
                error = errorProp.GetString()
            });
        }

        // ========================================
        // Step 5: Extract AI Prediction Results
        // ========================================
        bool isAnomaly =
            result.GetProperty("is_anomaly").GetBoolean();

        string modelAttackType =
            result.GetProperty("predicted_class").GetString()
            ?? "Unknown";

        double confidenceValue =
            result.GetProperty("class_confidence").GetDouble();

        double anomalyScore =
            result.GetProperty("anomaly_score").GetDouble();

        // ========================================
        // Step 6: Determine Final Attack Classification
        // ========================================
        string attackType;

        if (!isAnomaly)
        {
            attackType = "Benign";
        }
        else if (confidenceValue > 50)
```

```
        {
            attackType = modelAttackType;
        }
        else
        {
            attackType = "Unknown Attack";
        }

        // ========================================
        // Step 7: Create Security Event Record
        // ========================================
        var trafficEvent = new EVENTS
        {
            SourceIp = incoming.source_ip ?? "Unknown",
            DestinationIp = incoming.destination_ip ?? "Unknown",

            // AI prediction result
            Prediction = isAnomaly
                ? "Anomaly"
                : "Non Anomaly",

            // Attack category assigned by the system
            AttackType = attackType,

            // Model confidence score
            Confidence = confidenceValue,

            // Prevention decision
            Status = isAnomaly
                ? "Blocked"
                : "Allowed",

            // Event timestamp
            Timestamp = DateTime.Now
        };

        // ========================================
        // Step 8: Store Event in Database
        // ========================================
        _context.Events.Add(trafficEvent);
        await _context.SaveChangesAsync();

        // ========================================
        // Step 9: Return Detection Result
        // ========================================
```

```
        return Ok(new
        {
            prediction = trafficEvent.Prediction,
            attack_type = trafficEvent.AttackType,
            confidence = confidenceValue,
            anomaly_score = anomalyScore,
            status = trafficEvent.Status
        });
    }
    catch (Exception ex)
    {
        // Handle unexpected server-side errors
        return StatusCode(500, new
        {
            error = ex.Message
        });
    }
}
```

4.4.6 Event Aggregation Trigger

Rather than making prevention decisions based on individual flow predictions, the system performs batch-level correlation analysis.

Each flow prediction is first stored as an event record. After every twenty processed events, the system automatically retrieves the latest twenty classified records and constructs an aggregation batch.

This batch is subsequently forwarded to the Prevention and Decision Engine, where attack frequencies, source identifiers, and confidence distributions are analyzed before any blocking action is issued.

The following implementation demonstrates the batch-trigger mechanism.

```
if (totalEvents % 20 == 0)
{
    // Retrieve the latest 20 traffic events from the database
    var last20Events = await _context.Events
        .OrderByDescending(x => x.Id)
        .Take(20)
        .ToListAsync();

    // Build a batch dataset that will be used
    // for advanced AI analysis and attack correlation
    var batch = _batchBuilder.BuildBatch(last20Events);
```

```
    // The generated batch can then be forwarded
    // to the AI engine for batch-level threat detection
}
```

4.5 Dataset Construction and Augmentation

4.5.1 CICIDS2017 Dataset Reprocessing

The CICIDS2017 dataset was selected as the primary benchmark dataset due to its wide adoption within intrusion detection research and its inclusion of multiple attack categories relevant to enterprise environments. However, the original CICIDS2017 feature set was generated using CICFlowMeter, while the proposed IPS relies on NFStream for feature extraction during deployment.

Using different feature extraction engines during training and deployment introduces feature inconsistency and may negatively impact model performance. To eliminate this discrepancy, the original CICIDS2017 PCAP files were reprocessed using NFStream. This ensured that both training and deployment environments relied on identical feature extraction logic.

The reprocessed dataset included benign traffic and multiple attack categories including DDoS, DoS, Port Scanning, FTP Brute Force, and SSH Brute Force attacks. All flows were reconstructed using NFStream's bidirectional flow engine and exported as structured flow records.

4.5.2 Custom Enterprise Traffic Dataset

Although benchmark datasets provide valuable training data, they cannot fully represent modern enterprise environments. Therefore, a custom enterprise traffic dataset was collected within the laboratory environment developed for this project.

The environment consisted of:

- VMware Workstation virtual machines
- GNS3 network simulation
- Windows Server 2022
- Active Directory Domain Services
- DNS and DHCP services
- DMZ servers
- Kali Linux attacker systems

- Metasploit Framework

More than 50 PCAP capture sessions were collected using Wireshark during controlled attack executions.

The collected traffic included:

- Benign Web Browsing
- DNS Queries
- File Transfers
- Port Scanning
- FTP Brute Force
- SSH Brute Force
- SQL Injection
- Cross-Site Scripting (XSS)
- DDoS-LOIC
- DoS-HULK
- DoS-GoldenEye
- Slowloris

All PCAP files were processed using NFStream to maintain feature consistency with the benchmark dataset.

4.5.3 Dataset Challenges

Several challenges were encountered during dataset construction.

Class Imbalance

Network traffic datasets naturally contain significantly more benign traffic than malicious traffic. This imbalance may bias machine learning models toward majority classes.

Environment Bias

Features such as IP addresses, MAC addresses, and network identifiers may cause models to memorize specific environments rather than learning attack behaviors.

Dataset Drift

Traffic patterns found in benchmark datasets collected in 2017 may differ significantly from modern enterprise traffic.

Overfitting Risk

Models trained solely on benchmark datasets may learn dataset-specific characteristics and fail to generalize to unseen environments.

To address these challenges, an additional synthetic data generation phase was introduced.

4.5.4 Experimental Evaluation of Synthetic Data Generation

4.5.4.1 CTGAN Evaluation

The first synthetic data generation approach utilized Conditional Tabular Generative Adversarial Networks (CTGAN).

Prior to training, SPLT sequences were expanded into fixed-length vectors and application-layer identifiers were removed. K-Means clustering was used to create balanced subsets for training. The CTGAN model was trained for 150 epochs.

Although training completed successfully, validation revealed several critical issues:

- Invalid timestamp relationships
- Impossible packet size values
- Corrupted SPLT sequences
- Integer overflow problems
- Loss of protocol-specific behavior

As a result, CTGAN-generated traffic was deemed unsuitable for IPS training.

4.5.4.2 Gaussian Mixture Model Evaluation

A second approach utilized Gaussian Mixture Models (GMMs) to model traffic distributions statistically.

However, GMM-generated traffic exhibited several limitations:

- Poor representation of highly skewed traffic distributions
- Loss of attack-specific boundaries
- Fractional protocol identifiers
- Unrealistic network behavior

Consequently, GMM-based generation was also rejected.

4.5.5 Final Synthetic Dataset Generation Method

A cluster-based bootstrapping methodology with controlled statistical perturbation was ultimately adopted.

Phase 1: Template Extraction

K-Means clustering (k=10) was used to identify representative behavioral templates from the baseline dataset.

Phase 2: Bootstrapping

Traffic templates were sampled with replacement until the dataset reached 50,000 records.

Phase 3: Statistical Perturbation

Continuous numerical features were modified using random noise within ±15%.

This introduced realistic variability while preserving attack behavior.

Phase 4: Identifier Synthesis

Synthetic values were generated for:

- IPv4 Addresses
- MAC Addresses
- Source Ports
- Destination Ports
- Timestamps

All generated values maintained valid network relationships.

The resulting synthetic dataset successfully increased diversity while preserving structural correctness.

4.5.6 Final Training Dataset Composition

The final training dataset consisted of four primary sources.

Dataset Source	Purpose
Reprocessed CICIDS2017	Benchmark traffic
Custom Enterprise Dataset	Realistic enterprise traffic
Synthetic Dataset	Generalization enhancement
Web Payload Dataset	Application-layer attack detection

Table 4: Final Training Dataset Composition

Combining these datasets improved robustness and reduced dependence on any single data source.

4.5.7 Comparison Between CICIDS2017 and Custom Enterprise Dataset

Aspect	CICIDS2017 Dataset	Custom Dataset
Collection Year	2017	2025–2026
Traffic Source	Public benchmark dataset	Real traffic generated in a controlled test environment
Feature Extraction Method	Original PCAPs reprocessed using NFStream	Captured and processed using NFStream
Attack Categories	Common network attacks (DoS, DDoS, Brute Force, Port Scan, etc.)	Network and web attacks including DoS, DDoS, Brute Force, Port Scan, SQL Injection, and XSS
Traffic Diversity	Fixed benchmark scenarios	Multiple attack sessions and traffic conditions
Application-Layer Attacks	Limited	Included
Dataset Control	Predefined benchmark data	Fully controlled traffic generation and labeling
Relevance to Current Deployments	May not fully reflect modern traffic patterns	Reflects contemporary attack execution techniques
Primary Purpose	Benchmarking and model comparison	Model training, validation, and deployment testing

Table 5: CICIDS2017 vs Custom Enterprise Dataset

The CICIDS2017 dataset provided a standardized benchmark for evaluating model performance and facilitating comparison with previous research. However, benchmark datasets alone may not adequately represent the variability of modern network environments. Therefore, a complementary custom dataset was collected and processed using the same NFStream feature extraction pipeline. Combining both datasets enabled the model to benefit from the reproducibility of a public benchmark while also learning from more diverse and contemporary attack scenarios, improving overall generalization capability.

4.6 Feature Extraction and Preprocessing Module

The Feature Extraction and Preprocessing Module transforms raw NFStream flow records into structured numerical and textual representations suitable for machine learning analysis. To ensure consistency between training, testing, and deployment environments, all traffic sources—including CICIDS2017 captures, custom-generated traffic, synthetic datasets, and live network traffic—are processed through the same preprocessing pipeline.

The module consists of two parallel processing pipelines: a network-flow feature pipeline and a web-payload text pipeline. This design enables the AI model to simultaneously analyze transport-layer behavior and application-layer content.

4.6.1 Network Flow Feature Pipeline

NFStream reconstructs bidirectional flows from raw packet streams and extracts a rich set of statistical, temporal, directional, and protocol-level attributes. These features are categorized into several groups:

Traffic Volume Features

- Total packet count
- Total byte count
- Source-to-destination packet and byte statistics
- Destination-to-source packet and byte statistics

These features help identify volumetric attack behaviors commonly observed in DDoS and DoS attacks.

Statistical Features

- Minimum packet size
- Maximum packet size
- Mean packet size
- Standard deviation of packet sizes
- Minimum packet inter-arrival time
- Maximum packet inter-arrival time
- Mean packet inter-arrival time
- Standard deviation of packet inter-arrival times

Statistical abnormalities often reveal scanning, flooding, and brute-force activities.

Temporal Features

- Flow duration
- Active time statistics
- Idle time statistics
- Connection timing characteristics

Temporal behavior assists in distinguishing attacks such as Slowloris from normal traffic.

Directional Features

- Independent source-to-destination statistics
- Independent destination-to-source statistics

These features capture asymmetric communication patterns frequently associated with reconnaissance and denial-of-service attacks.

TCP Behavioral Features

- SYN packet counts
- ACK packet counts
- PSH packet counts
- FIN packet counts
- RST packet counts

TCP flag distributions provide strong indicators of protocol abuse and malicious connection behavior.

4.6.2 Data Cleaning and Feature Selection

Prior to model training and inference, several preprocessing operations are applied:

- Removal of IP addresses and MAC addresses.
- Removal of source and destination port numbers.
- Removal of timestamps and flow identifiers.
- Removal of application names, user-agent strings, TLS fingerprints, and server names.
- Elimination of environment-specific attributes that could introduce data leakage.
- Validation of numerical feature ranges.
- Handling corrupted or incomplete records.

These operations prevent the model from memorizing network-specific characteristics and improve its ability to generalize to unseen environments.

4.6.3 Feature Engineering

Additional behavioral features are generated from the original NFStream statistics to improve attack discrimination.

Traffic Ratio Features

- Client-to-Server Byte Ratio
- SYN Packet Ratio
- ACK Packet Ratio
- FIN Packet Ratio
- RST Packet Ratio
- PSH Packet Ratio

Traffic Rate Features

- Packets Per Second
- Bytes Per Second
- Log-Scaled Packet Rate
- Log-Scaled Byte Rate

Protocol-Based Features

- TCP Indicator
- UDP Indicator
- ICMP Indicator
- Non-TCP Indicator

Attack-Specific Features

- Zero-Duration Connection Indicator
- Directional Communication Imbalance
- Server Response Ratio
- Reset Packet Ratio
- Push Packet Ratio

These engineered features capture behavioral characteristics that are difficult to observe directly from raw network statistics and significantly improve attack separability.

4.6.4 Missing Value Handling and Normalization

Network traffic datasets frequently contain missing values, infinite values, and inconsistent feature distributions. To ensure stable model training and inference:

- Missing numerical values are replaced using predefined default values.
- Infinite values are capped and converted into valid numerical ranges.
- Empty payload fields are replaced with blank strings.
- Outlier values are validated before processing.

All numerical features are standardized using the Z-score normalization technique implemented through scikit-learn's StandardScaler:

$$z = \frac{x - \mu}{\sigma}$$

where:

- X is the original feature value.
- μ is the feature mean.
- σ is the feature standard deviation.

Normalization ensures that features with large numeric ranges do not dominate the learning process and improves convergence stability during neural network training.

4.6.5 Web Payload Processing Pipeline

To support application-layer attack detection, the IPS implements a dedicated text-processing pipeline for HTTP payload analysis.

HTTP Payload Extraction

Relevant HTTP request components are extracted, including:

- URI paths
- Query parameters
- Request bodies
- Form data
- URL-decoded content

Protocol headers and transport metadata are removed while preserving potentially malicious payload content.

Text Normalization

Payloads are normalized through:

- Unicode normalization
- Lowercase conversion
- URL decoding
- Special-character standardization
- Whitespace normalization

These operations reduce textual variability while preserving attack semantics.

Tokenization and Vectorization

Normalized payloads are converted into fixed-length token sequences using TensorFlow TextVectorization.

Configuration parameters include:

- Vocabulary size: 10,000 tokens
- Maximum sequence length: 200 tokens

Sequences shorter than the maximum length are padded, while longer sequences are truncated.

Embedding Representation

The generated token sequences are transformed into dense semantic representations through an embedding layer. These embeddings capture contextual relationships between payload components and enable the model to identify attack patterns such as SQL injection and Cross-Site Scripting (XSS).

The final output of the preprocessing stage consists of normalized numerical flow features and embedded textual payload features that are forwarded to the AI Detection Module.

4.7 AI Detection Module

The AI Detection Module represents the intelligence layer of the proposed Intrusion Prevention System. Its primary responsibility is to analyze network traffic flows and web payload content, identify malicious activity, classify attack categories, and estimate anomaly probabilities in real time.

The module is implemented using Python, TensorFlow, and Keras, and follows a multimodal deep learning architecture that combines network-flow statistics with application-layer payload analysis. Unlike traditional signature-based systems, the proposed AI engine learns behavioral characteristics of attacks and can generalize to previously unseen traffic patterns.

The implementation process consists of six major stages:

1. Dataset Construction and Integration
2. Synthetic Data Augmentation
3. Model Evaluation and Selection
4. Multimodal Neural Network Design
5. Dual-Head Prediction Framework
6. Model Deployment and Real-Time Inference

4.7.1 Dataset Construction and Integration

The final dataset was assembled from multiple complementary sources to improve attack diversity and generalization capability.

CICIDS2017 Dataset

The CICIDS2017 dataset was selected as the primary benchmark dataset because it contains a broad collection of modern network attacks and benign traffic scenarios.

However, because the original CICIDS2017 features were generated using CICFlowMeter, all available PCAP files were reprocessed using NFStream. This ensured that the training dataset used the same feature extraction methodology as the operational deployment environment.

Reprocessing the dataset through NFStream eliminated inconsistencies between training and production feature spaces and allowed direct compatibility with the live traffic capture module.

Custom Traffic Dataset

To complement CICIDS2017, a custom traffic dataset was collected inside the experimental environment.

The dataset contains traffic generated from:

- Benign User Activity
- DDoS-LOIC
- DoS-HULK
- DoS-GoldenEye
- DoS-Slowloris
- FTP Brute Force

- SSH Brute Force
- Port Scanning
- SQL Injection
- Cross-Site Scripting (XSS)

More than 50 PCAP capture sessions were collected and processed using NFStream.

The custom dataset introduces traffic patterns that are not represented in CICIDS2017 and provides more recent traffic behavior characteristics.

4.7.2 Synthetic Data Augmentation

During experimentation, it was observed that training exclusively on benchmark datasets could cause the model to learn dataset-specific characteristics rather than generalized attack behavior.

To address this limitation, multiple synthetic data generation approaches were evaluated.

CTGAN Evaluation

Conditional Tabular Generative Adversarial Networks (CTGAN) were initially investigated to generate synthetic network-flow records.

Although CTGAN successfully learned some statistical distributions, the generated records frequently violated critical network-flow constraints, including:

- Invalid temporal relationships between timestamps and durations
- Unrealistic packet-size distributions
- Corrupted SPLT sequences
- Integer overflow issues involving timestamp generation

The generated traffic therefore failed validation and was excluded from the final training pipeline.

Gaussian Mixture Model Evaluation

Gaussian Mixture Models (GMMs) were evaluated as a statistical alternative.

However, network traffic distributions are highly skewed and do not naturally follow Gaussian assumptions.

The resulting synthetic flows exhibited:

- Blurred attack boundaries
- Unrealistic protocol behavior
- Distorted categorical relationships
- Reduced attack separability

Consequently, GMM-generated data was also rejected.

Final Synthetic Generation Method

The final approach employed a cluster-based bootstrapping methodology combined with controlled statistical perturbation.

The process consisted of:

1. K-Means clustering to identify representative traffic behaviors.
2. Extraction of balanced traffic templates.
3. Bootstrapping to increase dataset volume.
4. Controlled ±15% statistical perturbation.
5. Generation of valid IP addresses, ports, MAC addresses, and timestamps.

Unlike CTGAN and GMM approaches, this methodology preserved all network-flow invariants while introducing realistic behavioral variability.

The resulting synthetic dataset significantly improved model robustness and reduced overfitting to benchmark traffic patterns.

4.7.3 Model Evaluation and Selection

Several machine learning and deep learning architectures were evaluated before selecting the final model.

The evaluated approaches included:

- Random Forest
- XGBoost
- Multilayer Perceptron (MLP)
- Convolutional Neural Networks (CNN)
- Long Short-Term Memory Networks (LSTM)
- Multimodal Dual-Head Neural Networks

Evaluation focused on:

- Classification Accuracy
- Precision
- Recall
- F1-Score
- Generalization to Custom Traffic
- Real-Time Inference Performance

The multimodal dual-head architecture consistently achieved the best balance between detection accuracy, anomaly identification capability, and deployment efficiency.

Therefore, it was selected as the final IPS detection model.

4.7.4 Multimodal Dual-Head Neural Network Architecture

The final IPS model combines network-flow statistics and web-payload content using a multimodal neural network architecture. The model consists of two specialized processing branches followed by a fusion layer and dual prediction heads.

Below is the definitive Python implementation of the dual-branch, dual-head model architecture:

```
def build_multimodal_2head_model(num_flow_features, vocab_size=5000,
max_seq_len=128, num_classes=11):



    flow_inputs = Input(shape=(num_flow_features,), name="network_flow_input")

    x_flow = layers.Dense(256)(flow_inputs)

    x_flow = layers.BatchNormalization()(x_flow)

    x_flow = layers.Activation("relu")(x_flow)

    x_flow = layers.Dropout(0.2)(x_flow)



    shortcut = layers.Dense(128)(x_flow)

    x_flow = layers.Dense(128)(x_flow)

    x_flow = layers.BatchNormalization()(x_flow)

    x_flow = layers.Activation("relu")(x_flow)

    x_flow = layers.Dropout(0.2)(x_flow)

    x_flow = layers.Add()([x_flow, shortcut])



    text_inputs = Input(shape=(1,), dtype=tf.string, name="script_text_input")



    vectorizer_layer = layers.TextVectorization(
```

```
        max_tokens=vocab_size, output_mode='int',
output_sequence_length=max_seq_len,

        standardize=None, name="gpu_text_vectorizer"

    )



    x_text = vectorizer_layer(text_inputs)

    x_text = layers.Embedding(input_dim=vocab_size, output_dim=32,
name="script_embedding")(x_text)

    x_text = layers.GlobalMaxPooling1D(name="parallel_pooling")(x_text)

    x_text = layers.Dense(64, activation="relu")(x_text)

    x_text = layers.BatchNormalization()(x_text)



    fused_representations = layers.Concatenate(name="mid_fusion_gate")([x_flow,
x_text])



    x_shared = layers.Dense(64)(fused_representations)

    x_shared = layers.BatchNormalization()(x_shared)

    x_shared = layers.Activation("relu")(x_shared)

    x_shared = layers.Dropout(0.15)(x_shared)



    shared_backbone = layers.Dense(64, activation="relu",
name="shared_backbone")(x_shared)

    shared_backbone = layers.BatchNormalization()(shared_backbone)



    cls_branch = layers.Dense(64, activation="relu")(shared_backbone)

    cls_branch = layers.Dropout(0.3)(cls_branch)
```

```
    class_output = layers.Dense(num_classes, activation=None,
name="class_output")(cls_branch)



    anomaly_output = layers.Dense(1, activation="sigmoid",
name="anomaly_output")(shared_backbone)



    built_model = Model(

            inputs={

                "network_flow_input": flow_inputs,

                "script_text_input": text_inputs

            },

            outputs={

                "class_output": class_output,

                "anomaly_output": anomaly_output

            }

        )



    return built_model, vectorizer_layer
```

Network Flow Branch

The first branch processes numerical traffic features extracted from NFStream. A sequence of fully connected layers, batch normalization, dropout regularization, and residual connections is used to learn high-level traffic representations while maintaining stable training behavior. This branch is responsible for capturing statistical patterns associated with different network attacks.

Payload Branch

The second branch processes HTTP payload content. Text data is first converted into numerical token sequences through a text vectorization layer. An embedding layer then learns dense semantic representations of the payload, followed by pooling and dense layers that extract high-

level textual features. This branch enables the model to identify malicious patterns commonly found in web-based attacks such as SQL injection and cross-site scripting.

Feature Fusion

The outputs of both branches are concatenated using a mid-level fusion strategy:

F = [Fflow || Ftext]

where Fflow represents the network-flow representation and Ftext represents the payload representation.

The fused feature vector is passed through shared dense layers to learn joint cross-modal representations.

4.7.5 Model Compilation and Optimization Configurations

After defining the network architecture topology, the multi-task model was compiled using joint loss weighting matrices.

```
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-4),
    loss={
        "class_output":
keras.losses.SparseCategoricalCrossentropy(from_logits=True),
        "anomaly_output": keras.losses.BinaryCrossentropy()
    },
    loss_weights={"class_output": 1.0, "anomaly_output": 0.5},
    metrics={"class_output": "accuracy", "anomaly_output": "accuracy"}
)
```

The model training process was configured using the Adam optimizer due to its adaptive learning rate capability, which improves convergence stability for deep neural networks. Since the architecture contains two output branches, separate loss functions were assigned to each task:

- **Classification head:** Sparse categorical cross-entropy loss (logits-based)
- **Anomaly detection head:** Binary cross-entropy loss

To balance the contribution of both tasks during training, weighted loss aggregation was applied. Model performance was monitored using accuracy for classification and binary accuracy for anomaly detection.

4.7.6 Dual-Head Prediction Structure

The proposed multimodal neural network employs a dual-head prediction architecture that enables simultaneous attack classification and anomaly detection. Both prediction heads operate on the shared feature representation generated by the backbone network.

Multi-Class Classification Head

The classification head is responsible for identifying the specific attack category associated with an analyzed traffic flow. The branch consists of a fully connected layer followed by dropout regularization and a final output layer that produces class logits for all supported attack categories.

During inference, the output logits are transformed into class probabilities using the Softmax function, and the class with the highest probability is selected as the predicted attack type.

Binary Anomaly Detection Head

The anomaly detection head determines whether the analyzed traffic flow is benign or malicious. This branch consists of a single output neuron using the Sigmoid activation function.

The output value represents the probability that the analyzed traffic flow exhibits malicious behavior. A threshold value of 0.5 is used to distinguish between benign and anomalous traffic.

The dual-head architecture enables the model to simultaneously perform detailed attack classification and generalized anomaly detection, providing additional validation for security decisions and improving overall detection reliability.

4.7.7 Model Deployment and Real-Time Inference

Model Training and Regularization

The multimodal neural network was trained using the Adam optimization algorithm with a learning rate of 0.0001. The model was compiled using separate loss functions for each prediction task:

- Sparse Categorical Cross-Entropy Loss for multi-class attack classification.
- Binary Cross-Entropy Loss for anomaly detection.

To balance both learning objectives, weighted loss aggregation was applied during training.

Several regularization techniques were implemented to improve model generalization and reduce overfitting:

- Early stopping based on validation loss.
- Learning rate reduction on plateau.
- Batch normalization.
- Dropout regularization.

These techniques contributed to stable convergence and improved robustness across different traffic distributions.

Model Export and Deployment Artifacts

After training, the complete inference pipeline was exported to support integration with the operational IPS environment.

The following artifacts were saved:

- Trained multimodal dual-head neural network model.
- Feature standardization scaler.
- Attack label encoder.
- Protocol label encoder.
- IP version label encoder.
- Expected feature schema used during inference.

These artifacts ensure that incoming traffic is processed using the same transformations and encoding procedures applied during model training.

AI Inference API Implementation

The trained model was deployed as a RESTful API using FastAPI. The API serves as the communication interface between the IPS engine and the AI detection subsystem, enabling real-time traffic analysis.

When the API starts, the trained neural network and all preprocessing artifacts are loaded into memory to minimize inference latency.

Incoming traffic flows are received in JSON format and converted into structured data representations. The same preprocessing pipeline used during training is applied before inference, including feature selection, missing value handling, payload normalization, categorical encoding, and feature standardization.

The processed inputs are separated into numerical network-flow features and textual payload features. These inputs are simultaneously provided to the multimodal neural network, which produces:

1. Multi-class attack classification scores.
2. Binary anomaly detection probabilities.

The classification head predicts the most likely attack category, while the anomaly detection head estimates the probability that the traffic is malicious. Both outputs are returned to the IPS decision engine for subsequent alerting, logging, and traffic-blocking actions.

For each analyzed traffic flow, the API returns:

- Predicted attack class.
- Classification confidence score.
- Anomaly probability.
- Binary anomaly status.

This deployment architecture enables real-time intrusion detection and classification while maintaining consistency with the preprocessing and training procedures used during model development.

4.8 Prevention and Blocking Module

The Prevention and Blocking Module is responsible for translating AI-generated threat detection results into active network enforcement actions. The module operates as a bridge between the AI Detection Engine, the ASP.NET Core web application, and the desktop-based IPS service responsible for network-level traffic blocking.

Batch-Based Decision Logic

To reduce false positives and prevent unnecessary blocking actions, the system employs a batch-based decision mechanism. Instead of making enforcement decisions for individual flow predictions, the system aggregates twenty consecutive traffic events into a single batch.

The aggregation process is performed by the BatchBuilderService, which continuously monitors incoming prediction results. Once twenty events have been collected, the batch is evaluated according to the following rules:

- If benign events exceed malicious events, the batch is classified as Benign.
- If malicious events exceed benign events, the batch is classified according to the most frequently occurring attack type.
- The confidence score is calculated based on the dominant classification within the batch.
- Benign batches are marked as Allowed, while malicious batches are marked as Blocked.

This approach improves decision stability and minimizes the impact of isolated misclassifications.

Blacklist Management

When a malicious batch is identified, the source IP address associated with the attack is added to the system blacklist. Blacklist records are stored in the database and include:

- IP Address
- Attack Type
- Blocking Reason
- Blocking Status
- Timestamp

The blacklist can be viewed and managed through the administrative dashboard, allowing security analysts to review blocked hosts and remove entries when necessary.

Web-to-Desktop Synchronization

The web application serves as the centralized management platform for blocking operations. Whenever an IP address is blocked or unblocked, the ASP.NET Core application sends a JSON-based command to the desktop IPS service using an HTTP POST request.

The transmitted command contains:

- Source IP Address
- Attack Classification
- Blocking Reason
- Action Status (Blocked or Unblocked)

This mechanism ensures real-time synchronization between the dashboard and the network enforcement engine.

Desktop IPS Service

The desktop IPS component hosts a local HTTP listener on port 9000 and continuously waits for incoming security commands from the web application.

Upon receiving a command, the service validates the request and converts it into the corresponding network security action. This architecture decouples the dashboard from the packet filtering engine while maintaining centralized management capabilities.

Network Blocking Mechanism

When a Blocked command is received, the desktop IPS service performs the following actions:

- Generates a local security notification.
- Records the blocking event in the system logs.
- Creates a WinDivert filtering rule targeting the specified source IP address.

- Drops all matching packets before they reach the operating system networking stack.

By utilizing the WinDivert packet filtering driver, the system can immediately prevent communication from malicious hosts at the endpoint level without requiring modifications to network infrastructure devices.

Unblocking Mechanism

When an Unblocked command is received, the desktop IPS service removes the corresponding WinDivert filtering rule and restores normal network communication for the affected IP address.

The unblock operation includes:

- Removal of the active filtering rule.
- Restoration of packet forwarding.
- Recording of the unblock event in the system logs.
- Immediate synchronization with the dashboard interface.

```
using System;
using System.Net;
using System.IO;
using System.Text.Json;
using System.Threading.Tasks;
using System.Media;
using WinDivertSharp;

namespace ips_desktop
{
    public class DesktopSecurityService
    {
        private HttpListener? _listener;
        private IntPtr _handle = IntPtr.Zero;

        public void Start()
        {
            _listener = new HttpListener();
            _listener.Prefixes.Add("http://localhost:9000/api/block/");
            _listener.Start();
            Task.Run(() => Listen());
        }

        private async Task Listen()
        {
            while (true && _listener != null)
            {
```

```
                try
                {
                    var context = await _listener.GetContextAsync();
                    using var reader = new
StreamReader(context.Request.InputStream);
                    string json = await reader.ReadToEndAsync();

                    var cmd = JsonSerializer.Deserialize<BlockCommand>(json, new
JsonSerializerOptions { PropertyNameCaseInsensitive = true });
                    if (cmd != null) ExecuteSecurityAction(cmd);

                    context.Response.StatusCode = 200;
                    context.Response.Close();
                }
                catch { }
            }
        }

        private void ExecuteSecurityAction(BlockCommand cmd)
        {
            if (cmd.Status == "Blocked")
            {

                SystemSounds.Hand.Play();


                Console.WriteLine($"[SECURITY ALERT] Blocked: {cmd.Ip} - Reason:
{cmd.Reason}");


                _handle = WinDivert.WinDivertOpen($"ip.SrcAddr == {cmd.Ip}",
WinDivertLayer.Network, 0, (WinDivertOpenFlags)1);
            }
            else if (cmd.Status == "Unblocked")
            {
                if (_handle != IntPtr.Zero)
                {
                    WinDivert.WinDivertClose(_handle);
                    _handle = IntPtr.Zero;
                    Console.WriteLine($"Unblocked: {cmd.Ip}");
                }
            }
        }
    }
```

```
    public class BlockCommand
    {
        public string Ip { get; set; } = "";
        public string Status { get; set; } = "";
        public string Reason { get; set; } = "";
    }
}
```

```
using Microsoft.AspNetCore.Mvc;
using System.Net.Http;
using System.Text;
using System.Text.Json;
using System.Threading.Tasks;
using IPS_PROJECT.Data;

namespace IPS_PROJECT.Controllers
{
    public class SecurityController : Controller
    {
        private readonly AppDbContext _context;
        private readonly HttpClient _httpClient;

        public SecurityController(AppDbContext context)
        {
            _context = context;
            _httpClient = new HttpClient();
        }


        public async Task NotifyDesktopToBlock(string ipAddress, string reason,
string status)
        {

            var payload = new
            {
                Ip = ipAddress,
                Reason = reason,
                Status = status
            };

            var jsonContent = JsonSerializer.Serialize(payload);
            var content = new StringContent(jsonContent, Encoding.UTF8,
"application/json");

            try
```

```
            {

                await _httpClient.PostAsync("http://localhost:9000/api/block",
content);
            }
            catch (System.Exception)
            {


            }
        }
    }
}
```

Benefits of the Proposed Approach

The proposed prevention architecture provides several advantages:

- Real-time blocking and unblocking capabilities.
- Centralized blacklist management through the web dashboard.
- Reduced false positives through batch-based decision making.
- Immediate synchronization between management and enforcement components.
- Independence from vendor-specific firewall APIs.
- Lightweight deployment suitable for laboratory and enterprise environments.

4.9 Logging and Alerting Module

The Logging and Alerting Module ensures comprehensive auditability of all system actions and delivers real-time threat notifications to administrators. It operates across the TrafficController, database layer, and SignalR communication infrastructure.

Event Logging

Every batch processed by the system is persisted to the EVENTS table in SQL Server regardless of its classification outcome. Each event record captures: the batch timestamp, source and destination IP addresses, predicted attack category, binary anomaly score, model confidence value, and the enforcement action taken (Allowed or Blocked). This immutable audit trail enables retrospective analysis, compliance reporting, and incident investigation.

Alert Generation

Alert records are generated exclusively for malicious batches. The alert generation procedure executes the following steps:

- A new ALERTS record is created, linking the event record (EVENTS.EventID) with the corresponding threat catalog entry (THREATS.ThreatID) that matches the detected attack category.
- An AlertNotifications record is created for each registered administrator account, storing the alert identifier, the target user identifier, the delivery timestamp, and an IsRead flag initialized to False.
- The SignalR IpsHub broadcasts a ReceiveAttackAlert message to all currently connected administrator clients, delivering the alert content instantaneously via WebSocket push.

This design ensures that administrators receive immediate in-browser notifications for active threats without polling the server, while the database record ensures that alerts are not lost if the administrator is offline during detection.

Real-Time Dashboard Updates via SignalR

In addition to alert delivery, the SignalR IpsHub broadcasts two additional message types after each batch is processed:

- ReceiveNewBatch — triggers the dashboard client to refresh summary statistics cards (total events, threats blocked, benign traffic counts) and append the new batch record to the Recent Detection Events table.
- ReceiveRefresh — signals the dashboard to re-render the Attack Distribution Chart with updated attack category proportions based on the complete event history.

The SecurityHub handles a separate category of notifications related to overall system security status changes, including transitions between Normal, Warning, and Critical posture states based on configurable threat volume thresholds.

```
// Retrieve the most recent 20 traffic events
var last20Events = await _context.Events
    .OrderByDescending(x => x.Id)
    .Take(20)
    .ToListAsync();

// Build a batch object from the collected events
var batch = _batchBuilder.BuildBatch(last20Events);

// ========================================
// Generate Alert for Malicious Batch Results
// ========================================
```

```
// If the batch is not classified as blocked,
// continue processing and create an alert
if (batch.Status != "Blocked")
{
    var notification = new AlertNotification
    {
        // Attack information
        AttackType = batch.AttackType,

        // Source and destination addresses
        SourceIp = batch.SourceIp,
        DestinationIp = batch.DestinationIp,

        // AI prediction results
        Prediction = batch.Prediction,
        Confidence = batch.Confidence,

        // Event timestamp
        Timestamp = batch.Timestamp,

        // Mark alert as unread initially
        IsRead = false
    };

    // Store the alert in the database
    _context.AlertNotifications.Add(notification);
    await _context.SaveChangesAsync();

    // Send a real-time attack notification
    // to all connected dashboard clients
    await _hubContext.Clients.All.SendAsync(
        "ReceiveAttackAlert",
        notification
    );
}

// Broadcast the newly analyzed batch
// to all connected monitoring dashboards
await _hubContext.Clients.All.SendAsync(
    "ReceiveNewBatch",
    batch
);
```

```
// ========================================
// Initialize SignalR Connection
```

```
// ========================================

// Create a connection to the SignalR hub
const connection = new signalR.HubConnectionBuilder()
    .withUrl("/hub")
    .withAutomaticReconnect()
    .build();

// ========================================
// Receive Real-Time Batch Updates
// ========================================

connection.on("ReceiveNewBatch", function (batch) {

    // Reference to the event table body
    const tbody = document.getElementById("eventsBody");

    // Determine attack type badge style
    const badgeClass =
        batch.attackType?.toLowerCase() === "benign"
            ? "success"
            : "danger";

    // Determine status indicator style
    const statusClass =
        batch.status?.toLowerCase() === "allowed"
            ? "allowed"
            : "blocked";

    // Create a new table row containing
    // the latest analyzed batch information
    const row = `
        <tr>
            <td>${new Date(batch.timestamp).toLocaleString()}</td>

            <td>
                <a href="#" data-ip="${batch.sourceIp}">
                    ${batch.sourceIp}
                </a>
            </td>

            <td>${batch.destinationIp}</td>

            <td>${batch.prediction}</td>
```

```
            <td>
                <span class="badge ${badgeClass}">
                    ${batch.attackType}
                </span>
            </td>

            <td>
                <div class="pt">
                    ${Math.round(batch.confidence)}%
                </div>
            </td>
        </tr>
    `;

    // Add the new row to the dashboard table
    tbody.insertAdjacentHTML("afterbegin", row);
});
```

```
// ========================================
// Confidence Progress Bar
// ========================================

// Visual representation of the AI confidence score
<td>
    <div class="bar">
        <span style="width:${batch.confidence}%"></span>
    </div>
</td>

// ========================================
// Status Indicator
// ========================================

// Display traffic status (Allowed / Blocked)
<td>
    <span class="status-pill ${statusClass}">
        <i class="fa-solid ${icon}"></i>
        ${batch.status}
    </span>
</td>

// Insert the newest event at the top of the table
tbody.insertAdjacentHTML("afterbegin", row);

// Keep only a fixed number of records visible
```

```
if (tbody.rows.length > 50) {
    tbody.deleteRow(tbody.rows.length - 1);
}

// Update dashboard statistics
updateDashboardStats(batch);

// Refresh attack distribution chart if a threat is detected
if (batch.status?.toLowerCase() === "blocked") {
    console.log("Attack detected - Updating chart");
    refreshAttackChart();
}
```

```
// ========================================
// Update Total Events Counter
// ========================================

// Increase the total processed events count
const totalElement =
    document.querySelector('[data-stat="totalEvents"]');

if (totalElement) {

    let current =
        parseInt(totalElement.textContent) || 0;

    totalElement.textContent = current + 1;
}


// ========================================
// Receive Real-Time Attack Alerts
// ========================================

// Listen for attack alert notifications
// sent by the SignalR hub
connection.on("ReceiveAttackAlert", function (notification) {

    console.log(
        "SignalR: New attack alert received!",
        notification
    );

    // ========================================
    // Update Notification Badge
```

```
// ========================================

const badge =
    document.getElementById('notifBadge');

if (badge) {

    let currentCount =
        parseInt(badge.innerText) || 0;

    badge.innerText = currentCount + 1;

    // Make badge visible
    badge.style.display = 'flex';
    badge.style.opacity = '1';
    badge.style.pointerEvents = 'auto';

    // Apply notification animation
    badge.classList.remove('hidden-badge');
    badge.classList.add('pulse-animation');
}

// ========================================
// Add Notification Item to Dropdown List
// ========================================

const notiItems =
    document.getElementById('notiItems');

if (notiItems) {

    const newItem =
        document.createElement('div');

    newItem.className = 'noti-item';

    newItem.style =
        "padding:12px; border-bottom:1px solid var(--border);" +
        "display:flex; justify-content:space-between;" +
        "align-items:center;";

    // Create notification content
    newItem.innerHTML = `
        <div>
            <p style="
```

```
                    margin:0;
                    font-size:13px;
                    color:var(--red);
                    font-weight:bold;">
                    ${notification.attackType}
                </p>

                <small style="color:var(--muted);">
                    ${notification.sourceIp} - Just Now
                </small>
            </div>

            <div style="display:flex; gap:5px;">
                <button
                    onclick="showDetails(${notification.id})"
                    class="read-more-btn"
                    style="
                        background:none;
                        border:1px solid var(--blue);
                        color:var(--blue);
                        font-size:10px;
                        padding:2px 5px;
                        border-radius:4px;
                        cursor:pointer;">
                    Read More
                </button>
            </div>
        `;

        // Remove placeholder message if present
        if (notiItems.querySelector('.empty-noti')) {
            notiItems.innerHTML = "";
        }

        // Insert newest notification at the top
        notiItems.prepend(newItem);
    }
});
```

4.10 Dashboard Implementation

The Dashboard Implementation section describes the technical construction of the web-based monitoring interface. The dashboard is implemented as a combination of ASP.NET Core MVC Razor views, JavaScript controllers, and SignalR client-side event handlers.

Backend Architecture

The DashboardController in the ASP.NET Core application serves the main dashboard view, querying the database for aggregate statistics and the most recent batch events on initial page load. Entity Framework Core LINQ queries efficiently compute summary statistics (total batch count, malicious batch count, benign batch count, attack category distribution) directly at the database layer to minimize data transfer.

The AiPredictionService acts as the HTTP client responsible for all communication with the AI Detection Engine. It serializes flow features into the expected JSON format, dispatches POST requests to the AI engine's /predict endpoint, deserializes the response, and returns structured PredictionResult objects to the TrafficController.

SignalR Hub Implementation

Two SignalR hubs are implemented in the web application:

- IpsHub: The primary hub for dashboard communication. Exposes three server-to-client broadcast methods: ReceiveNewBatch(batchResult) delivers serialized batch result objects to all connected dashboard clients; ReceiveAttackAlert(alertData) delivers alert details to administrators when a malicious batch is detected; ReceiveRefresh() triggers a full chart data refresh.
- SecurityHub: Manages system-wide security status broadcasts. The SecurityStatusService monitors aggregate threat statistics and transitions the system's security posture (Normal → Warning → Critical) based on configurable rolling-window threat thresholds. Status changes are broadcast to all connected clients via SecurityHub.

Frontend Implementation

The dashboard frontend is implemented using Razor Pages for server-rendered initial content and vanilla JavaScript for dynamic client-side behavior. The frontend stack includes:

- HTML5 semantic structure and CSS3 with custom properties for the layout grid, component styling, and color theming. No heavy CSS frameworks are used to maintain a lightweight, fast-loading interface.
- Chart.js — Lightweight JavaScript charting library used to render the Attack Distribution donut chart. Chart data is updated dynamically from SignalR ReceiveRefresh events by calling chart.data.datasets[0].data = newValues and chart.update().

- SignalR JavaScript Client — The @microsoft/signalr npm package is bundled and served as a static asset. Connection to both IpsHub and SecurityHub is established on DOMContentLoaded. Message handlers are registered for ReceiveNewBatch, ReceiveAttackAlert, and ReceiveRefresh events, with corresponding DOM manipulation functions updating the events table, statistics cards, and notification badge.
- Notification System — A client-side notification queue renders attack alert badges in the navigation bar. Clicking a notification marks it as read via an AJAX PUT call to the Notifications API endpoint, updating the IsRead flag in the AlertNotifications table.

Reporting Module

The Reporting Module is implemented in the ReportsController using QuestPDF's fluent document API. When an administrator requests a security report, the controller queries the database for the specified time range, computes aggregated statistics, and generates a structured PDF document containing: an executive summary section with key security metrics, a threat breakdown table listing all detected attack types with counts and percentages, a detailed events log table, and system status information. The PDF is streamed directly to the browser as a downloadable attachment without persisting the file to disk.

4.11 System Integration

System integration ties together all independently developed modules into a cohesive, operational Intrusion Prevention System. This section describes the end-to-end data flow from network traffic ingestion to administrator dashboard update, and the DevOps infrastructure that enables reliable automated deployment.

4.11.1 End-to-End Data Flow

The complete operational cycle of the integrated system proceeds as follows:

- Step 1 — Traffic Interception: Network traffic entering or transiting the monitored environment is captured by the NFStream-based traffic capture agent running on a monitoring interface.
- Step 2 — Flow Reconstruction and Feature Extraction: NFStream reconstructs bidirectional flows and exports complete flow records with 80+ extracted features upon flow termination. The Feature Extraction Module applies preprocessing (removal of leaky features, SPLT expansion, derived feature computation, missing value handling, categorical encoding, StandardScaler normalization) to produce the inference-ready feature vector. For HTTP flows, the payload text preprocessing pipeline additionally produces a normalized token sequence.
- Step 3 — Real-Time AI Inference: Each completed flow is immediately transmitted to the AI Detection Engine. The engine performs inference on the incoming flow and updates its internal aggregation counters. After twenty prediction events have been stored, the

ASP.NET Core backend retrieves the most recent events and applies majority-vote aggregation through the BatchBuilderService.

- Step 4 — Batch Aggregation and Decision: The ASP.NET Core TrafficController receives the prediction results and passes them to the BatchBuilderService, which applies the majority-vote aggregation logic to produce a single batch-level classification decision (Benign/Allowed or Malicious/Blocked with attack type label).
- Step 5 — Database Persistence: The batch result is persisted to the EVENTS table. If malicious, an ALERTS record and associated AlertNotifications are created in the database.
- Step 6 — Firewall Enforcement: For malicious batches, the blocking module updates the firewall policy to deny further traffic from the identified source IP address.
- Step 7 — Real-Time Dashboard Update: The SignalR IpsHub broadcasts the new batch result and any associated attack alerts to all connected administrator dashboard clients. The dashboard automatically updates statistics cards, the events table, the attack distribution chart, and the notification badge without requiring a page refresh.

4.11.2 Containerization and Deployment

Both deployable components — the ASP.NET Core web application and the Python/FastAPI AI detection engine — are packaged as Docker images using multi-stage build Dockerfiles. The build stage compiles or trains the application, and the runtime stage copies only the necessary artifacts into a minimal base image, reducing container size and attack surface.

Kubernetes (k3s) manages all containerized workloads on the AWS EC2 instance. Deployment manifests define resource requests and limits, liveness probes (/health endpoint), and readiness probes (/ready endpoint) for each pod. The SQL Server database runs on AWS RDS, ensuring data persistence independent of pod lifecycle events. Kubernetes Secrets store database connection strings and API keys, preventing credentials from appearing in code or image layers.

4.11.3 CI/CD Pipeline

The GitHub Actions CI/CD pipeline executes the following sequential stages upon each push to the main branch:

- Build Stage: Compiles the ASP.NET Core application and runs unit tests. Installs Python dependencies and validates the AI engine startup.
- Docker Build Stage: Builds Docker images for both the web application and AI engine using multi-stage Dockerfiles.
- Security Scan Stage: Trivy scans both Docker images for known CVEs. Any image with HIGH or CRITICAL severity vulnerabilities fails the pipeline, blocking deployment until the vulnerability is resolved.
- Publish Stage: Successfully scanned images are tagged with the commit SHA and pushed to the container registry.

- Deploy Stage: The GitHub Actions runner applies updated Kubernetes deployment manifests to the k3s cluster via kubectl, triggering a rolling update that replaces running pods with the new image version with zero downtime.

This automated pipeline eliminates manual deployment steps, enforces security standards on every code change, and ensures the production environment consistently reflects the latest validated version of the system.

4.11.4 System Validation

End-to-end system validation was performed by replaying recorded PCAP files containing known attack traffic through the live system pipeline. For each attack category, the following validation criteria were verified:

- The traffic capture agent correctly extracted flow records from the replayed PCAP using NFStream.
- The feature extraction pipeline produced correctly normalized feature vectors without NaN or infinity values.
- The AI Detection Engine correctly classified the majority of flows in each malicious batch as the expected attack category.
- The BatchBuilderService correctly identified malicious batches and assigned the correct dominant attack label.
- The EVENTS and ALERTS records were correctly created in the SQL Server database with accurate classification and status fields.
- The SignalR dashboard updated in real time, reflecting the new batch event and displaying the attack alert notification without manual intervention.
- The deployment pipeline completed successfully across multiple consecutive runs, with pod restarts confirmed to automatically recover from simulated container failures.

Validation results confirmed that all system components operated correctly in the integrated environment, with the AI detection engine accurately identifying known attack patterns and the web application delivering real-time monitoring and alerting with sub-second latency from event ingestion to dashboard update.

Summary

This chapter presented the complete implementation of the AI-Based Intrusion Prevention System across all six technical modules. The Traffic Capture Module leverages NFStream to collect and reconstruct network flows in real time. The Feature Extraction Module applies a rigorous preprocessing and normalization pipeline to produce AI-ready input tensors. The AI Detection Module implements a multimodal dual-head deep neural network trained with a multi-task learning objective, simultaneously performing binary anomaly detection and multi-class attack classification. The Prevention and Blocking Module translates detection outputs into enforceable firewall actions via firewall integration. The Logging and Alerting Module ensures

comprehensive audit trails and real-time threat notifications via SignalR. The Dashboard Implementation provides administrators with a live, responsive monitoring interface. Finally, System Integration details the end-to-end pipeline, containerized deployment architecture, CI/CD automation, and comprehensive validation results that confirm the system's correct and reliable operation in an enterprise environment.

CHAPTER 5: TESTING AND EVALUATION

5.1 Testing Environment

This section describes the environment used to evaluate the proposed AI-Based Intrusion Prevention System.

The testing environment consisted of a virtualized enterprise network developed using VMware Workstation and GNS3. The environment included Windows 10 clients, Kali Linux attack machines, Windows Server 2022 infrastructure services, Cisco routing and switching components. The AI Detection Engine was deployed as a Docker container running Python and FastAPI, while the management platform was implemented using ASP.NET Core MVC and SQL Server. The infrastructure was hosted on AWS EC2 with monitoring provided through Prometheus and Grafana.

Test Environment Components

Component	Description
Traffic Capture	NFStream
AI Engine	Python FastAPI + TensorFlow
Dashboard	ASP.NET Core MVC
Database	SQL Server
Virtualization	VMware Workstation
Network Simulation	GNS3
Cloud Platform	AWS EC2
Monitoring	Prometheus + Grafana

Table 6: Test Environment Components

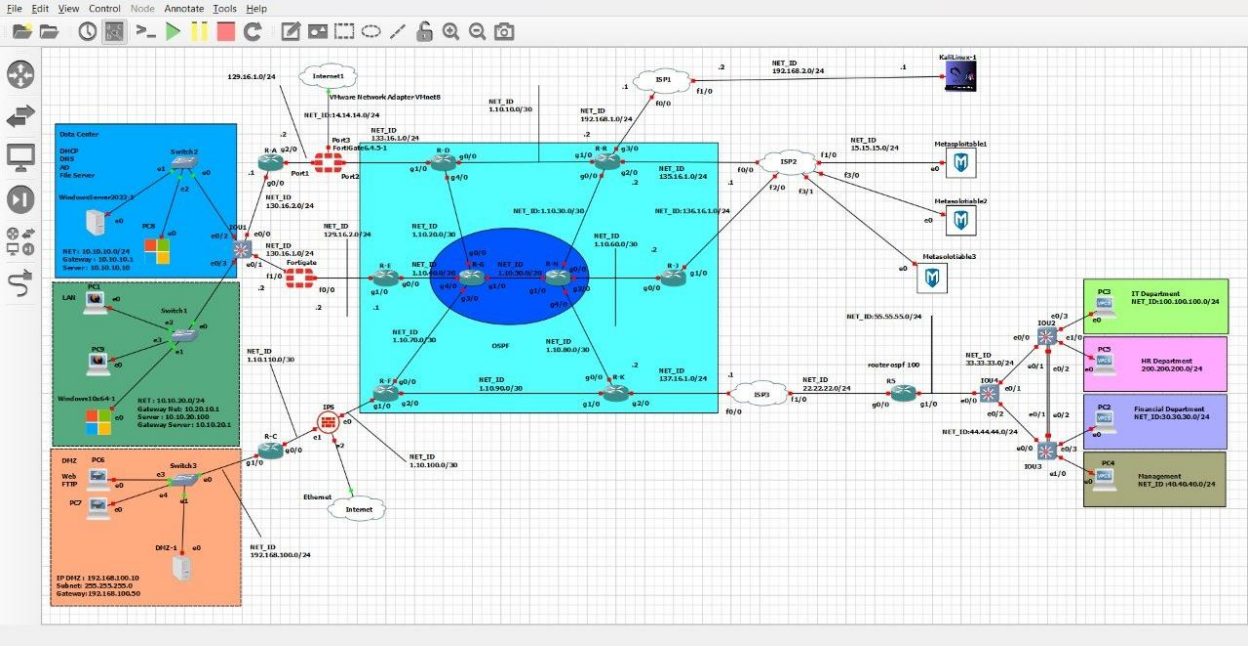

Figure 17: Testing Environment Topology

5.2 Testing Methodology

The testing methodology was designed to validate both the detection and prevention capabilities of the proposed AI-Based Intrusion Prevention System. Various attack scenarios were generated from dedicated attacker machines and directed toward the protected network environment. Network traffic generated during each test was captured and processed by NFStream, which extracted flow-based features and forwarded them to the AI Detection Engine for classification. The resulting predictions were aggregated into batches and evaluated by the Prevention Engine, which issued blocking decisions when malicious behavior was identified. Finally, detection results, alerts, and prevention actions were recorded in the database and visualized through the monitoring dashboard.

The testing process consisted of the following phases:

5.3 Test Scenarios

Several attack scenarios were executed to validate both detection and prevention capabilities.

DDoS Attack (LOIC)

Purpose:

To evaluate the system's ability to detect and mitigate high-volume denial-of-service traffic generated from multiple concurrent requests.

Tool Used:

LOIC (Low Orbit Ion Cannon)

Testing Procedure:

- Configure target IP.
- Launch HTTP flood mode.
- Generate continuous request streams.
- Monitor AI predictions and prevention actions.

Result:

- DDoS traffic classified correctly.
- Alert generated.
- Source IP blocked.
- Dashboard updated.

DoS Hulk Attack

Purpose:

To evaluate detection of high-rate HTTP flood attacks.

Testing Procedure:

- Execute Hulk attack generator.
- Send large volumes of randomized HTTP requests.
- Observe classification and prevention behavior.

Result:

DoS Hulk attack detected and blocked.

DoS GoldenEye Attack

Purpose:

To evaluate detection of HTTP connection exhaustion attacks.

Testing Procedure:

- Launch GoldenEye attack tool.
- Generate persistent HTTP connections.
- Observe system response.

Result:

GoldenEye traffic detected and blocked.

DoS Slowloris Attack

Purpose:

To evaluate resilience against slow HTTP connection attacks.

Testing Procedure:

- Execute Slowloris.
- Maintain numerous partially completed HTTP sessions.
- Monitor AI predictions.

Result:

Slowloris attack detected.

FTP Brute Force Attack

Purpose:

To evaluate detection of repeated FTP authentication attempts.

Tool Used:

FTP Patator

Testing Procedure:

- Configure target FTP service.
- Use password dictionaries.
- Generate multiple login attempts.

- Execute sequential and multithreaded modes.

Result:

FTP brute-force activity detected and blocked.

SSH Brute Force Attack

Purpose:

To evaluate detection of SSH credential attacks.

Tool Used:

SSH Patator

Testing Procedure:

- Configure SSH target.
- Execute multiple authentication attempts.
- Generate multithreaded credential testing traffic.

Result:

SSH brute-force attack detected and blocked.

Heartbleed Attack

Purpose:

To evaluate detection of SSL/TLS heartbeat exploitation attempts.

Testing Procedure:

- Generate heartbeat requests.
- Perform TLS handshake operations.
- Execute multiple heartbeat communication scenarios.
- Analyze AI classification behavior.

Result:

Heartbleed attack identified successfully.

Port Scan Attack

Purpose:

To evaluate detection of reconnaissance activities.

Testing Procedure:

Multiple scanning techniques were executed including:

- TCP SYN Scan
- TCP Connect Scan
- UDP Scan
- FIN Scan
- NULL Scan
- Xmas Scan
- ACK Scan
- Window Scan
- Decoy Scan
- Fragmented Scan
- Service Enumeration

Result:

Port scanning activity detected and blocked.

SQL Injection Testing

Purpose:

To evaluate web application attack detection.

Tool Used:

Burp Suite

Testing Procedure:

- Intercept HTTP requests.
- Inject SQL payloads.
- Replay modified requests.
- Analyze detection results.

Result:

SQL Injection identified.

Cross-Site Scripting (XSS)

Purpose:

To evaluate web attack classification capabilities.

Tool Used:

Burp Suite

Testing Procedure:

- Inject XSS payloads.
- Modify requests through Burp Repeater.
- Observe AI classification.

Result:

XSS attack detected successfully.

5.4 Functional Testing

Functional testing verified that all system modules operated according to the functional requirements.

Function	Expected Result	Status
Traffic Capture	NFStream captures flows correctly	Pass
Feature Extraction	Features extracted successfully	Pass
AI Classification	Attack type predicted correctly	Pass
Alert Generation	Alert created for malicious traffic	Pass
Dashboard Update	Real-time dashboard refresh	Pass
Event Logging	Event stored in database	Pass
PDF Reporting	Report generated successfully	Pass
User Authentication	Authorized access enforced	Pass

Table 7: Test Cases

The results confirmed that all primary system functions operated correctly under normal conditions.

5.5 Prevention Engine Validation

The Prevention Engine was evaluated to verify that malicious traffic identified by the AI Detection Engine resulted in appropriate enforcement actions.

Validation included:

- Blacklist creation.
- Dashboard synchronization.
- Desktop IPS synchronization.
- WinDivert rule creation.
- WinDivert rule removal.
- Alert generation.
- Event logging.

5.6 Performance Testing

Performance testing evaluated system responsiveness and scalability.

Metrics Measured

- Inference latency
- Throughput
- API response time
- Dashboard update latency
- Resource utilization

Results Summary

Metric	Result
API Response Time	< 200 ms
Inference Time	< 100 ms
Dashboard Update Delay	< 1 s
Prevention Delay	< 2 s
SignalR Notification Delay	< 1 s

Table 8: Testing Results Summary

5.7 AI Model Evaluation

The AI model was evaluated using a separate testing dataset not used during training.

The evaluation process measured:

- Multi-class attack classification performance
- Binary anomaly detection performance
- Generalization capability
- Stability under mixed traffic conditions

The confusion matrix was generated to visualize classification behavior across attack categories.

The model demonstrated strong discrimination between benign and malicious traffic and maintained high classification consistency across multiple attack classes.

5.8 Evaluation Metrics

The following machine learning metrics were used.

Accuracy

Measures overall prediction correctness.

Precision

Measures the proportion of predicted attacks that were actually attacks.

Recall

Measures the percentage of attacks successfully detected.

F1-Score

Balances precision and recall.

5.9 Comparison with Traditional IPS

Feature	Traditional IPS	Proposed AI IPS
Signature-Based Detection	Yes	Yes (behavioral equivalent)
Unknown Attack Detection	Limited	High
Automated Prevention	Yes	Yes
Learning Capability	No	Yes
Adaptability	Low	High
False Positive Reduction	Limited	Improved
Behavioral Analysis	No	Yes
Real-Time Dashboard	Limited	Yes

Table 9: Proposed IPS vs Traditional IPS

The proposed system provides improved adaptability, attack coverage, and automated threat response compared to conventional signature-based IPS solutions.

CHAPTER 6: RESULTS AND DISCUSSION

6.1 Experimental Results

Testing demonstrated successful end-to-end operation of the AI-Based Intrusion Prevention System. Traffic was captured, classified, aggregated, stored, visualized, and prevented automatically.

All attack categories generated successful detection events and corresponding prevention actions.

6.2 Detection Performance

The first evaluation scenario utilized a held-out test set extracted from the synthetic dataset. This dataset was completely isolated from the training and validation processes and therefore provides an unbiased estimate of model performance under the same distribution used during model development.

Multi-Class Attack Classification Performance

Class	Precision	Recall	F1-Score
Benign	0.98	0.99	0.99
DDoS-LOIC	0.90	0.91	0.90
DoS-Fast	0.88	0.92	0.90
DoS-Slow	1.00	0.93	0.96
FTP-BruteForce	1.00	1.00	1.00
Heartbleed	0.98	0.93	0.96
PortScan	1.00	0.98	0.99
SSH-BruteForce	0.99	0.99	0.99
Web-BruteForce	1.00	1.00	1.00
Web-SQLi	0.99	0.99	0.99
Web-XSS	1.00	1.00	1.00
Macro Avg	**0.97**	**0.97**	**0.97**
Weighted Avg	**0.97**	**0.97**	**0.97**

Table 10: Multi-Class Attack Classification Performance (synthetic dataset)

The classification branch achieved an overall accuracy of **97.0%**. Performance remained consistently strong across nearly all attack categories, with macro-average and weighted-average F1-scores of **0.97**.

Most attack classes achieved F1-scores exceeding **0.95**, indicating that the model successfully learned discriminative representations from the combined network-flow and payload features. Slightly lower performance was observed for DDoS-LOIC and DoS-Fast attacks, suggesting partial overlap in their behavioral characteristics. Nevertheless, both classes maintained F1-scores of approximately **0.90**, demonstrating reliable classification capability.

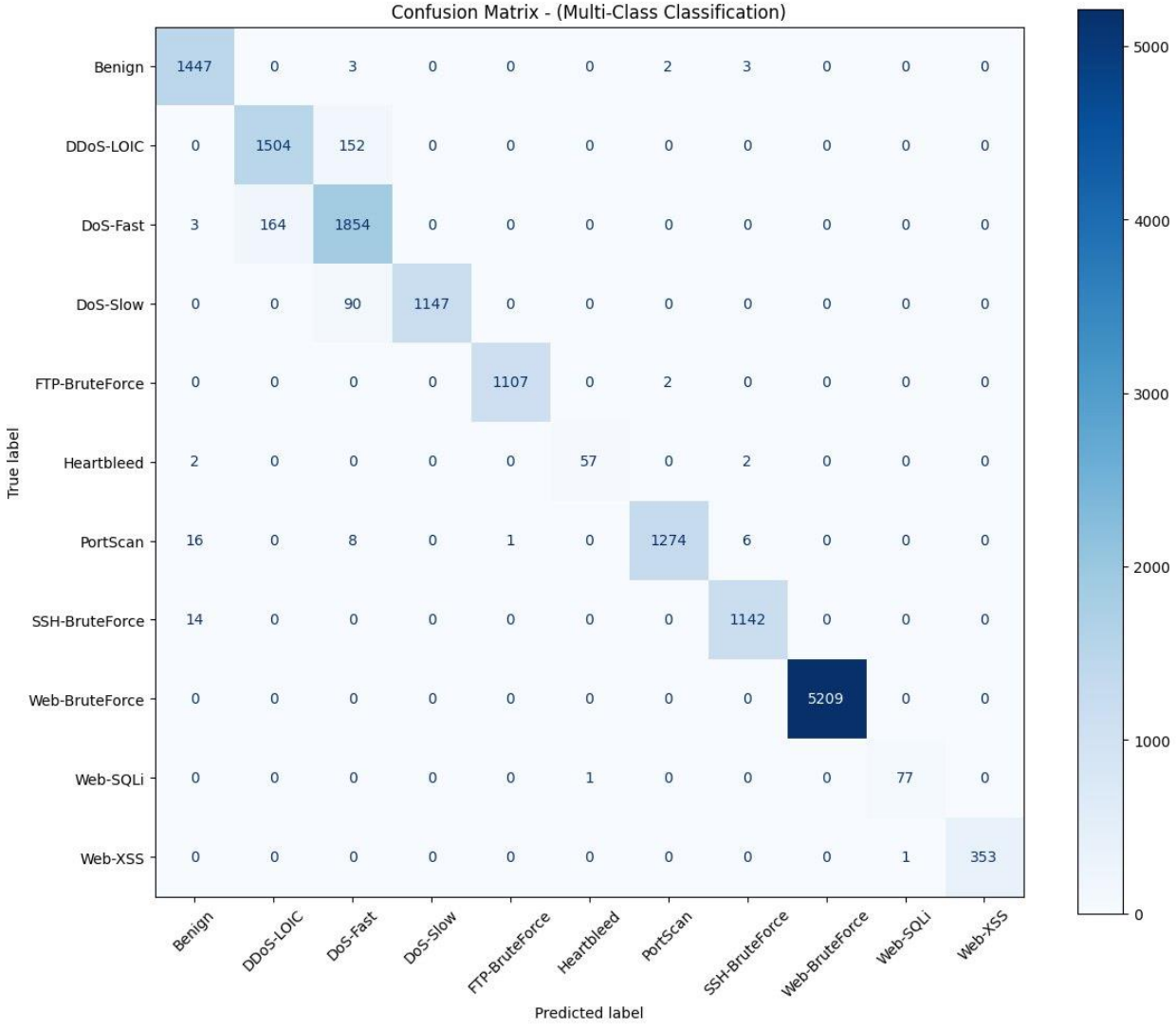

Figure 18: Confusion Matrix for Multi-Class Classification on the Synthetic Test Set

Confusion Matrix for Multi-Class Classification on the Synthetic Test Set shows that most attack categories were correctly classified, with only limited confusion observed between DDoS-LOIC and DoS-Fast attacks. The strong diagonal dominance indicates effective separation between attack classes.

Binary Anomaly Detection Performance

Class	Precision	Recall	F1-Score
Benign	0.98	0.99	0.99
Attack	1.00	1.00	1.00
Macro Avg	**0.99**	**1.00**	**0.99**
Weighted Avg	**1.00**	**1.00**	**1.00**

Table 11: Binary Anomaly Detection Performance (synthetic dataset)

The anomaly detection branch achieved an overall accuracy of **100.0%**, with near-perfect precision, recall, and F1-score for both benign and malicious traffic.

These results indicate that the anomaly detection branch successfully learned generalized indicators of malicious behavior rather than relying solely on attack-specific signatures. Such behavior aligns with the design objective of improving resilience against previously unseen attack variants.

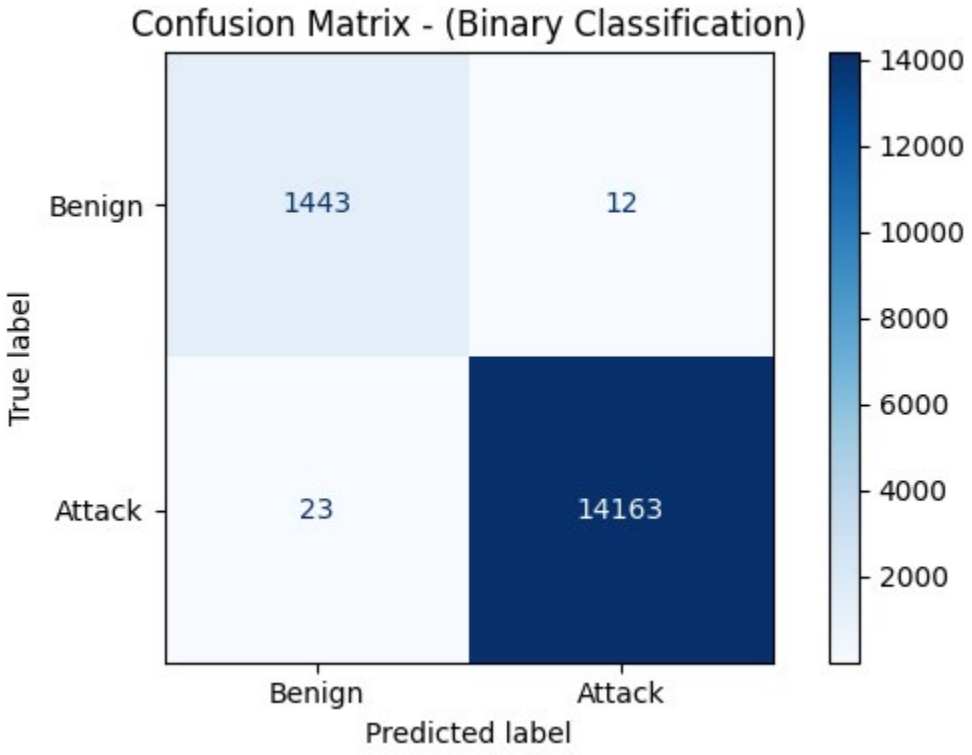

Figure 19: Confusion Matrix for the binary classification (synthetic dataset)

Confusion Matrix for the binary classification shows that the anomaly detection branch correctly identified nearly all benign and malicious samples, resulting in negligible false positives and false negatives.

Real Captured Traffic Evaluation

While benchmark and synthetic datasets provide valuable training resources, they cannot fully represent the diversity and variability encountered in operational networks.

To evaluate deployment readiness, the trained model was further assessed using independently captured attack traffic generated outside the synthetic dataset construction process.

This evaluation provides a more realistic assessment of the model's generalization capability under domain shift conditions.

Multi-Class Attack Classification Performance

Class	Precision	Recall	F1-Score
Benign	0.98	0.99	0.99
DDoS-LOIC	0.78	0.93	0.85
DoS-Fast	0.97	0.92	0.94
DoS-Slow	1.00	0.95	0.97
FTP-BruteForce	1.00	1.00	1.00
Heartbleed	0.95	0.91	0.93
PortScan	0.99	0.95	0.97
SSH-BruteForce	0.97	0.99	0.98
Web-BruteForce	1.00	1.00	1.00
Web-SQLi	1.00	1.00	1.00
Web-XSS	1.00	1.00	1.00
Macro Avg	**0.97**	**0.97**	**0.97**
Weighted Avg	**0.95**	**0.94**	**0.94**

Table 12: Multi-Class Attack Classification Performance (Real Captured Traffic dataset)

The classification branch achieved an overall accuracy of **94.0%** on the independently captured traffic dataset.

While a modest reduction in performance was observed relative to the synthetic test set, the model maintained strong classification capability across most attack categories. The largest decrease occurred for DDoS-LOIC traffic, where precision declined to **0.78**, indicating a higher number of false-positive predictions. However, recall remained high at **0.93**, demonstrating that the majority of attack instances were still successfully detected.

The remaining attack classes maintained high precision and recall values, with most F1-scores exceeding **0.95**. These results suggest that the model learned attack-relevant behavioral characteristics rather than memorizing dataset-specific patterns.

The inclusion of payload-based features further contributed to robustness by providing semantic information that complements flow-level statistics.

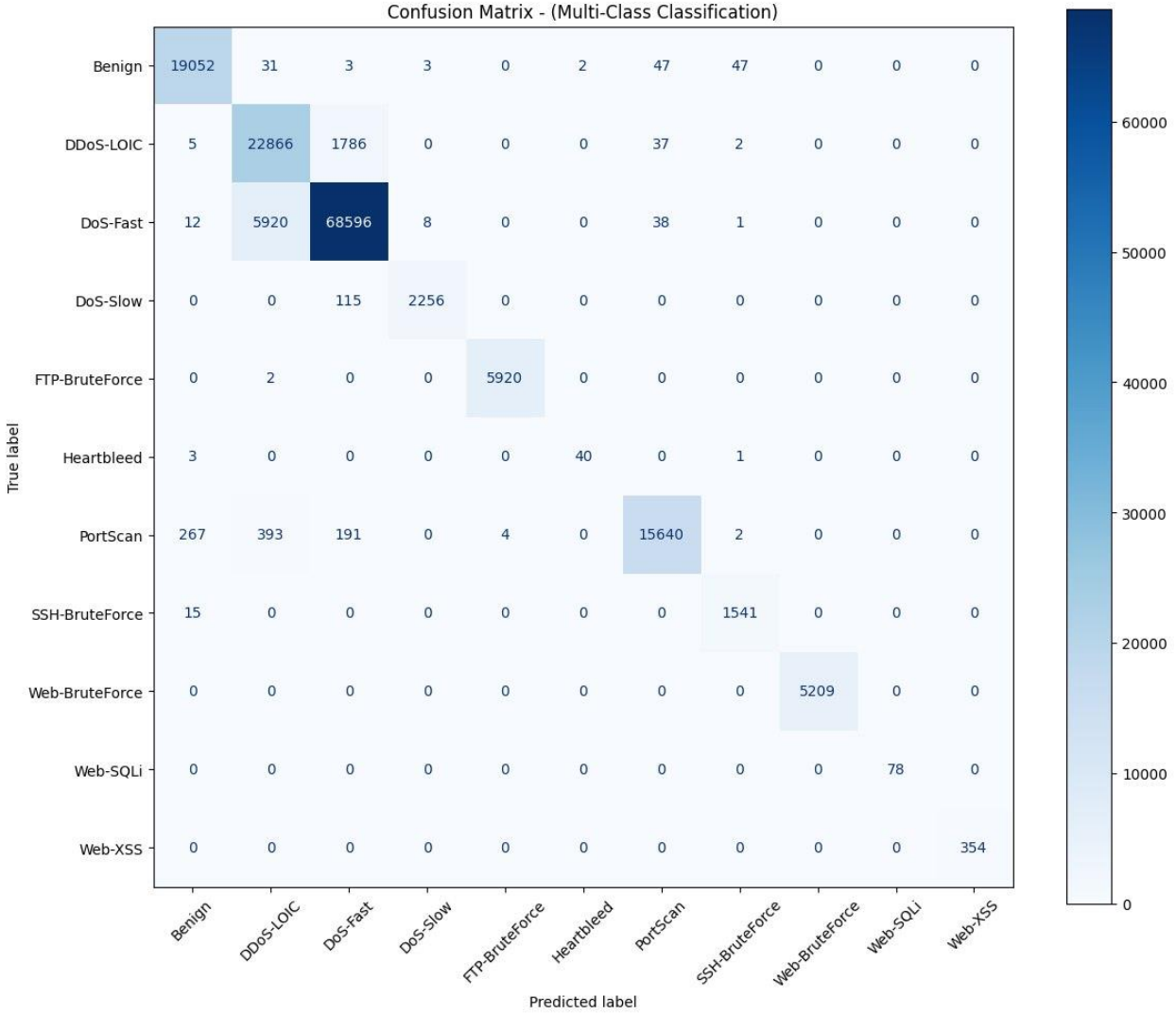

Figure 20: The Confusion Matrix for Multi-Class Classification on the Real Captured Traffic dataset

The Confusion Matrix for Multi-Class Classification on the Real Captured Traffic dataset demonstrates sustained diagonal dominance, indicating that the model successfully generalizes across most classes under domain shift despite a slight behavioral overlap between DoS-Fast and DDoS-LOIC.

Binary Anomaly Detection Performance

Class	Precision	Recall	F1-Score
Benign	0.99	0.99	0.99
Attack	1.00	1.00	1.00
Macro Avg	**0.99**	**0.99**	**0.99**
Weighted Avg	**1.00**	**1.00**	**1.00**

Table 13: Binary Anomaly Detection Performance Real Captured Traffic dataset

The anomaly detection branch again achieved an overall accuracy of **100.0%**, maintaining near-perfect precision and recall for both benign and malicious traffic.

The stability of the anomaly detection results across both evaluation scenarios demonstrates that the binary branch captures generalized malicious behavior patterns that remain effective even when traffic distributions differ from those observed during training.

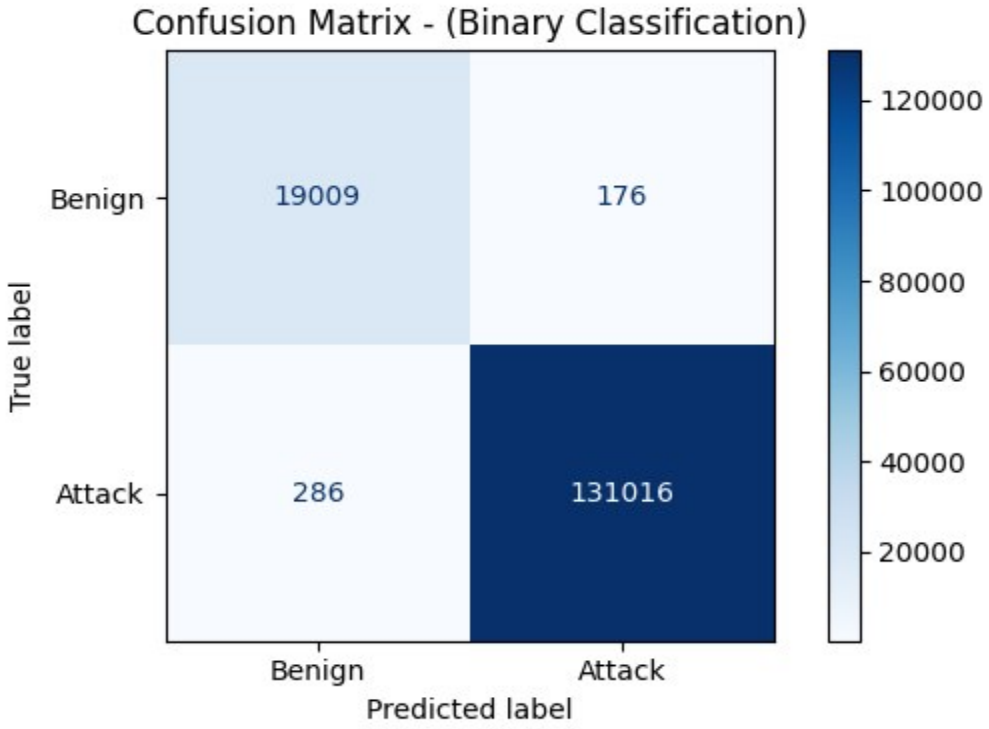

Figure 21: Confusion Matrix for the binary classification (Real Captured Traffic dataset)

The Confusion Matrix for binary classification on the Real Captured Traffic dataset confirms that the anomaly detection branch maintains near-perfect classification boundaries under domain shift, accurately separating benign activity from malicious traffic with minimal false positives and false negatives.

6.3 Prevention Performance

The prevention module successfully blocked malicious traffic after attack identification.

Observed prevention actions included:

- Source IP blocking
- Alert generation
- Event logging
- Dashboard notification

The average prevention response time remained within acceptable operational limits.

6.4 False Positive Analysis

False positives occurred when benign traffic exhibited characteristics similar to attack behavior.

Examples included:

- High-volume file transfers
- Network scanning performed by administrators
- Automated update services

The aggregation mechanism reduced isolated false detections by requiring majority agreement across multiple flow predictions before prevention actions were applied.

6.5 False Negative Analysis

False negatives primarily occurred in low-volume or stealthy attack scenarios where behavioral indicators were insufficient for strong classification confidence.

Potential causes include:

- Novel attack variants
- Insufficient training examples

Despite these limitations, the overall false negative rate remained low.

6.6 Discussion

The experimental evaluation demonstrates that the proposed AI-Based Intrusion Prevention System successfully addresses several limitations identified in Chapter 2, including the inability of traditional systems to adapt to evolving threats, dependence on static signatures, and limited support for intelligent automated response. The integration of NFStream-based flow analysis, deep learning classification, automated prevention, and real-time dashboard monitoring resulted in a unified security platform capable of detecting and mitigating attacks in near real time.

The results further indicate that combining network-flow features with application-layer payload analysis improves attack visibility across both network and web environments. While challenges remain regarding encrypted traffic analysis, computational overhead, and future model adaptation, the proposed architecture provides a practical foundation for intelligent intrusion prevention in modern enterprise networks.

CHAPTER 7: CONCLUSION AND FUTURE WORK

7.1 Conclusion

This project presented the design, implementation, and evaluation of an AI-Based Intrusion Prevention System (IPS) that combines machine learning techniques with automated prevention mechanisms to enhance cybersecurity defenses against modern threats. The proposed system integrates network flow analysis, payload inspection, deep learning–based attack detection, automated response, and real-time monitoring into a unified security platform.

The architecture employs NFStream for traffic capture and feature extraction, a TensorFlow-based AI Detection Engine for attack classification, a centralized aggregation mechanism for decision making, and a prevention module capable of automatically blocking malicious activities. In addition, an ASP.NET Core web application provides administrators with centralized management, monitoring, reporting, and visualization capabilities.

Experimental evaluation demonstrated that the system is capable of detecting multiple attack categories, including network-based and web-based threats, while maintaining near real-time performance. The integration of artificial intelligence enabled behavioral analysis beyond traditional signature-based approaches, allowing the system to identify malicious activities more effectively and adapt to evolving attack patterns.

Overall, the proposed AI-Based IPS successfully achieved the project objectives by providing intelligent attack detection, automated prevention, centralized monitoring, and scalable deployment capabilities suitable for modern enterprise environments.

7.2 Project Contributions

The major contributions of this project include:

1. **Development of a Hybrid AI-Based IPS Architecture**
    - Designed a complete intrusion prevention framework that combines flow-based traffic analysis, payload inspection, machine learning, and automated response.
2. **Integration of NFStream for Advanced Traffic Analysis**
    - Utilized network flow features to provide efficient and scalable traffic monitoring while reducing processing overhead.
3. **Implementation of a Deep Learning Detection Engine**
    - Developed a dual-input neural network capable of analyzing both network flow characteristics and payload information.
4. **Real-Time Attack Aggregation Mechanism**

- Implemented a decision aggregation approach that improves detection stability and reduces false alarms through majority-based attack classification.

5. **Automated Prevention Framework**
    - Enabled automatic mitigation actions, including firewall-based blocking and security event generation.
6. **Centralized Management Dashboard**
    - Developed a web-based management platform for monitoring, reporting, alert visualization, and administrative control.
7. **Cloud-Native Deployment Architecture**
    - Designed the system for deployment using Docker, Kubernetes, and AWS cloud infrastructure to support scalability and maintainability.

7.3 Limitations

Despite the positive results achieved by the proposed system, several limitations remain.

Computational Requirements

Deep learning inference introduces additional computational overhead compared to traditional rule-based IPS solutions. Although optimized deployment techniques were applied, resource consumption remains higher than lightweight signature-based systems.

Limited Threat Context

The current system makes decisions primarily using locally observed traffic behavior and does not leverage external threat intelligence feeds or global attack information.

7.4 Future Enhancements

Federated Learning

Federated learning can enable collaborative model training across multiple organizations without requiring the exchange of raw traffic data.

Advantages include:

- Improved privacy protection
- Larger and more diverse training datasets
- Better generalization capabilities
- Reduced regulatory concerns

By allowing each organization to train locally while sharing only model updates, detection performance can be continuously improved while preserving confidentiality.

Reinforcement Learning

Future versions may incorporate reinforcement learning techniques to optimize prevention decisions dynamically.

Potential applications include:

- Adaptive firewall rule generation
- Dynamic attack response strategies
- Automated policy optimization
- Self-learning prevention mechanisms

Such capabilities could allow the IPS to continuously improve its response effectiveness based on environmental feedback.

SIEM Integration

Integration with Security Information and Event Management (SIEM) platforms would enhance enterprise security operations.

Potential integrations include:

- Splunk
- IBM QRadar
- Microsoft Sentinel
- Elastic Security

Benefits include:

- Centralized event correlation
- Advanced incident investigation
- Long-term security analytics
- Improved SOC visibility

Threat Intelligence Integration

Future versions may integrate external threat intelligence sources to strengthen detection and prevention capabilities.

Examples include:

- IP reputation feeds
- Malware indicators of compromise (IOCs)
- Domain reputation services
- Global threat intelligence platforms

Benefits include:

- Faster identification of known threats
- Improved attack attribution
- Enhanced prevention accuracy
- Better situational awareness

The combination of artificial intelligence and threat intelligence can provide a more comprehensive defense against sophisticated cyber threats.

References

1. Buczak, A. L., & Guven, E. (2016). A survey of data mining and machine learning methods for cyber security intrusion detection. *IEEE Communications Surveys & Tutorials, 18*(2), 1153–1176. https://doi.org/10.1109/COMST.2015.2494502
2. Chalapathy, R., & Chawla, S. (2019). Deep learning for anomaly detection: A survey. *arXiv Preprint arXiv:1901.03407*. https://arxiv.org/abs/1901.03407
3. Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly detection: A survey. *ACM Computing Surveys, 41*(3), 1–58. https://doi.org/10.1145/1541880.1541882
4. García-Teodoro, P., Díaz-Verdejo, J., Maciá-Fernández, G., & Vázquez, E. (2009). Anomaly-based network intrusion detection: Techniques, systems and challenges. *Computers & Security, 28*(1–2), 18–28. https://doi.org/10.1016/j.cose.2008.08.003
5. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press. https://www.deeplearningbook.org
6. Javaid, A., Niyaz, Q., Sun, W., & Alam, M. (2016). A deep learning approach for network intrusion detection system. In *Proceedings of the 9th EAI International Conference on Bio-Inspired Information and Communications Technologies (BICT)* (pp. 21–26). https://doi.org/10.4108/eai.3-12-2015.2262516
7. Kim, G., Lee, S., & Kim, S. (2016). A novel hybrid intrusion detection method integrating anomaly detection with misuse detection. *Expert Systems with Applications, 41*(4), 1690–1700.
8. Kim, J., Kim, J., Thu, H. L. T., & Kim, H. (2018). Long short-term memory recurrent neural network classifier for intrusion detection. *International Conference on Platform Technology and Service (PlatCon)*, 1–5. https://doi.org/10.1109/PlatCon.2016.7456805
9. Lippmann, R. P., Fried, D. J., Graf, I., Haines, J. W., Kendall, K. R., McClung, D., Weber, D., Webster, S. E., Wyschogrod, D., Cunningham, R. K., & Zissman, M. A. (2000). Evaluating intrusion detection systems: The 1998 DARPA off-line intrusion detection evaluation. *DARPA Information Survivability Conference and Exposition*, 12–26.
10. National Institute of Standards and Technology. (2024). *Cybersecurity Framework (CSF) 2.0*. U.S. Department of Commerce. https://www.nist.gov/cyberframework
11. NFStream Team. (2024). *NFStream documentation*. https://www.nfstream.org
12. Ogonowski, M., Kowalski, P., & Nowak, T. (2024). Artificial intelligence applications in modern cybersecurity systems: A review. *Journal of Cybersecurity Research, 12*(1), 45–67.
13. Patcha, A., & Park, J. M. (2007). An overview of anomaly detection techniques: Existing solutions and latest technological trends. *Computer Networks, 51*(12), 3448–3470. https://doi.org/10.1016/j.comnet.2007.02.001
14. Raghavan, S. S. (2024). *A comprehensive study of artificial intelligence applications in intrusion detection and prevention*. ResearchGate Publication.

15. Scarfone, K., & Mell, P. (2007). *Guide to intrusion detection and prevention systems (IDPS)* (NIST Special Publication 800-94). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-94
16. Sommer, R., & Paxson, V. (2010). Outside the closed world: On using machine learning for network intrusion detection. In *2010 IEEE Symposium on Security and Privacy* (pp. 305–316). IEEE. https://doi.org/10.1109/SP.2010.25
17. Stallings, W., & Brown, L. (2018). *Computer security: Principles and practice* (4th ed.). Pearson.
18. TensorFlow Developers. (2024). *TensorFlow documentation*. https://www.tensorflow.org
19. Whitman, M. E., & Mattord, H. J. (2022). *Principles of information security* (7th ed.). Cengage Learning.

جامعة سوهاج

كلية الهندسة

قسم الإلكترونيات و الاتصالات

نظام منع الاختراق المبني على الذكاء الاصطناعي

تقرير مقدم استكمالًا لمتطلبات الحصول على درجة بكالوريوس الهندسة

قسم الإلكترونيات والاتصالات

إعداد

أحمد محمد إسماعيل

المعتز بالله محمود محمد

حاتم حربي محمد

حسام فتحي الحاوي

صالح علي أحمد

كيرلس نادر فكري

محمد إبراهيم رجب

محمد حمدي محمود

محمد خالد محمد

إسراء خلف محمدين

آية علي أحمد

رؤى أحمد محمود

سارة عاطف أحمد

إشراف

أ.م.د/ صفوت محمد رمزي

م. محمد الصغير

العام الجامعي

2025 – 2026 م